



Universidad de Buenos Aires
Facultad de Ingeniería

Reconstrucción de Superficies de Volatilidad Mediante Deep Learning Bajo Restricciones de No Arbitraje

Pablo Ariel Rodriguez

Tesis de Ingeniería Electrónica

Director: Ing. Ricardo Veiga

Co-director: Ing. Alberto Dams

A mi abuela, la persona más inteligente que conocí.

Esta tesis también es suya.

Agradecimientos

Esta tesis es el resultado de un largo camino que no habría sido posible sin la ayuda y el apoyo de muchas personas que estuvieron a mi lado a lo largo de estos años.

Quiero agradecer primero a mis directores de tesis, el Ing. Ricardo Veiga y el Ing. Alberto Dams, no solo por la guía durante el proyecto, sino principalmente por haber sido ellos quienes me contactaron y motivaron a completar este trabajo cuando incluso yo ya no lo creía posible.

A la universidad pública y a todos los profesores de esta facultad que siempre han demostrado vocación, esfuerzo y un fuerte compromiso con la formación de ingenieros al servicio de este país.

A mis amigos, compañeros, colegas y a toda la gente que ha estado a mi lado durante estos años y que han sido imprescindibles en este trayecto.

Por último, agradecer a mi familia. A mis padres y hermana, por mostrarme el camino y por siempre creer en mí, incluso en los momentos en los que yo dejé de hacerlo. Y a mi abuela, quien me enseñó muchos de mis valores y quien no pudo ver este resultado. Me queda la tranquilidad de saber que el tiempo es solo una mera ilusión.

Resumen

Las superficies de volatilidad implícita son fundamentales para la valuación de instrumentos financieros derivados y la gestión de riesgo. Sin embargo, en la práctica suelen estar incompletas debido a la baja liquidez en ciertas regiones de strike y vencimiento. En esta tesis, el problema de reconstruir una superficie de volatilidad completa a partir de observaciones parciales se plantea como un problema de reconstrucción de una señal bidimensional, y se realiza una comparación sistemática de seis arquitecturas de deep learning (MLP, CNN, U-Net, Transformer y dos variantes de autoencoders variacionales) frente a la parametrización Stochastic Volatility Inspired (SVI), estándar en la industria.

Todos los modelos neuronales se ajustan para tener una cantidad similar de parámetros para aislar el efecto de las propiedades estructurales de cada arquitectura. Se propone una arquitectura Transformer codificador-decodificador con codificación posicional de Fourier basada en coordenadas. Los resultados muestran que el Transformer y la U-Net alcanzan la mayor exactitud de reconstrucción en condiciones estándar, pero el Transformer se destaca por su robustez cuando faltan datos en regiones específicas de la superficie y en escenarios de escasez extrema.

Las condiciones de no arbitraje de calendar spread y convexidad butterfly se formulan como términos de penalización diferenciables dentro de la función de costo, y se analiza la relación entre exactitud y arbitraje para distintos niveles de penalización. En datos sintéticos, uno de los hallazgos centrales es que, para las arquitecturas convolucionales, estas restricciones actúan como una forma de regularización que mejora simultáneamente la exactitud de reconstrucción y reduce las violaciones de arbitraje, en lugar de generar un trade-off entre ambos objetivos.

Todos los modelos se validan sobre 3.900 superficies reales de opciones sobre SPY entre 2008 y 2025. El análisis de transfer learning muestra que el preentrenamiento sobre datos sintéticos mejora la exactitud del Transformer, pero no la de los modelos convolucionales, mientras que reduce de manera consistente las tasas de violación de arbitraje en todas las arquitecturas, lo que sugiere que las propiedades estructurales aprendidas a partir de datos sintéticos se transfieren de manera efectiva a mercados reales.

El código fuente se encuentra disponible en <https://github.com/pablisho/vol-surface-reconstruction>.

Índice general

Agradecimientos	2
Resumen	3
1. Introducción	8
1.1. Motivación	8
1.2. Planteamiento del Problema	9
1.3. Trabajo Relacionado	10
1.4. Contribuciones	13
1.5. Estructura de la Tesis	14
2. Fundamentos Financieros y Matemáticos	15
2.1. Fundamentos de Valuación de Opciones	15
2.2. El Modelo de Black–Scholes–Merton	16
2.3. Volatilidad Implícita	17
2.4. La Superficie de Volatilidad	18
2.5. Volatilidad Estocástica: El Modelo de Heston	19
2.6. Condiciones de No Arbitraje	20
2.7. Modelos Paramétricos de Superficie: SVI	21
3. Fundamentos de <i>Deep Learning</i>	23
3.1. Aprendizaje Supervisado y Funciones de Costo	23
3.2. Perceptrones Multicapa	24
3.3. Redes Neuronales Convolucionales	25
3.4. La Arquitectura U-Net	26
3.5. La Arquitectura Transformer	27
3.6. Autoencoders Variacionales	29
3.7. Técnicas de Entrenamiento	30
3.8. Predicción Enmascarada e <i>Inpainting</i>	31
4. Datos y Representación de Superficies	33
4.1. Representación de la Superficie	33
4.2. Generación de Datos Sintéticos	34
4.3. Datos Reales de Mercado	37
4.4. Superficies Sintéticas vs Reales	39
4.5. Estrategia de Enmascaramiento	40
4.6. Pipeline del Conjunto de Datos	42

5. Metodología	43
5.1. Formulación del Problema	43
5.2. Arquitecturas de Modelos	44
5.3. Normalización de la Cantidad de Parámetros	55
5.4. Protocolo de Entrenamiento	55
5.5. Restricciones de No Arbitraje	57
5.6. Métricas de Evaluación	58
6. Experimentos y Resultados	61
6.1. Resultados de Referencia sobre Datos Sintéticos	61
6.2. Robustez ante Escasez de Datos	66
6.3. Análisis Regional de Errores	69
6.4. Resultados sobre Datos Reales	71
6.5. Análisis de <i>Transfer Learning</i>	73
6.6. Análisis de Atención del Transformer	75
6.7. Eficiencia Computacional	76
6.8. Estudio de Ablación del Transformer	77
7. Análisis de No Arbitraje	79
7.1. El Problema de Arbitraje en las Reconstrucciones Neuronales	79
7.2. Metodología de Restricciones	80
7.3. Impacto de las Restricciones a Lambda Fijo	80
7.4. Barrido de Lambda y Fronteras de Pareto	82
7.5. Modelos con Restricciones sobre Datos Reales	84
7.6. Discusión	85
8. Conclusión	87
8.1. Resumen de Contribuciones	87
8.2. Hallazgos Principales y Discusión	88
8.3. Limitaciones	90
8.4. Trabajo Futuro	91

Índice de figuras

1.1. Superficie de volatilidad implícita de Heston	9
1.2. Ilustración de la tarea de reconstrucción	10
4.1. Distribución de parámetros de Heston	35
4.2. Superficies de Heston: smile simétrico y skew de renta variable	36
4.3. Sensibilidad a los parámetros de Heston	36
4.4. Distribución de datos faltantes y superficies por año en SPY	39
4.5. Comparación de superficie sintética y real	39
4.6. Distribuciones de volatilidad implícita: sintético vs. real	40
4.7. Ilustración del enmascaramiento	41
5.1. Arquitectura MLP	45
5.2. Arquitectura CNN	46
5.3. Arquitectura U-Net	48
5.4. Arquitectura Transformer	51
5.5. Arquitecturas VAE (FC y Conv)	54
6.1. RMSE por modelo en datos sintéticos	62
6.2. Reconstrucción de una superficie de test representativa	63
6.3. Cortes de volatilidad implícita en tres vencimientos	64
6.4. Distribución de RMSE por superficie	65
6.5. Degradación del RMSE vs. fracción de datos faltantes	66
6.6. Enmascaramiento aleatorio vs. estructurado	68
6.7. Comparación regional del RMSE	69
6.8. Mapas de error absoluto medio por punto de grilla	70
6.9. RMSE en datos reales de SPY	72
6.10. Diagrama de <i>transfer learning</i>	73
6.11. Pesos de atención cruzada del Transformer	75
7.1. Impacto de las restricciones en las seis arquitecturas	81
7.2. Barrido de λ : RMSE, violaciones y severidad	83

Índice de tablas

4.1. Rangos de parámetros de Heston	34
4.2. Partición del conjunto de datos sintético	37
4.3. Partición del conjunto de datos real de SPY	38
5.1. Cantidad de parámetros por modelo	55
5.2. Hiperparámetros de entrenamiento	56
5.3. Hiperparámetros de ajuste fino	56
5.4. Regiones de moneyness y tenor	59
6.1. Rendimiento de referencia sobre datos sintéticos	61
6.2. Validación con múltiples semillas	62
6.3. RMSE por región de moneyness y tenor	69
6.4. Rendimiento sobre datos reales de SPY	71
6.5. Rendimiento computacional	76
6.6. Ablación: Fourier vs. embeddings aprendibles	77
7.1. Impacto de las restricciones a $\lambda = 0,1$	81
7.2. Restricciones sobre datos reales de SPY	85

Capítulo 1

Introducción

Este capítulo presenta la motivación del trabajo: las superficies de volatilidad son esenciales para la valuación de instrumentos financieros derivados y la gestión de riesgo, pero en la práctica suelen estar incompletas. Se formula el problema central como la reconstrucción de una señal bidimensional a partir de observaciones parciales. Luego se revisan los enfoques previos y se posicionan las contribuciones de esta tesis respecto del trabajo existente. Finalmente, se describe la estructura de los capítulos restantes.

1.1. Motivación

Las superficies de volatilidad son centrales para la valuación de instrumentos financieros derivados y la gestión de riesgo. Surgen de la observación de que las volatilidades implícitas varían según el strike y el vencimiento, un fenómeno que contradice el supuesto de volatilidad constante del modelo de Black-Scholes [8, 30]. En la industria financiera se destinan recursos significativos a la construcción de superficies de volatilidad actualizadas, ya que una valuación exacta depende de contar con una superficie completa para un activo dado en cada momento.

En la práctica, las superficies de volatilidad observadas en el mercado son tanto ruidosas como incompletas, particularmente en los extremos de la superficie (strikes lejanos al precio actual) y en los vencimientos de corto plazo, donde la liquidez es baja. Sin embargo, para que una superficie de volatilidad sea útil, debe ser completa, suave y libre de arbitraje, es decir, debe satisfacer ciertas restricciones estructurales impuestas por la ausencia de oportunidades de ganancia sin riesgo.

En esta tesis planteamos el problema de obtener una superficie de volatilidad completa a partir de observaciones incompletas como la reconstrucción de una señal bidimensional a partir de entradas parciales. Esta formulación es análoga al inpainting de imágenes, donde se rellenan las regiones faltantes de una imagen explotando la estructura espacial, y a la reconstrucción de imágenes de resonancia magnética, donde se recuperan imágenes completas a partir de mediciones submuestreadas aprovechando la regularidad anatómica.

La industria ha recurrido tradicionalmente a métodos paramétricos para construir superficies de volatilidad, siendo la parametrización Stochastic Volatility Inspired (SVI) [17] la más ampliamente adoptada. Más recientemente, se han explorado técnicas de aprendizaje automático para este problema y otros relacionados. Los métodos paramétricos pueden imponer superficies libres de arbitraje por construcción, pero a costa de una exactitud de reconstrucción limitada, particularmente cuando los datos observados son escasos. Los métodos de aprendizaje automático han demostrado mejor exactitud de reconstrucción,

Skew ($\rho = -0.7, \xi = 0.4$)

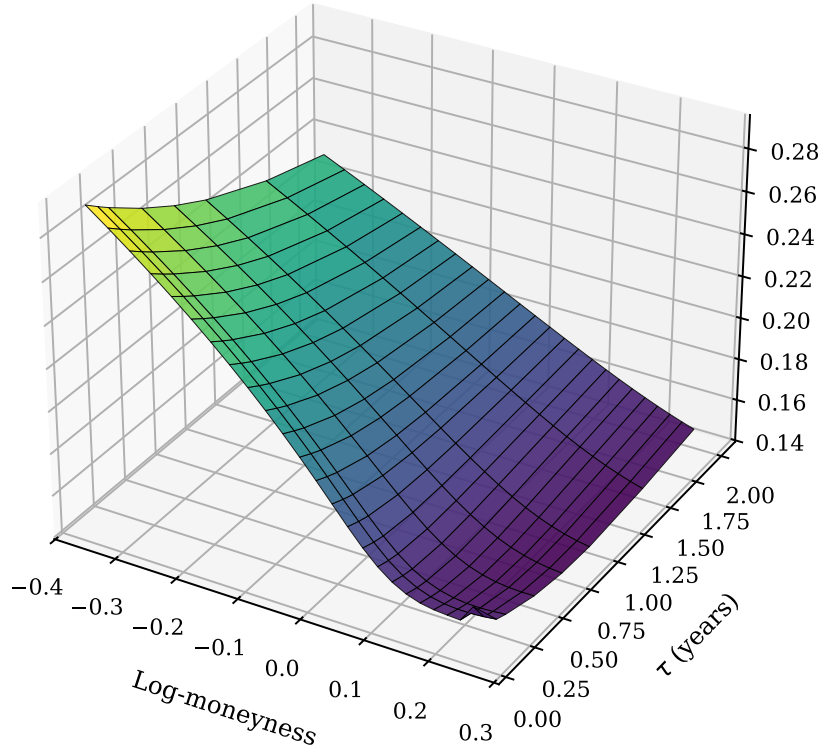


Figura 1.1: Ejemplo de superficie de volatilidad implícita generada con el modelo de volatilidad estocástica de Heston. La superficie exhibe el sesgo (smirk) característico a lo largo de los strikes y una estructura temporal a lo largo de los vencimientos.

pero pueden introducir violaciones de arbitraje no deseadas. Además, el campo del *deep learning* ha avanzado rápidamente, con nuevas arquitecturas y procedimientos de entrenamiento que ofrecen oportunidades aún no evaluadas de forma sistemática para esta tarea.

A pesar de estos avances, ningún trabajo previo ha comparado sistemáticamente arquitecturas de *deep learning* para la reconstrucción de superficies de volatilidad bajo condiciones controladas, ni ha estudiado el efecto de incorporar restricciones de no arbitraje directamente en la función objetivo de entrenamiento. En particular, la arquitectura Transformer [39], que ha impulsado resultados de vanguardia en procesamiento de lenguaje natural y *computer vision*, no ha sido aplicada a este problema. Esta tesis busca cubrir esas brechas.

1.2. Planteamiento del Problema

El problema central que aborda esta tesis es la reconstrucción de una superficie de volatilidad implícita completa a partir de una observación parcial. Formalmente, la entrada consiste en un tensor de dos canales: una grilla de 8×25 valores de volatilidad implícita (organizada en 8 vencimientos y 25 strikes en log-moneyness) junto con una máscara binaria que indica qué puntos fueron observados. La salida es una superficie completa de 8×25 valores de volatilidad implícita.

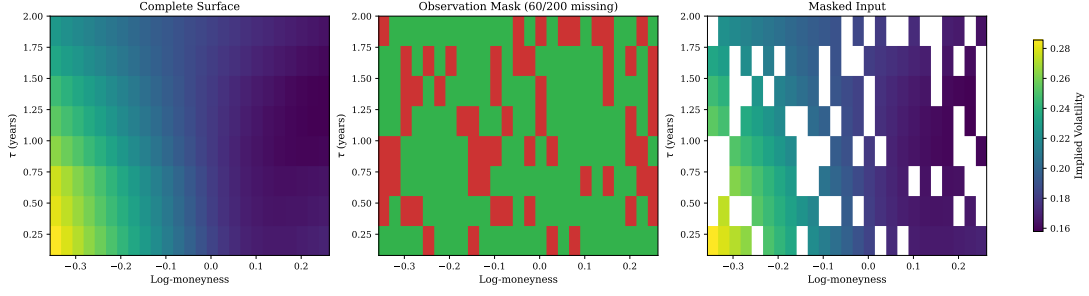


Figura 1.2: Ilustración de la tarea de reconstrucción. Izquierda: una superficie de volatilidad completa. Derecha: la misma superficie con el 30 % de los puntos enmascarados (faltantes). El objetivo es recuperar la superficie completa a partir de la observación parcial.

El objetivo principal es minimizar el error de reconstrucción sobre los puntos faltantes, medido por la raíz del error cuadrático medio (RMSE). Más allá de la exactitud, también se investiga si las superficies reconstruidas pueden satisfacer condiciones de no arbitraje: la varianza implícita total debe ser no decreciente en el vencimiento (condición de calendar spread) y la varianza implícita debe ser convexa en log-moneyness para cada vencimiento (condición de butterfly). Primero se evalúan todas las arquitecturas en exactitud de reconstrucción (Capítulo 6), y luego se estudia el efecto de incorporar estas restricciones como penalizaciones diferenciables en la función de costo (Capítulo 7).

Esta formulación establece una conexión directa con tareas de predicción enmascarada en otros dominios. En *computer vision*, los autoencoders enmascarados [21] reconstruyen imágenes a partir de parches aleatoriamente enmascarados aprendiendo la estructura espacial. En datos tabulares, se han aplicado enfoques similares para imputar valores faltantes [14]. La reconstrucción de superficies de volatilidad se sitúa entre estos dos escenarios: los datos se disponen sobre una grilla 2D regular (como una imagen) pero poseen restricciones estructurales específicas del dominio que los métodos genéricos de inpainting no imponen.

1.3. Trabajo Relacionado

1.3.1. Ajuste Paramétrico de Superficies de Volatilidad

El enfoque más utilizado para construir superficies de volatilidad es la parametrización Stochastic Volatility Inspired (SVI) introducida por Gatheral y Jacquier [17], que ajusta una función de cinco parámetros a cada corte por vencimiento de forma independiente. La fórmula SVI expresa la varianza implícita total como función del log-moneyness utilizando cinco parámetros que controlan el nivel, la pendiente, la curvatura y la inclinación de cada curva de volatilidad. Extensiones como la parametrización Surface SVI (SSVI) imponen consistencia entre cortes restringiendo los parámetros a variar suavemente entre vencimientos, reduciendo el riesgo de arbitraje de calendar spread.

Estos métodos son interpretables y, cuando se los restringe adecuadamente, pueden garantizar superficies libres de arbitraje de forma analítica. Sin embargo, operan sobre cada corte de vencimiento de manera aislada: la información de un vencimiento no puede informar el ajuste en otro. Esta independencia se convierte en una limitación crítica cuando los datos observados son escasos, donde cortes enteros pueden tener muy pocos

puntos observados.

1.3.2. Redes Neuronales para Superficies de Volatilidad

Varios trabajos recientes han aplicado *deep learning* al modelado de volatilidad. Bayer y Stemper [4] utilizaron redes neuronales para la calibración rápida de modelos de volatilidad estocástica, entrenando redes para mapear precios de opciones observados a parámetros del modelo. Este enfoque acelera la calibración pero no aborda la reconstrucción de superficies a partir de datos incompletos. Horvath et al. [25] exploraron el uso de redes neuronales profundas para la valuación y calibración de modelos de volatilidad estocástica, aprendiendo representaciones continuas de volatilidad implícita. Ackerer et al. [1] aplicaron aprendizaje profundo para suavizar precios de opciones directamente, usando redes neuronales como interpoladores flexibles de la función de precios. Más recientemente, Zhang et al. [40] introdujeron VolNP, un proceso neuronal basado en atención para el ajuste de volatilidad implícita que utiliza meta-learning para adaptarse a nuevas superficies con pocas observaciones.

La línea de trabajo más estrechamente relacionada utiliza autoencoders variacionales (VAEs) para el modelado de superficies de volatilidad. Bergeron et al. [5] propusieron un enfoque basado en VAE en el que el modelo primero aprende una representación latente de superficies de volatilidad completas, y luego reconstruye superficies incompletas optimizando el vector latente para que coincida con los puntos observados durante la inferencia. Ning et al. [32] extendieron este enfoque para imponer superficies libres de arbitraje incorporando restricciones suaves en el entrenamiento del VAE, demostrando que el marco generativo puede producir superficies consistentes con las condiciones de no arbitraje. Nteumagné et al. [33] evaluaron la reconstrucción de superficies basada en VAE, comparando contra interpolación por thin-plate spline, SABR, SVI y autoencoders determinísticos, y reportaron un buen desempeño incluso con muy pocos datos observados.

Estos enfoques basados en VAE abordan directamente la tarea de reconstrucción de superficies, pero se enfocan exclusivamente en la arquitectura VAE. No existe un trabajo previo que presente una comparación controlada entre múltiples familias de arquitecturas de *deep learning* para esta tarea, ni que evalúe arquitecturas no generativas como CNNs, U-Nets o Transformers bajo las mismas condiciones.

1.3.3. Relación con el Trabajo Previo

En comparación con el trabajo previo más estrechamente relacionado, esta tesis se diferencia en tres aspectos clave. Primero, mientras que los estudios existentes evalúan una única familia de arquitecturas (típicamente VAEs [5, 32, 33] o procesos neuronales basados en atención [40]), se realiza una comparación controlada entre seis arquitecturas neuronales y SVI, todas igualadas en cantidad aproximada de parámetros, aislando el efecto del diseño de cada arquitectura en lugar de la capacidad del modelo. Segundo, integramos penalizaciones diferenciables de no arbitraje directamente en la función de costo y estudiamos sistemáticamente el compromiso entre exactitud y arbitraje mediante análisis de Pareto, en lugar de verificar el cumplimiento a posteriori o depender de garantías arquitectónicas. Tercero, evaluamos la robustez bajo patrones de faltantes tanto aleatorios como estructurados a lo largo de un amplio rango de porcentajes de datos faltantes (10 %–90 %), proporcionando un panorama más completo del desempeño práctico

que las evaluaciones con una única condición.

1.3.4. Inpainting de Imágenes y Predicción Enmascarada

El problema de reconstruir regiones faltantes a partir de observaciones parciales tiene una larga historia en *computer vision*. Los métodos clásicos de inpainting como el trabajo de Bertalmío et al. [6] propagan información desde el borde de las regiones conocidas hacia las áreas faltantes utilizando ecuaciones diferenciales parciales. Más recientemente, los enfoques basados en *deep learning* han logrado resultados notables: las redes convolucionales aprenden a rellenar grandes regiones faltantes aprovechando conocimiento previo aprendido sobre imágenes naturales.

Un desarrollo particularmente relevante es el marco de Masked Autoencoder (MAE) de He et al. [21], que entrena Vision Transformers [39] enmascarando aleatoriamente parches de una imagen y reconstruyendo el contenido faltante. Este enfoque autosupervisado aprende representaciones visuales ricas resolviendo una tarea de reconstrucción que es estructuralmente similar a la completación de superficies de volatilidad. Du et al. [14] extendieron este paradigma de autocodificación enmascarada a datos tabulares con ReMasker, demostrando que el enfoque se generaliza más allá de imágenes a datos estructurados con valores faltantes.

Sin embargo, estos métodos están diseñados para datos genéricos y no incorporan restricciones estructurales específicas del dominio. Las superficies de volatilidad requieren no solo plausibilidad visual sino exactitud cuantitativa y consistencia con las condiciones de no arbitraje, algo que los enfoques estándar de inpainting no garantizan.

1.3.5. Aprendizaje Informado por la Física y con Restricciones

La idea de incorporar conocimiento del dominio como penalizaciones diferenciables en la función de costo ha sido ampliamente estudiada en el ámbito de la simulación numérica y la resolución de ecuaciones diferenciales. El ejemplo más prominente son las Physics-Informed Neural Networks (PINNs) introducidas por Raissi et al. [35], que imponen restricciones de ecuaciones diferenciales parciales agregando términos de residuo a la función de costo. En lugar de requerir datos de entrenamiento que satisfagan la EDP, las PINNs aprenden soluciones que satisfacen aproximadamente la física mediante penalizaciones suaves, permitiendo que la red balancee la fidelidad a los datos contra la consistencia física.

El mismo paradigma puede aplicarse a restricciones financieras: las condiciones de no arbitraje como calendar spread y convexidad butterfly pueden expresarse como funciones diferenciables de la salida del modelo y agregarse a la función de costo con una intensidad de penalización ajustable. Esto permite un estudio controlado de la relación entre exactitud de reconstrucción y cumplimiento de las condiciones de arbitraje, un enfoque que no ha sido explorado sistemáticamente para la reconstrucción de superficies de volatilidad.

1.3.6. Aprendizaje por Transferencia en Finanzas

La transferencia de datos sintéticos a reales, frecuentemente denominada transferencia sim-to-real, es una técnica bien establecida en robótica y *computer vision*, donde los modelos se preentrenan con grandes cantidades de datos simulados antes de ajustarse con conjuntos de datos reales más pequeños. El entorno simulado proporciona datos

abundantes y perfectamente etiquetados, mientras que el ajuste fino cierra la brecha entre la simulación y la realidad.

Este enfoque ha sido menos explorado en finanzas cuantitativas, a pesar de la disponibilidad de simuladores realistas como los modelos de volatilidad estocástica calibrados. El modelo de Heston [23], por ejemplo, puede generar una cantidad ilimitada de superficies de volatilidad sintéticas con valores de referencia conocidos, lo que lo convierte en un candidato natural para el preentrenamiento. Si las propiedades estructurales aprendidas a partir de estos datos sintéticos se transfieren efectivamente a superficies de mercado reales es una pregunta que permanecía abierta.

1.4. Contribuciones

Las principales contribuciones de esta tesis son las siguientes:

1. Se propone una arquitectura Transformer codificador-decodificador con codificación posicional de Fourier basada en coordenadas [38] para la reconstrucción de superficies de volatilidad. Un estudio de ablación confirma una mejora significativa en exactitud (18 %) respecto de embeddings posicionales aprendibles, validando la elección de diseño. Hasta donde sabemos, esta es la primera aplicación de esta arquitectura a este problema específico.
2. Se realiza un estudio comparativo controlado de seis arquitecturas de *deep learning* (MLP, CNN, U-Net [36], Transformer [39], FC-VAE, Conv-VAE [28]) y el modelo de referencia paramétrico SVI [17], todas igualadas a aproximadamente 288.000 parámetros. Experimentos con múltiples semillas validan la estabilidad del ranking: el Transformer y la U-Net están estadísticamente empatados en exactitud de reconstrucción, con CNN y MLP claramente detrás.
3. Se integran penalizaciones diferenciables de no arbitraje (calendar spread y convexidad butterfly) en la función de costo y se realiza un análisis de Pareto completo para distintas intensidades de penalización. Se encuentra que, para las arquitecturas convolucionales, las restricciones actúan como regularizadores que mantienen o mejoran la exactitud de reconstrucción mientras reducen sustancialmente las violaciones de arbitraje.
4. Se evalúa la robustez bajo patrones de faltantes tanto aleatorios como estructurados (alas), variando la tasa de faltantes del 10 % al 90 % sin reentrenar. El Transformer logra $9\times$ menor error que SVI con 90 % de datos faltantes y resulta prácticamente inalterado por el enmascaramiento estructurado de alas (menos del 20 % de degradación), mientras que los modelos convolucionales sufren pérdidas de exactitud del 30–47 % cuando la información de las alas está ausente.
5. Se validan todos los modelos sobre datos reales de opciones sobre SPY que comprenden 3.900 superficies entre 2008 y 2025, y se analiza la transferencia de aprendizaje de datos sintéticos a reales. Se encuentra que el entrenamiento desde cero supera al ajuste fino para los modelos convolucionales en exactitud, mientras que los modelos ajustados exhiben menores tasas de arbitraje, lo que sugiere que el preentrenamiento sintético enseña representaciones de superficies más suaves.

1.5. Estructura de la Tesis

El resto de esta tesis se organiza de la siguiente manera.

El Capítulo 2 presenta los fundamentos financieros y matemáticos, incluyendo el modelo de Black-Scholes, la volatilidad implícita, la construcción de superficies de volatilidad, el modelo de volatilidad estocástica de Heston, las condiciones de no arbitraje y la parametrización SVI.

El Capítulo 3 introduce los fundamentos de *deep learning*, incluyendo las arquitecturas utilizadas en este trabajo (MLP, CNN, U-Net, Transformer, VAE), los procedimientos de entrenamiento y la conexión con la predicción enmascarada.

El Capítulo 4 describe los datos y la representación de superficies, incluyendo la discretización en grilla, el conjunto de datos sintético generado a partir del modelo de Heston, el conjunto de datos real construido a partir de opciones sobre SPY, y la estrategia de enmascaramiento utilizada para simular observaciones incompletas.

El Capítulo 5 detalla la metodología: la formulación formal del problema, las siete arquitecturas de modelos y su implementación, el protocolo de entrenamiento, la formulación de restricciones de no arbitraje y las métricas de evaluación.

El Capítulo 6 presenta los resultados experimentales, incluyendo las comparaciones de referencia sobre datos sintéticos, la robustez ante distintos niveles de datos faltantes, el análisis regional de errores, los experimentos con datos reales, la transferencia de aprendizaje, la visualización de atención y los benchmarks computacionales.

El Capítulo 7 presenta un análisis dedicado de las restricciones de no arbitraje, incluyendo el impacto de la intensidad de restricción, los experimentos de barrido de λ y las fronteras de Pareto entre exactitud y cumplimiento de arbitraje.

El Capítulo 8 concluye con un resumen de contribuciones, discusión de limitaciones y posibles líneas de trabajo futuro.

Todo el código fuente, incluyendo los modelos, los experimentos y los scripts de generación de datos, se encuentra disponible en <https://github.com/pablisho/vol-surface-reconstruction>.

Capítulo 2

Fundamentos Financieros y Matemáticos

Este capítulo introduce los conceptos financieros y matemáticos que sustentan la reconstrucción de superficies de volatilidad. Se comienza con los fundamentos de valuación de opciones, se introduce la volatilidad implícita y la superficie de volatilidad, se describe el modelo de volatilidad estocástica de Heston utilizado para la generación de datos sintéticos, se formalizan las condiciones de no arbitraje que restringen las superficies válidas, y se presenta la parametrización SVI utilizada como referencia.

2.1. Fundamentos de Valuación de Opciones

2.1.1. Opciones Europeas

Una opción europea es un contrato financiero que otorga a su tenedor el derecho, pero no la obligación, de comprar (call) o vender (put) un activo subyacente a un precio de ejercicio (*strike*) predeterminado K en una fecha de vencimiento específica T . Los pagos al vencimiento son:

$$\text{Pago del call} = \max(S_T - K, 0) \quad (2.1)$$

$$\text{Pago del put} = \max(K - S_T, 0) \quad (2.2)$$

donde S_T es el precio del activo subyacente al vencimiento. El tenedor ejerce la opción solo cuando resulta rentable, limitando la pérdida máxima a la prima pagada por el contrato.

2.1.2. Valuación Neutral al Riesgo

Bajo el principio de no arbitraje, el teorema fundamental de valuación de activos [20] garantiza la existencia de una medida de probabilidad $\mathbb{Q}$, llamada medida neutral al riesgo, bajo la cual los precios descontados de los activos son martingalas, es decir, su valor esperado futuro coincide con el valor actual, sin tendencia predecible. Esta medida no necesita construirse explícitamente: queda definida de forma implícita por los precios observados en el mercado. Bajo $\mathbb{Q}$, el precio de cualquier instrumento financiero derivado puede expresarse como el valor esperado descontado de su pago:

$$V_0 = e^{-r\tau} \mathbb{E}^{\mathbb{Q}}[\text{pago}(S_T)] \quad (2.3)$$

donde r es la tasa libre de riesgo y $\tau = T - t$ es el tiempo al vencimiento.

2.1.3. Paridad Put-Call

Los precios de calls y puts europeos sobre el mismo subyacente, strike y vencimiento están vinculados por la paridad put-call:

$$C - P = e^{-q\tau} S_0 - e^{-r\tau} K \quad (2.4)$$

donde q es el rendimiento por dividendos continuo. Esta identidad se deriva directamente del principio de no arbitraje: comprar una opción call y vender una opción put con el mismo strike y vencimiento replica un contrato forward. La paridad put-call implica que cualquier información contenida en los precios de los puts también está contenida en los precios de los calls, y viceversa. En la práctica, se utilizan opciones *out of the money* (OTM) para cada strike (puts para $K < S$, calls para $K \geq S$) porque son más líquidas y tienen spreads bid-ask más ajustados.

2.2. El Modelo de Black–Scholes–Merton

2.2.1. Supuestos del Modelo

El modelo de Black–Scholes–Merton (BSM) [8, 30] deriva una fórmula cerrada de valuación de opciones bajo los siguientes supuestos:

1. El activo subyacente sigue un movimiento browniano geométrico con volatilidad constante σ :
$$dS_t = (r - q)S_t dt + \sigma S_t dW_t \quad (2.5)$$
2. Los mercados no tienen fricciones (sin costos de transacción, negociación continua, venta en corto ilimitada).
3. La tasa libre de riesgo r y el rendimiento por dividendos q son constantes.

2.2.2. La Fórmula de Black–Scholes

Bajo estos supuestos, el precio de una opción call europea es:

$$C = e^{-q\tau} S_0 \Phi(d_1) - e^{-r\tau} K \Phi(d_2) \quad (2.6)$$

donde $\Phi(\cdot)$ es la función de distribución acumulada normal estándar y:

$$d_1 = \frac{\log(S_0/K) + (r - q + \sigma^2/2)\tau}{\sigma\sqrt{\tau}}, \quad d_2 = d_1 - \sigma\sqrt{\tau} \quad (2.7)$$

2.2.3. La Variante de Valuación Forward: Black-76

Cuando se valúan opciones sobre futuros o forwards, el subyacente es el precio forward $F = S_0 e^{(r-q)\tau}$ en lugar del precio spot. La fórmula de Black-76 [7] simplifica la fórmula de BSM absorbiendo el drift en el forward:

$$C = e^{-r\tau} [F \Phi(d_1) - K \Phi(d_2)] \quad (2.8)$$

con:

$$d_1 = \frac{\log(F/K) + \sigma^2\tau/2}{\sigma\sqrt{\tau}}, \quad d_2 = d_1 - \sigma\sqrt{\tau} \quad (2.9)$$

En esta tesis se utiliza la formulación de Black-76, ya que brinda una separación más clara entre el nivel del precio forward y la estructura de volatilidad. El motor de valuación implementa la Ecuación (2.8) utilizando la función de distribución normal estándar calculada mediante la función de error ($\Phi(x) = \frac{1}{2}[1 + \text{erf}(x/\sqrt{2})]$), sin dependencias externas.

2.2.4. Limitaciones

La limitación principal del modelo BSM es el supuesto de volatilidad constante. Si este supuesto fuera válido, un único parámetro de volatilidad σ valuaría correctamente todas las opciones sobre el mismo subyacente, independientemente del strike y el vencimiento. En la práctica, las volatilidades implícitas varían sistemáticamente entre strikes y vencimientos, produciendo el *volatility smile* y la superficie descritas en la Sección 2.4. Esta discrepancia motiva todo el campo del modelado de volatilidad estocástica y, por extensión, el problema de reconstrucción abordado en esta tesis.

2.3. Volatilidad Implícita

2.3.1. Definición

La volatilidad implícita (IV) σ_{IV} de una opción es el valor de σ que, al sustituirse en la fórmula de Black-76, reproduce el precio de mercado observado:

$$C_{\text{mercado}} = C_{\text{Black-76}}(F, K, \tau, r, \sigma_{IV}) \quad (2.10)$$

La volatilidad implícita no es directamente observable; debe recuperarse numéricamente invirtiendo la fórmula de valuación. Dado que $C_{\text{Black-76}}$ es estrictamente creciente en σ (la vega $\partial C / \partial \sigma > 0$ para todo $\sigma > 0$), la inversión tiene una solución única para cualquier precio de opción válido.

2.3.2. Inversión Numérica

Se implementan dos métodos para la inversión:

Bisección. Un método de bracketing que mantiene un intervalo $[\sigma_l, \sigma_u]$ que contiene la solución. En cada paso, se evalúa el punto medio y se reduce el intervalo a la mitad. La bisección converge de forma incondicional pero lenta (convergencia lineal). Se utiliza como respaldo para garantizar robustez.

Newton-Raphson. Utilizando la derivada del precio respecto de la volatilidad (conocida como vega en la terminología de opciones), Newton-Raphson converge cuadráticamente:

$$\sigma_{n+1} = \sigma_n - \frac{C_{\text{Black-76}}(\sigma_n) - C_{\text{mercado}}}{\text{vega}(\sigma_n)} \quad (2.11)$$

Se mantiene un intervalo de bisección como resguardo: si los pasos de Newton-Raphson caen fuera del intervalo o no convergen en 50 iteraciones, el solver recurre a bisección pura. La tolerancia de convergencia es 10^{-12} tanto en espacio de precios como de volatilidad.

2.3.3. El *Volatility Smile*

Si el supuesto de volatilidad constante del modelo BSM fuera válido, la volatilidad implícita sería la misma para todos los strikes en un vencimiento dado. En cambio, las observaciones empíricas muestran consistentemente que la volatilidad implícita varía con el strike, produciendo un patrón característico conocido como *volatility smile* (para formas simétricas) o *skew* (para formas asimétricas). En mercados de renta variable, el patrón típico es un skew: la volatilidad implícita aumenta para strikes bajos (puts fuera del dinero) y disminuye para strikes altos (calls fuera del dinero), reflejando la expectativa del mercado de movimientos más grandes a la baja.

2.4. La Superficie de Volatilidad

2.4.1. Definición de la Superficie

La superficie de volatilidad es la función bidimensional $\sigma_{IV}(K, \tau)$ que mapea el precio de ejercicio K y el tiempo al vencimiento τ a la volatilidad implícita. Codifica la visión colectiva del mercado sobre cómo varía la distribución de retornos del activo subyacente a lo largo de strikes y horizontes temporales. En lugar de utilizar precios de ejercicio crudos, es habitual expresar el eje de strikes en *log-moneyness*:

$$m = \log \left(\frac{K}{F} \right) \quad (2.12)$$

donde F es el precio forward. El log-moneyness centra la superficie en $m = 0$ (at-the-money, ATM), normaliza entre distintos niveles de precio del subyacente y produce formas de smile aproximadamente simétricas para modelos de volatilidad estocástica. En estas coordenadas, la superficie se expresa como $\sigma_{IV}(m, \tau)$, que es la representación utilizada en toda esta tesis.

2.4.2. Propiedades Empíricas

Las superficies de volatilidad en mercados de renta variable exhiben tres características:

1. **Smile/skew en strike.** Para cada vencimiento, la volatilidad implícita varía con el strike. La forma es típicamente un skew en renta variable (mayor volatilidad en strikes bajos) y un smile en mercados de divisas (mayor volatilidad en ambos extremos). El skew refleja la correlación negativa entre los retornos del activo y los cambios en volatilidad, conocida como *efecto de apalancamiento* [11].
2. **Estructura temporal.** La volatilidad implícita at-the-money varía con el vencimiento. En mercados calmos, la estructura temporal tiene pendiente positiva (opciones de mayor plazo tienen mayor IV, reflejando la acumulación de incertidumbre). Durante crisis, la estructura temporal se invierte cuando la IV de corto plazo sube abruptamente.
3. **Aplanamiento del smile.** El smile se vuelve menos pronunciado en vencimientos más largos. Esto es consistente con el teorema central del límite: a medida que el

horizonte temporal crece, la distribución de retornos converge hacia una gaussiana, y el supuesto de volatilidad constante del modelo BSM se vuelve una mejor aproximación.

2.4.3. Importancia Práctica

La superficie de volatilidad es central para la valuación de instrumentos financieros derivados y la gestión de riesgo:

- **Valuación.** Las opciones exóticas (barreras, asiáticas, entre otras) dependen de la superficie de volatilidad completa, no solo de una única volatilidad implícita. Se necesita una superficie completa y libre de arbitraje para calibrar los modelos de valuación utilizados para estos instrumentos.
- **Cobertura.** Las coberturas de delta y vega se calculan a partir de la superficie. Errores o huecos en la superficie se propagan directamente a errores de cobertura.
- **Gestión de riesgo.** El cálculo de valor en riesgo y los tests de estrés requieren superficies de volatilidad consistentes para todos los strikes y vencimientos. Puntos faltantes o inconsistentes pueden llevar a una subestimación del riesgo.

2.5. Volatilidad Estocástica: El Modelo de Heston

Las limitaciones del supuesto de volatilidad constante del modelo BSM motivan los modelos de volatilidad estocástica, que permiten que la volatilidad evolucione aleatoriamente en el tiempo. El modelo de Heston [23] es el modelo de referencia en esta clase.

2.5.1. Especificación del Modelo

Bajo el modelo de Heston, el precio del activo S_t y su varianza instantánea v_t siguen las ecuaciones diferenciales estocásticas acopladas:

$$dS_t = (r - q)S_t dt + \sqrt{v_t} S_t dW_t^S \quad (2.13)$$

$$dv_t = \kappa(\theta - v_t) dt + \xi \sqrt{v_t} dW_t^v \quad (2.14)$$

donde dW_t^S y dW_t^v son movimientos brownianos correlacionados con $\text{Corr}(dW_t^S, dW_t^v) = \rho dt$. Los cinco parámetros controlan aspectos distintos de la superficie:

- v_0 (varianza inicial): establece el nivel de volatilidad at-the-money actual ($\sigma_{\text{ATM}} \approx \sqrt{v_0}$).
- κ (velocidad de reversión a la media): controla cuán rápido la varianza retorna a su nivel de largo plazo. Un κ mayor produce un aplanamiento más rápido de la estructura temporal.
- θ (varianza de largo plazo): el nivel de varianza de equilibrio. Determina la volatilidad ATM de largo plazo ($\sigma_{\text{ATM}}^\infty \approx \sqrt{\theta}$).
- ξ (volatilidad de la volatilidad): controla la curvatura del smile. Un ξ mayor produce smiles más pronunciados en todos los vencimientos.

- ρ (correlación spot-volatilidad): controla la asimetría del skew. Un ρ negativo (típico en renta variable) produce el skew descendente característico; $\rho \approx 0$ produce un smile simétrico.

2.5.2. La Condición de Feller

El proceso de varianza v_t debe permanecer estrictamente positivo para que el modelo esté bien definido. La condición de Feller lo garantiza:

$$2\kappa\theta \geq \xi^2 \quad (2.15)$$

Cuando esta condición se cumple, el proceso de varianza nunca alcanza cero. Se la impone al generar datos sintéticos (Sección 4.2).

2.5.3. Valuación Semi-Analítica

Una ventaja clave del modelo de Heston es la disponibilidad de una fórmula de valuación semi-analítica. El precio de un call europeo se obtiene mediante la inversión de Fourier de Gil-Pelaez [18]:

$$C(K, \tau) = e^{-r\tau} [F \cdot P_1 - K \cdot P_2] \quad (2.16)$$

donde P_1 y P_2 se calculan a partir de integrales que involucran la función característica del logaritmo del precio. Se utiliza la formulación de Albrecher et al. [2], que evita la discontinuidad de rama conocida como el “Little Heston Trap” que afecta a implementaciones ingenuas de la función característica.

La función característica se evalúa mediante integración numérica con cuadratura adaptativa. Esto hace que el modelo de Heston sea computacionalmente tratable para generar grandes conjuntos de datos sintéticos: cada precio de opción requiere una única integral numérica unidimensional, que puede evaluarse con precisión de máquina en microsegundos.

2.6. Condiciones de No Arbitraje

Una superficie de volatilidad debe satisfacer ciertas restricciones para ser consistente con una distribución de probabilidad válida para el precio del activo subyacente. Las violaciones de estas restricciones crean *arbitraje estático*: oportunidades de ganancia sin riesgo que pueden explotarse utilizando portafolios de opciones estándar (*vanilla*) sin necesidad de cobertura dinámica.

2.6.1. Arbitraje de Calendar Spread

La varianza implícita total $w(m, \tau) = \sigma_{IV}^2(m, \tau) \cdot \tau$ debe ser no decreciente en el vencimiento para cada log-moneyness fijo:

$$\frac{\partial w}{\partial \tau}(m, \tau) \geq 0 \quad \text{para todo } m \quad (2.17)$$

Intuición. Si la varianza total decreciera con el vencimiento para algún strike, un operador podría vender la opción de menor plazo y comprar la de mayor plazo (un

calendar spread), cobrando una prima neta. La opción de menor plazo vencería primero, y la opción de mayor plazo restante habría sido adquirida con descuento, generando una ganancia sin riesgo.

2.6.2. Arbitraje de Butterfly Spread

Para cada vencimiento fijo, la varianza implícita total debe ser convexa en log-moneyness:

$$\frac{\partial^2 w}{\partial m^2}(m, \tau) \geq 0 \quad \text{para todo } \tau \quad (2.18)$$

Intuición. Si la curva de varianza fuera localmente cóncava en algún strike, un butterfly spread (comprar opciones en los dos strikes adyacentes y vender dos opciones en el strike central) tendría un costo negativo pero un pago no negativo, constituyendo un arbitraje.

Equivalencia. La convexidad de la varianza total en log-moneyness está relacionada (a través del resultado de Breeden y Litzenberger [9]) con la no negatividad de la densidad de probabilidad neutral al riesgo. Una región cóncava en la varianza total implica una densidad de probabilidad negativa, lo cual carece de sentido económico.

2.6.3. Verificación Discreta sobre una Grilla

Sobre la grilla discreta de 8×25 utilizada en esta tesis, estas condiciones continuas se aproximan mediante diferencias finitas:

- **Calendar:** $w(\tau_{i+1}, m_j) - w(\tau_i, m_j) \geq 0$ para todos los pares de vencimientos adyacentes y todos los strikes.
- **Butterfly:** la diferencia finita de segundo orden de w en la dirección de log-moneyness debe ser no negativa en cada strike interior (considerando el espaciado no uniforme).

Se aplica una tolerancia numérica de 10^{-10} para evitar contar ruido de punto flotante como violaciones. Las fórmulas específicas de diferencias finitas se presentan en la Sección 5.5.

2.7. Modelos Paramétricos de Superficie: SVI

La parametrización Stochastic Volatility Inspired (SVI) [17] ofrece un modelo parsimonioso para curvas individuales de volatilidad implícita. Es ampliamente utilizada en la práctica para la interpolación de curvas de volatilidad y sirve como referencia tradicional en esta tesis.

2.7.1. La Fórmula SVI

Para un único corte por vencimiento, SVI parametriza la varianza implícita total como:

$$w(k) = a + b \left[\rho(k - m) + \sqrt{(k - m)^2 + \sigma^2} \right] \quad (2.19)$$

donde k denota el log-moneyness (equivalente a m definido en la Ecuación (2.12); se usa k aquí para seguir la convención de Gatheral para la fórmula SVI) y los cinco parámetros tienen las siguientes interpretaciones [16]:

- a : nivel general de varianza (desplazamiento vertical).
- b : magnitud de la pendiente (controla la inclinación de los extremos).
- ρ : asimetría ($\rho < 0$ produce el skew de renta variable; $\rho = 0$ produce un smile simétrico).
- m : traslación horizontal (desplaza el centro de la curva respecto del ATM).
- σ : control de curvatura (un σ mayor redondea el mínimo de la curva; un σ menor agudiza el quiebre).

La fórmula SVI está diseñada para imitar el comportamiento asintótico de los modelos de volatilidad estocástica para valores extremos de moneyness (de ahí su nombre): para $|k| \rightarrow \infty$, $w(k) \sim a + b(1 + |\rho|)|k|$, produciendo el comportamiento lineal en los extremos que se observa en mercados reales.

2.7.2. Fortalezas y Limitaciones

Fortalezas. Con restricciones apropiadas sobre los parámetros ($b > 0$, $\sigma > 0$, $|\rho| < 1$), SVI produce curvas suaves y convexas que satisfacen naturalmente la condición de no arbitraje de butterfly. Es interpretable, rápida de ajustar y requiere solo cinco parámetros por corte.

Limitaciones. SVI ajusta cada corte de vencimiento de forma independiente: no existe un mecanismo para compartir información entre vencimientos. Esto tiene dos consecuencias. Primero, la consistencia entre cortes (la condición de calendar spread) no se impone y debe verificarse por separado. Segundo, cuando las observaciones son escasas en un vencimiento dado, el ajuste queda subdeterminado: cinco parámetros no pueden estimarse de forma confiable a partir de dos o tres puntos de datos. Esta limitación es la motivación fundamental de los enfoques basados en redes neuronales explorados en esta tesis: procesan la superficie 2D completa de forma conjunta, compartiendo información tanto entre strikes como entre vencimientos.

Capítulo 3

Fundamentos de *Deep Learning*

Este capítulo introduce los conceptos y arquitecturas de *deep learning* utilizados en esta tesis. Se cubre el marco de aprendizaje supervisado, las cuatro familias principales de arquitecturas (MLPs, CNNs, U-Nets, Transformers), los autoencoders variacionales, las técnicas estándar de entrenamiento, y la conexión entre la reconstrucción de superficies de volatilidad y el paradigma de predicción enmascarada.

3.1. Aprendizaje Supervisado y Funciones de Costo

3.1.1. El Problema de Regresión

El aprendizaje supervisado busca aprender una función $f_\theta : \mathcal{X} \rightarrow \mathcal{Y}$ a partir de un conjunto de pares entrada-salida $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$. En regresión (a diferencia de clasificación), el espacio de salida $\mathcal{Y} \subseteq \mathbb{R}^d$ es continuo. La función f_θ se parametriza mediante una red neuronal con parámetros entrenables θ , y el entrenamiento consiste en encontrar θ que minimice una función de costo $\mathcal{L}$ sobre los datos de entrenamiento.

Para la reconstrucción de superficies de volatilidad, la entrada es una superficie parcialmente observada junto con su máscara ($\mathbf{x} \in \mathbb{R}^{2 \times 8 \times 25}$) y la salida es la superficie completa ($\mathbf{y} \in \mathbb{R}^{1 \times 8 \times 25}$), un problema de regresión de 200 dimensiones.

3.1.2. Error Cuadrático Medio

La función de costo estándar para regresión es el error cuadrático medio (MSE, por sus siglas en inglés):

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|f_\theta(\mathbf{x}_i) - \mathbf{y}_i\|^2 \quad (3.1)$$

El MSE penaliza los errores grandes de forma cuadrática, lo que lo hace sensible a valores atípicos pero adecuado para problemas donde la exactitud de predicción es uniformemente importante en todo el espacio de salida. Minimizar el MSE es equivalente a la estimación por máxima verosimilitud bajo un modelo de ruido gaussiano.

3.1.3. Conjuntos de Datos y Generalización

Para evaluar si la función aprendida generaliza más allá de los datos de entrenamiento, el conjunto de datos se particiona en tres subconjuntos disjuntos:

- **Conjunto de entrenamiento:** se utiliza para calcular gradientes y actualizar θ .
- **Conjunto de validación:** se utiliza para monitorear la generalización durante el entrenamiento (por ejemplo, para parada temprana y selección de hiperparámetros). No se usa para el cálculo de gradientes.
- **Conjunto de test:** se reserva completamente hasta la evaluación final. Ofrece una estimación insesgada del rendimiento de generalización.

3.1.4. Sobreajuste y Regularización

Un modelo que se ajusta demasiado a los datos de entrenamiento puede fallar en generalizar, un fenómeno conocido como *sobreajuste* (*overfitting*). La brecha entre el costo de entrenamiento y el de validación es el principal indicador: si el costo de entrenamiento sigue disminuyendo mientras el de validación aumenta, el modelo está memorizando patrones del conjunto de entrenamiento en lugar de aprender características generalizables.

Las técnicas de regularización combaten el sobreajuste restringiendo la capacidad del modelo:

- **Weight decay** (regularización L2): agrega una penalización $\lambda \|\theta\|^2$ al costo, desincentivando pesos de magnitud grande.
- **Dropout** [37]: anula aleatoriamente una fracción de las activaciones durante el entrenamiento, evitando la co-adaptación de características.
- **Parada temprana:** detiene el entrenamiento cuando el rendimiento en validación deja de mejorar (Sección 3.7.3).

3.2. Perceptrones Multicapa

3.2.1. Arquitectura

El perceptrón multicapa (MLP, por *multilayer perceptron*) es la arquitectura de red neuronal más simple. Consiste en una secuencia de capas completamente conectadas (densas), donde cada capa computa:

$$\mathbf{h}^{(\ell)} = \phi(\mathbf{W}^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}) \quad (3.2)$$

donde $\mathbf{W}^{(\ell)}$ y $\mathbf{b}^{(\ell)}$ son la matriz de pesos y el vector de sesgo entrenables de la capa ℓ , y ϕ es una función de activación no lineal. Sin la no linealidad, una secuencia de capas lineales colapsaría en una única transformación lineal, independientemente de la profundidad.

3.2.2. Funciones de Activación

Las funciones de activación más comunes incluyen:

- **ReLU** (*Rectified Linear Unit*) [31]: $\phi(x) = \max(0, x)$. Simple, computacionalmente eficiente y efectiva para evitar el problema del gradiente evanescente. Es la opción por defecto en la mayoría de las arquitecturas.

- **GELU** (*Gaussian Error Linear Unit*) [22]: $\phi(x) = x \cdot \Phi(x)$, donde Φ es la CDF normal estándar. Es una aproximación suave de ReLU, preferida en arquitecturas Transformer por su diferenciabilidad en $x = 0$.
- **ELU** (*Exponential Linear Unit*) [12]: $\phi(x) = x$ para $x > 0$ y $\alpha(e^x - 1)$ para $x \leq 0$. Produce gradientes no nulos para entradas negativas, lo que a veces mejora la convergencia.

3.2.3. Aproximación Universal

El teorema de aproximación universal [24] establece que un MLP con una única capa oculta de ancho suficiente puede aproximar cualquier función continua sobre un dominio compacto con exactitud arbitraria. Si bien es teóricamente poderoso, este resultado no garantiza que la aproximación sea *eficiente*: en la práctica, el ancho requerido puede ser exponencialmente grande. Las redes profundas (múltiples capas de ancho moderado) suelen aportar representaciones mucho más compactas que las redes amplias y poco profundas.

3.2.4. Limitaciones para Datos Estructurados

El MLP trata su entrada como un vector plano sin estructura espacial. Cuando se aplica a datos organizados en una grilla como las superficies de volatilidad, la operación de aplanamiento descarta toda información sobre qué valores son espacialmente adyacentes. Cualquier patrón espacial debe aprenderse enteramente a partir de los datos, lo que requiere más ejemplos de entrenamiento y modelos más grandes en comparación con arquitecturas que explotan la estructura espacial por diseño.

3.3. Redes Neuronales Convolucionales

Las redes neuronales convolucionales (CNNs) [29] explotan la estructura espacial de datos organizados en grillas mediante operaciones locales y equivariantes a la traslación.

3.3.1. La Operación de Convolución

Una capa convolucional 2D aplica un conjunto de filtros (kernels) entrenables al mapa de características de entrada. Para un único filtro $\mathbf{w}$ de tamaño $k \times k$, la salida en la posición (i, j) es:

$$(\mathbf{x} * \mathbf{w})_{ij} = \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} w_{pq} \cdot x_{i+p, j+q} \quad (3.3)$$

Cada filtro produce un canal de salida (mapa de características), y una capa convolucional típicamente aplica n_c filtros para producir n_c canales de salida. Las propiedades clave son:

- **Conectividad local**: cada salida depende solo de una vecindad local de $k \times k$, explotando el supuesto de que los valores cercanos están más correlacionados que los distantes.

- **Pesos compartidos:** el mismo filtro se aplica en cada posición espacial, lo que reduce drásticamente el número de parámetros en comparación con una capa completamente conectada (un filtro de 3×3 tiene 9 parámetros independientemente del tamaño de la entrada).
- **Equivarianza a la traslación:** una entrada desplazada produce una salida idénticamente desplazada, lo que significa que la red detecta características independientemente de su posición en la grilla.

3.3.2. Campo Receptivo

El *campo receptivo* de una neurona es la región de la entrada que influye en su valor. Para una secuencia de L capas convolucionales con kernels de $k \times k$, el campo receptivo teórico crece a $(1+L(k-1)) \times (1+L(k-1))$. Por ejemplo, cinco capas de convoluciones 3×3 producen un campo receptivo de 11×11 . Esto permite que las CNNs profundas agreguen información de regiones espaciales amplias a través de operaciones locales sucesivas.

3.3.3. Relleno y Preservación Espacial

El *same-padding* (rellenar la entrada con $\lfloor k/2 \rfloor$ ceros en cada lado) asegura que las dimensiones espaciales se preserven a través de cada capa. Esto es esencial para tareas de predicción densa como la reconstrucción de superficies, donde la salida debe tener las mismas dimensiones espaciales que la entrada. Sin relleno, cada capa convolucional reduciría las dimensiones espaciales, eventualmente colapsando la grilla a un único punto.

3.3.4. Pooling y Submuestreo

Las capas de *pooling* (por ejemplo, *max pooling* o *average pooling*) reducen las dimensiones espaciales agregando regiones locales. Si bien son útiles para clasificación (donde no se necesita precisión espacial), el *pooling* descarta información espacial y generalmente se evita o se maneja con cuidado en tareas de predicción densa. Las convoluciones con *stride* > 1 brindan una alternativa entrenable al *pooling*.

3.4. La Arquitectura U-Net

La U-Net [36] es una arquitectura codificador-decodificador con conexiones de salto, originalmente desarrollada para segmentación de imágenes biomédicas.

3.4.1. Estructura Codificador-Decodificador

El *codificador* reduce progresivamente la resolución de la entrada mediante capas de convolución y *pooling*, construyendo representaciones de características a múltiples escalas espaciales. El *decodificador* aumenta progresivamente la resolución de estas representaciones de vuelta a la resolución original mediante interpolación bilineal o convoluciones transpuestas. Esta estructura captura tanto el detalle local (en las capas tempranas de alta resolución) como el contexto global (en el cuello de botella profundo de baja resolución).

3.4.2. Conexiones de Salto

La característica definitoria de U-Net es el uso de conexiones de salto (*skip connections*) que concatenan los mapas de características del codificador con los correspondientes del decodificador en cada nivel de resolución. Sin conexiones de salto, la información espacial de alta frecuencia perdida durante el submuestreo sería irrecuperable. Las conexiones de salto ofrecen un camino directo para que el detalle de alta frecuencia fluya del codificador al decodificador, mientras que el cuello de botella captura la estructura global de baja frecuencia. Esta combinación hace que U-Net sea particularmente efectiva para tareas que requieren tanto localización precisa como comprensión semántica.

3.4.3. Aplicación a Predicción Densa

U-Net y sus variantes son la arquitectura dominante para tareas de predicción densa donde cada punto de la entrada requiere una predicción de salida: segmentación semántica, estimación de profundidad e *inpainting* de imágenes. La reconstrucción de superficies de volatilidad encaja naturalmente en este paradigma: cada punto de la grilla requiere una salida (la volatilidad implícita reconstruida), y el modelo debe combinar suavidad local (las volatilidades implícitas cercanas son similares) con consistencia global (la superficie debe ser coherente en todo el rango de vencimientos).

3.5. La Arquitectura Transformer

La arquitectura Transformer [39] reemplaza las operaciones locales de las CNNs con mecanismos de atención globales, permitiendo que cada posición en la secuencia interactúe directamente con todas las demás.

3.5.1. Auto-Atención

La operación central es la *atención de producto punto escalado* (*scaled dot-product attention*). Dados un conjunto de consultas $\mathbf{Q}$, claves $\mathbf{K}$ y valores $\mathbf{V}$ (todos derivados de la entrada mediante proyecciones lineales aprendidas), la atención computa una suma ponderada de los valores donde los pesos se determinan por la similitud entre consultas y claves:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V} \quad (3.4)$$

donde d_k es la dimensión de cada clave (el factor de escala evita que el softmax sature en dimensiones altas).

En la *auto-atención*, las consultas, claves y valores provienen de la misma secuencia de entrada: cada posición atiende a todas las demás, con pesos de atención determinados por funciones de compatibilidad aprendidas. Esto le da a cada posición un campo receptivo global en una sola capa, a diferencia de las CNNs, que requieren múltiples capas apiladas para lograr interacciones de largo alcance.

3.5.2. Atención Multi-Cabeza

En lugar de computar una única función de atención, la atención multi-cabeza (*multi-head attention*) ejecuta h funciones de atención en paralelo, cada una con sus propias

proyecciones aprendidas:

$$\text{MHA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O \quad (3.5)$$

donde $\text{head}_i = \text{Attention}(\mathbf{QW}_i^Q, \mathbf{KW}_i^K, \mathbf{VW}_i^V)$. Cada cabeza puede aprender patrones de atención distintos; por ejemplo, una cabeza podría atender a posiciones cercanas mientras que otra atiende a posiciones distantes.

3.5.3. Codificación Posicional

A diferencia de las CNNs, la operación de atención es *equivariante a permutaciones*: produce la misma salida independientemente del orden de la secuencia de entrada. Para inyectar información posicional, los Transformers agregan una *codificación posicional* a los embeddings de entrada. El Transformer original [39] utiliza codificaciones sinusoidales fijas indexadas por posición en la secuencia. Alternativas incluyen embeddings posicionales aprendidos y características de Fourier basadas en coordenadas [38], estas últimas son las que se utilizan en esta tesis (Sección 5.2.4).

3.5.4. Transformer Codificador-Decodificador

La arquitectura Transformer completa consiste en un codificador y un decodificador:

- **Codificador:** una pila de capas de auto-atención que procesan la secuencia de entrada, produciendo una representación contextualizada (memoria).
- **Decodificador:** una pila de capas que primero aplican auto-atención entre las consultas de salida, y luego *atención cruzada* (*cross-attention*) a la memoria del codificador. La atención cruzada permite que cada posición del decodificador atienda a todas las posiciones del codificador, lo que permite el flujo de información desde la entrada hacia la salida.

Cada capa (tanto del codificador como del decodificador) también incluye una red feed-forward aplicada posición por posición (dos capas lineales con una activación no lineal) y conexiones residuales con normalización por capa.

3.5.5. Enmascaramiento de Atención

Las máscaras de atención permiten excluir selectivamente ciertas posiciones del cómputo de atención. Al asignar valores de $-\infty$ antes del softmax, posiciones específicas reciben peso de atención cero. Este mecanismo se utiliza de varias formas:

- **Máscaras de relleno:** excluyen posiciones de relleno en secuencias de longitud variable.
- **Máscaras causales:** impiden que las posiciones atiendan a posiciones futuras (se usan en generación autorregresiva).
- **Máscaras de observación:** en la aplicación de esta tesis, excluyen los puntos faltantes de la grilla de la auto-atención del codificador, asegurando que el codificador solo procese valores observados.

3.5.6. Atención para Entradas con Datos Faltantes

Más allá de sus aplicaciones generales, el mecanismo de atención tiene una ventaja natural para procesar datos observados de forma dispersa e irregular, lo que motiva su aplicación al problema de reconstrucción de superficies de volatilidad (desarrollada en la Sección 5.2.4). A diferencia de las CNNs, que aplican filtros de tamaño fijo independientemente de si las posiciones cubiertas están observadas, la atención dirige dinámicamente la información desde donde existan observaciones. Cuando las observaciones son densas, cada consulta atiende a muchas fuentes cercanas; cuando son escasas, el mismo mecanismo se adapta atendiendo a fuentes más distantes. Esta flexibilidad es la ventaja arquitectónica clave de la arquitectura Transformer para la reconstrucción de superficies de volatilidad.

3.6. Autoencoders Variacionales

Los autoencoders variacionales (VAEs) [28] combinan *deep learning* con modelado generativo probabilístico. A diferencia de los modelos discriminativos anteriores, que aprenden un mapeo directo de entrada a salida, los VAEs aprenden una representación latente de la distribución de los datos.

3.6.1. Modelos de Variables Latentes

Un VAE postula que los datos observados $\mathbf{x}$ se generan a partir de una variable latente de baja dimensionalidad $\mathbf{z}$ mediante una red decodificadora (generadora):

$$p_{\theta}(\mathbf{x}) = \int p_{\theta}(\mathbf{x}|\mathbf{z}) p(\mathbf{z}) d\mathbf{z} \quad (3.6)$$

donde $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ es un prior gaussiano estándar y $p_{\theta}(\mathbf{x}|\mathbf{z})$ es la red decodificadora parametrizada por θ . La maximización directa de la log-verosimilitud $\log p_{\theta}(\mathbf{x})$ es intratable porque requiere integrar sobre todos los posibles códigos latentes.

3.6.2. La Cota Inferior de Evidencia (ELBO)

En lugar de maximizar la log-verosimilitud intratable, los VAEs maximizan una cota inferior conocida como ELBO (*Evidence Lower Bound*):

$$\log p_{\theta}(\mathbf{x}) \geq \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z})) \quad (3.7)$$

donde $q_{\phi}(\mathbf{z}|\mathbf{x})$ es una red codificadora que aproxima la posterior intratable $p_{\theta}(\mathbf{z}|\mathbf{x})$. El primer término incentiva la reconstrucción exacta; el segundo término regulariza el espacio latente para que se mantenga cercano al prior.

3.6.3. El Truco de Reparametrización

El entrenamiento requiere diferenciar a través de la operación de muestreo $\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})$. El truco de reparametrización expresa la muestra como una función determinística de la salida del codificador y ruido independiente:

$$\mathbf{z} = \boldsymbol{\mu}_{\phi}(\mathbf{x}) + \boldsymbol{\sigma}_{\phi}(\mathbf{x}) \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (3.8)$$

Esto traslada la estocasticidad fuera del grafo computacional, permitiendo la retropropagación estándar a través del codificador.

3.6.4. VAEs para Reconstrucción

En el contexto de la reconstrucción de superficies a partir de observaciones parciales, el espacio latente del VAE ofrece una variedad aprendida de superficies completas plausibles. Dada una observación parcial, el modelo puede buscar el código latente $\mathbf{z}^*$ cuya superficie decodificada mejor coincida con los valores observados, proyectando los datos incompletos sobre la variedad aprendida. Este enfoque se detalla en la Sección 5.2.5.

3.7. Técnicas de Entrenamiento

3.7.1. Descenso de Gradiente Estocástico y Adam

Las redes neuronales se entrenan minimizando la función de costo mediante optimización basada en gradientes. El *descenso de gradiente estocástico* (SGD, por *stochastic gradient descent*) actualiza los parámetros utilizando gradientes calculados sobre mini-lotes aleatorios de los datos de entrenamiento, reduciendo el costo computacional e introduciendo ruido beneficioso que puede ayudar a escapar de mínimos locales.

El optimizador **Adam** [27] extiende SGD con tasas de aprendizaje adaptativas por parámetro basadas en estimaciones del primer y segundo momento del gradiente:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t \quad (3.9)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2 \quad (3.10)$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \quad (3.11)$$

donde $\mathbf{g}_t$ es el gradiente, $\hat{\mathbf{m}}_t$ y $\hat{\mathbf{v}}_t$ son las estimaciones de momento corregidas por sesgo, η es la tasa de aprendizaje, y $\beta_1 = 0,9$, $\beta_2 = 0,999$ son las tasas de decaimiento por defecto. Adam es el optimizador más ampliamente utilizado en *deep learning* y se utiliza en todos los experimentos de esta tesis.

3.7.2. Tasa de Aprendizaje

La tasa de aprendizaje η es el hiperparámetro más crítico. Si es demasiado grande, la optimización oscila o diverge; si es demasiado pequeña, la convergencia es imprácticamente lenta. La tasa de aprendizaje óptima varía entre arquitecturas: los modelos basados en atención (Transformers) típicamente requieren tasas de aprendizaje menores que los modelos convolucionales o completamente conectados, porque el cómputo de atención amplifica las magnitudes de los gradientes.

Los *schedules* de tasa de aprendizaje (por ejemplo, *cosine annealing*, *warm-up*) ajustan η durante el entrenamiento. Si bien suelen ser beneficiosos, los *schedules* introducen hiperparámetros adicionales que pueden interactuar de forma diferente con cada arquitectura, complicando las comparaciones justas.

3.7.3. Parada Temprana

La parada temprana monitorea el costo de validación durante el entrenamiento y lo detiene cuando no ha mejorado durante un número especificado de iteraciones de entrenamiento (la *paciencia*). Se retiene el checkpoint del modelo con el menor costo de validación. Esta es una forma de regularización: evita que el modelo ajuste ruido en los datos de entrenamiento que degradaría la generalización. En la práctica, la parada temprana es frecuentemente la técnica de regularización más efectiva y simple.

3.7.4. *Batch Normalization* y *Layer Normalization*

La *batch normalization* [26] normaliza las activaciones dentro de cada mini-lote, estabilizando el entrenamiento y permitiendo tasas de aprendizaje más altas. Sin embargo, su comportamiento depende del tamaño del lote y difiere entre los modos de entrenamiento e inferencia.

La *layer normalization* [3] normaliza a través de la dimensión de características dentro de cada muestra, haciéndola independiente del tamaño del lote. Es la técnica de normalización estándar en arquitecturas Transformer, donde la variante “pre-LN” (normalizar antes de cada sub-capas en lugar de después) se ha convertido en el estándar para la estabilidad del entrenamiento.

3.8. Predicción Enmascarada e *Inpainting*

3.8.1. Modelos de Lenguaje Enmascarados

El paradigma de predicción enmascarada fue popularizado en procesamiento de lenguaje natural por BERT [13], que enmascara aleatoriamente tokens en una secuencia de texto y entrena una arquitectura Transformer para predecir los tokens enmascarados a partir del contexto. Este objetivo de entrenamiento auto-supervisado resultó notablemente efectivo para aprender representaciones transferibles.

3.8.2. Modelado de Imágenes Enmascaradas

El concepto se extendió a *computer vision* con los *Masked Autoencoders* (MAE) [21], que enmascaran parches aleatorios de una imagen y entrenan un *Vision Transformer* para reconstruir el original. MAE demostró que enmascarar el 75 % de los parches de imagen durante el entrenamiento produce representaciones visuales poderosas, y que la arquitectura Transformer codificador-decodificador es naturalmente adecuada para esta tarea porque puede procesar solo los parches visibles (reduciendo el cómputo) y usar el decodificador para reconstruir todos los parches.

3.8.3. *Inpainting* de Imágenes

El *inpainting* de imágenes, es decir, completar regiones faltantes o dañadas de una imagen, es un problema clásico de *computer vision*. Los enfoques modernos basados en *deep learning* utilizan CNNs o U-Nets codificador-decodificador que toman la imagen dañada y una máscara como entrada y producen la imagen completada. El objetivo de entrenamiento es el error de reconstrucción sobre las regiones faltantes.

3.8.4. Reconstrucción de Superficies de Volatilidad como Predicción Enmascarada

La tarea de reconstrucción de superficies de volatilidad combina elementos de los tres paradigmas:

- Al igual que BERT y MAE, se *enmascaran aleatoriamente* porciones de la entrada y se entrena al modelo para reconstruirlas. El enmascaramiento dinámico aporta aumento de datos y previene la memorización.
- Al igual que el *inpainting* de imágenes, la entrada es una grilla 2D con entradas faltantes, y el modelo debe completarlas respetando la estructura de los datos.
- A diferencia de MAE (que es auto-supervisado), esta tarea es *completamente supervisada*: se dispone de superficies de referencia para el entrenamiento. Y a diferencia del *inpainting* genérico de imágenes, los datos tienen estructura específica del dominio (suavidad, convexidad) que puede explotarse mediante elecciones arquitectónicas y penalizaciones de no arbitraje.

Este encuadre de la reconstrucción de superficies de volatilidad combina elementos de ingeniería financiera y aprendizaje de representaciones, motivando el conjunto diverso de arquitecturas evaluadas en esta tesis.

Capítulo 4

Datos y Representación de Superficies

Los capítulos anteriores introdujeron los conceptos financieros que subyacen a las superficies de volatilidad y las arquitecturas de *deep learning* utilizadas para modelarlas. Este capítulo describe la representación concreta de los datos, los dos conjuntos de datos utilizados a lo largo de este trabajo (uno sintético y otro de mercados reales de opciones sobre renta variable) y la estrategia de enmascaramiento que enmarca la reconstrucción de superficies como un problema de aprendizaje supervisado.

4.1. Representación de la Superficie

Una superficie de volatilidad es una función bidimensional $\sigma(K, \tau)$ que mapea el precio de ejercicio (strike) K y el tiempo al vencimiento τ a la volatilidad implícita. Para fines computacionales, se discretiza esta función sobre una grilla fija.

4.1.1. Discretización en Grilla

Cada superficie se representa sobre una grilla de 8 vencimientos y 25 strikes, lo que produce 200 puntos por superficie. El eje de vencimientos utiliza los siguientes tenores (en años):

$$\tau = [0,08; 0,17; 0,25; 0,5; 0,75; 1,0; 1,5; 2,0] \quad (4.1)$$

abarcando aproximadamente desde un mes hasta dos años. El eje de strikes utiliza 25 puntos equiespaciados:

$$\mathbf{K} = \text{linspace}(70, 130, 25) = [70,0; 72,5; 75,0; \dots; 127,5; 130,0] \quad (4.2)$$

con un precio forward de referencia $F = 100$. Esta grilla cubre strikes desde un 30% *out of the money* en puts hasta un 30% *out of the money* en calls, lo que abarca la porción líquida de la mayoría de las cadenas de opciones sobre renta variable.

4.1.2. Coordenadas en Log-Moneyness

En lugar de utilizar precios de ejercicio crudos, el eje de strikes se expresa en log-moneyness $m = \log(K/F)$ (Ecuación (2.12)), que varía de $-0,357$ a $+0,262$ en nuestra grilla. Como se discutió en la Sección 2.4, el log-moneyness centra la superficie en el nivel

at-the-money y normaliza entre distintos niveles de precio del subyacente, haciendo las superficies comparables entre activos y períodos temporales.

4.1.3. Representación de Entrada

Cada superficie se almacena como una matriz $\Sigma \in \mathbb{R}^{8 \times 25}$ de volatilidades implícitas. Para la tarea de reconstrucción, el modelo recibe un tensor de entrada de dos canales $\mathbf{X} \in \mathbb{R}^{2 \times 8 \times 25}$:

- **Canal 0:** La superficie de volatilidad implícita enmascarada, donde los puntos no observados se establecen en cero.
- **Canal 1:** Una máscara binaria de observación $\mathbf{M} \in \{0, 1\}^{8 \times 25}$, donde $M_{ij} = 1$ indica que la volatilidad implícita en el tenor τ_i y strike K_j es observada, y $M_{ij} = 0$ indica que falta.

El objetivo es la superficie completa Σ , y la función de costo de entrenamiento se calcula solo sobre los puntos faltantes (donde $M_{ij} = 0$), evitando que el modelo simplemente aprenda la función identidad sobre los valores observados.

4.2. Generación de Datos Sintéticos

El conjunto de datos de entrenamiento principal consiste en 10.000 superficies de volatilidad generadas a partir del modelo de volatilidad estocástica de Heston introducido en la Sección 2.5. Se eligió el modelo de Heston porque es el modelo de referencia para volatilidad estocástica: captura las características esenciales de las superficies de volatilidad reales (smile, skew, estructura temporal) mientras permanece analíticamente tratable gracias a la fórmula de valuación semi-analítica de la Ecuación (2.16).

4.2.1. Muestreo de Parámetros

Para generar un conjunto de datos diverso que abarque condiciones de mercado realistas, los parámetros de cada superficie se muestrean uniformemente de los rangos mostrados en la Tabla 4.1.

Tabla 4.1: Rangos de muestreo de parámetros de Heston para la generación de datos sintéticos.

Parámetro	Mín	Máx	Interpretación
v_0	0,01	0,16	Volatilidad spot 10 %–40 %
κ	0,5	5,0	Reversión a la media lenta a rápida
θ	0,01	0,16	Volatilidad de largo plazo 10 %–40 %
ξ	0,1	0,8	Curvatura del smile baja a alta
ρ	−0,9	−0,1	Skew de renta variable moderado a fuerte

Estos rangos están basados en calibraciones típicas de renta variable [16]. La correlación ρ se restringe a valores negativos, reflejando el efecto de apalancamiento bien documentado en mercados de renta variable: los precios de las acciones y la volatilidad

se mueven en direcciones opuestas. Se impone la condición de Feller $2\kappa\theta \geq \xi^2$ para asegurar que el proceso de varianza permanezca estrictamente positivo, remuestreando los conjuntos de parámetros que la violen.

La Figura 4.1 muestra la distribución empírica de los cinco parámetros de Heston a lo largo de las 10.000 superficies generadas.

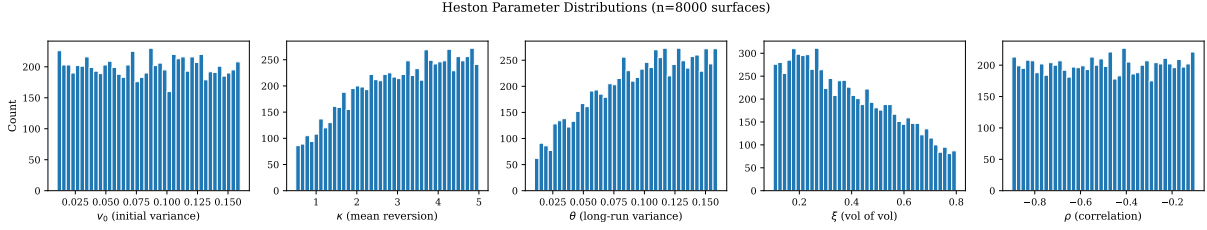


Figura 4.1: Distribución de los parámetros de Heston en las 10.000 superficies sintéticas. El muestreo es uniforme dentro de cada rango, sujeto a la condición de Feller $2\kappa\theta \geq \xi^2$, que modifica ligeramente la distribución conjunta de κ , θ y ξ .

4.2.2. Valuación y Recuperación de Volatilidad Implícita

Para cada conjunto de parámetros, se calculan los precios de opciones call europeas en los 200 puntos de la grilla (K_j, τ_i) utilizando la fórmula de valuación semi-analítica de Heston (Ecuación (2.16)), con la formulación de Albrecher et al. [2] para evitar el “Little Heston Trap”. La integración numérica se realiza con cuadratura adaptativa con una tolerancia de 10^{-10} .

Los precios de opciones resultantes se convierten luego a volatilidades implícitas utilizando el solver de Newton-Raphson descrito en la Sección 2.3, con una tolerancia de convergencia de 10^{-12} que asegura exactitud de precisión de máquina en las volatilidades implícitas recuperadas.

4.2.3. Ejemplos de Superficies

La Figura 4.2 muestra dos superficies representativas del conjunto de datos, ilustrando cómo los parámetros de Heston controlan la forma de la superficie. Con correlación cercana a cero ($\rho = -0,1$) y alta volatilidad de la volatilidad ($\xi = 0,5$), la superficie exhibe un smile simétrico. Con correlación fuertemente negativa ($\rho = -0,7$), emerge el skew característico de renta variable: la volatilidad implícita aumenta para strikes bajos (puts *out of the money*) y disminuye para strikes altos.

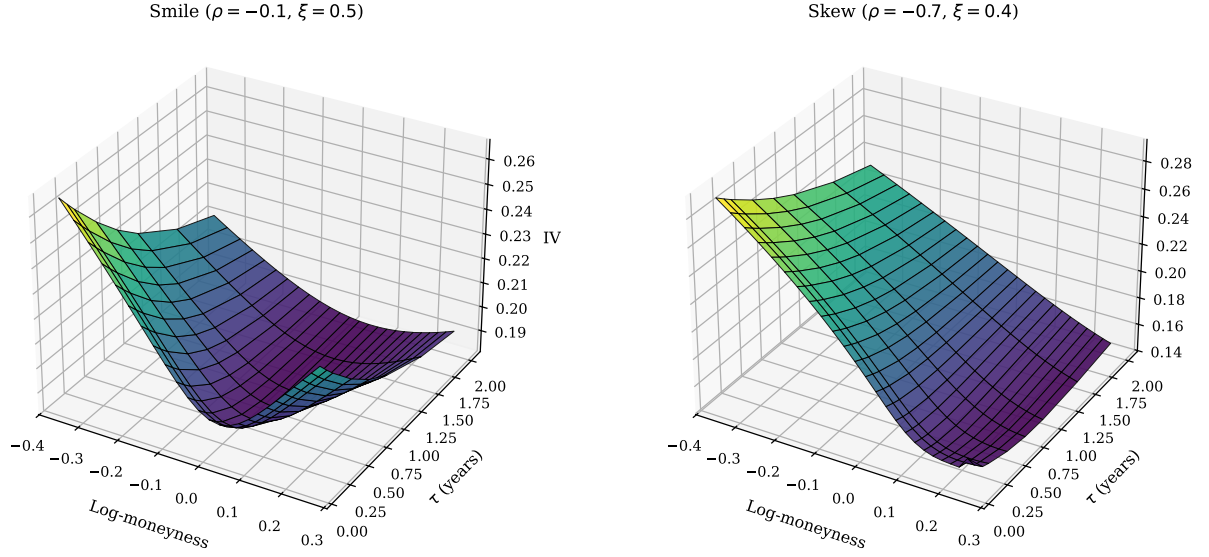


Figura 4.2: Dos superficies de volatilidad de Heston del conjunto de datos sintético. Izquierda: smile simétrico ($\rho = -0,1$, $\xi = 0,5$). Derecha: skew de renta variable ($\rho = -0,7$, $\xi = 0,4$). El parámetro de correlación ρ controla la asimetría, mientras que la volatilidad de la volatilidad ξ controla la curvatura general.

La Figura 4.3 aísla el efecto de cada parámetro, variando ρ y ξ individualmente mientras se mantienen los demás parámetros fijos.

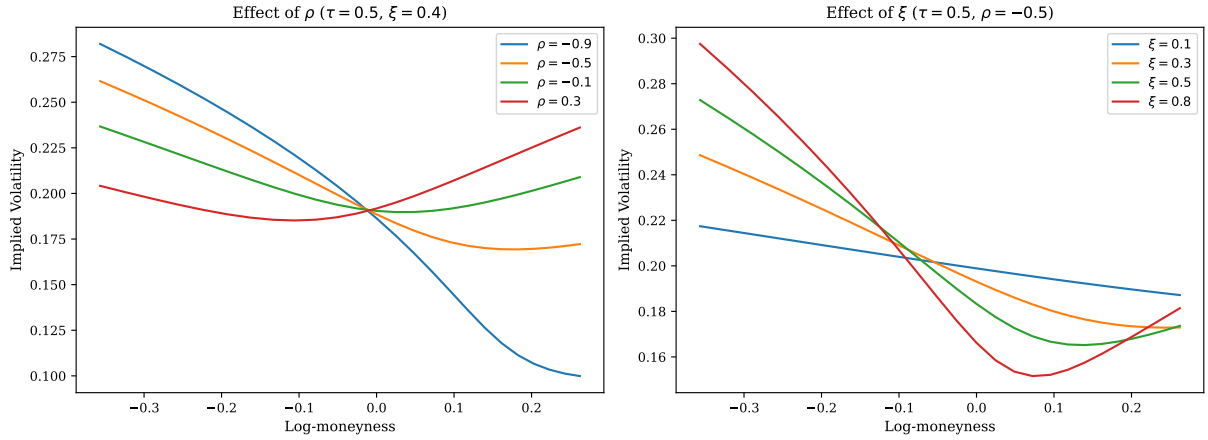


Figura 4.3: Sensibilidad a los parámetros de Heston en $\tau = 0,5$. Izquierda: variando ρ de $-0,9$ a $-0,1$ se controla la inclinación del skew. Derecha: variando ξ de $0,1$ a $0,8$ se controla la curvatura del smile.

4.2.4. Escala del Conjunto de Datos y Partición

Se generan 10.000 superficies en total, divididas en conjuntos de entrenamiento, validación y test:

Tabla 4.2: Partición del conjunto de datos sintético. Cada partición utiliza una semilla aleatoria fija para reproducibilidad.

Partición	Superficies	Semilla	Propósito
Entrenamiento	8.000	42	Entrenamiento del modelo
Validación	1.000	123	Parada temprana, ajuste de hiperparámetros
Test	1.000	456	Evaluación final

La generación de superficies se paraleliza entre núcleos de CPU, donde cada superficie requiere aproximadamente 200 evaluaciones de la función característica (una por punto de la grilla). El conjunto de datos completo de 10.000 superficies se genera en menos de una hora en una máquina de 8 núcleos.

4.3. Datos Reales de Mercado

Para validar que los modelos entrenados con datos sintéticos pueden generalizar a mercados reales, y para evaluar si los datos reales ofrecen suficiente estructura para el entrenamiento directo, se construye un segundo conjunto de datos a partir de opciones históricas sobre SPY (el ETF del S&P 500).

4.3.1. Fuente de Datos

Se utiliza el conjunto de datos de opciones históricas compilado por Dubach [15], que cubre opciones sobre SPY desde 2008 hasta 2025. El conjunto de datos original contiene aproximadamente 24,7 millones de registros de contratos de opciones a lo largo de 18 años, incluyendo precios de cierre diarios del subyacente.

4.3.2. Pipeline de Construcción de Superficies

Construir una superficie de volatilidad a partir de cotizaciones de opciones requiere varios pasos de procesamiento, ya que los datos de mercado llegan como una colección de triplas (K, τ, σ_{IV}) en posiciones irregulares en lugar de sobre una grilla regular.

Filtrado. Se aplican los siguientes filtros a la cadena de opciones de cada día:

- **Moneyness:** $K/S \in [0,70; 1,30]$, coincidiendo con el rango de nuestra grilla.
- **Días al vencimiento:** $DTE \in [7, 800]$, excluyendo opciones de muy corto plazo (dominadas por efectos de microestructura) y de muy largo plazo (ilíquidas).
- **Liquidez:** Precio bid > 0 y spread bid-ask relativo $< 50\%$.
- **Límites de IV:** $\sigma_{IV} \in [0,01; 2,0]$, excluyendo errores de datos.
- **Selección OTM:** Para cada strike, se utilizan opciones *out of the money* (puts para $K < S$, calls para $K \geq S$), que son más líquidas y tienen spreads más ajustados que sus contrapartes *in the money*.

Emparejamiento de tenores. Para cada uno de los 8 vencimientos estándar τ_i , se busca el vencimiento disponible más cercano dentro de una tolerancia relativa del 30 %. Si dos vencimientos de mercado enmarcan un tenor estándar, se interpola en el espacio de varianza total ($\sigma^2\tau$), lo que preserva la condición de no arbitraje de que la varianza total debe ser no decreciente en el vencimiento.

Interpolación en strike. Dentro de cada tenor emparejado, se interpolan las volatilidades implícitas cotizadas en el mercado sobre nuestra grilla de 25 puntos en log-moneyness utilizando interpolación por spline cúbico. No se extrapola más allá del rango de cotizaciones disponibles: los puntos de la grilla fuera del rango de strikes observado se marcan como faltantes.

Filtro de calidad. Se retienen solo las superficies con al menos un 75 % de cobertura de tenores (6 de 8 tenores presentes) y al menos un 70 % de cobertura promedio de strikes en los tenores presentes. Esto asegura un nivel mínimo de información para cada superficie.

4.3.3. Estadísticas del Conjunto de Datos

Después del filtrado, se obtienen 3.900 superficies que abarcan el período 2008–2025. El conjunto de datos se divide temporalmente (no aleatoriamente) para evitar fuga de información:

Tabla 4.3: Partición del conjunto de datos real de SPY. La partición temporal asegura que no se filtre información futura al entrenamiento.

Partición	Superficies	Años	Propósito
Entrenamiento	2.912	2008–2021	Entrenamiento del modelo
Validación	501	2022–2023	Parada temprana
Test	487	2024–2025	Evaluación final

El conjunto de datos real presenta datos faltantes naturales, con un promedio de aproximadamente el 22,5 % de los puntos de la grilla por superficie, concentrados en los extremos (strikes muy *out of the money*) y en vencimientos cortos donde puede no haber opciones semanales disponibles. La Figura 4.4 muestra la distribución de fracciones faltantes y la cobertura temporal.

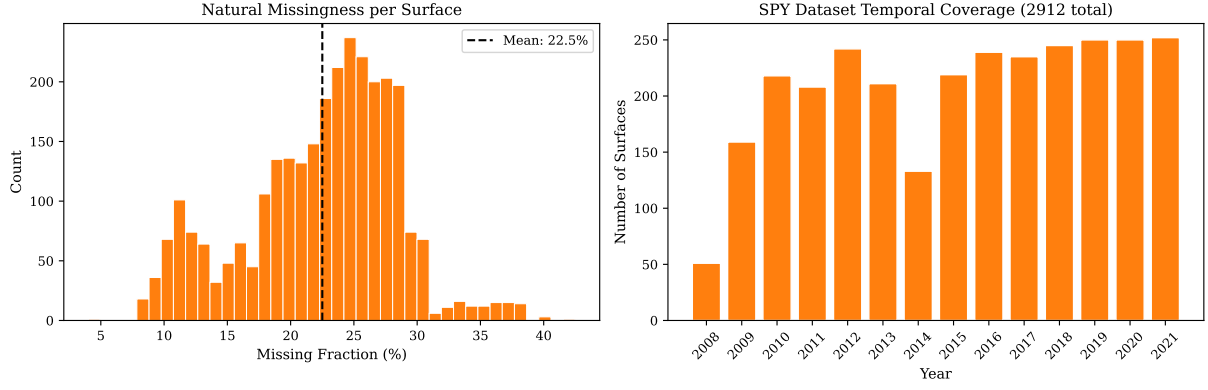


Figura 4.4: Izquierda: distribución de la fracción de puntos faltantes en la partición de entrenamiento (2.912 superficies de SPY). La media es aproximadamente 22,5 %. Derecha: distribución temporal de superficies por año, reflejando la disponibilidad de mercado y el crecimiento de las opciones listadas a lo largo del tiempo.

4.4. Superficies Sintéticas vs Reales

Si bien el modelo de Heston captura las principales características cualitativas de las superficies de volatilidad reales (skew, estructura temporal y smile), existen diferencias estructurales importantes entre los datos sintéticos y reales.

La Figura 4.5 muestra una superficie sintética representativa junto a una superficie real de SPY. La superficie de Heston es suave y está determinada paramétricamente: dados cinco parámetros, la superficie completa queda totalmente especificada. La superficie real, en cambio, exhibe una estructura local más rica (quiebres, asimetrías y formas de smile dependientes del vencimiento) que no puede ser capturada por ningún modelo de cinco parámetros. La superficie real también presenta huecos naturales (regiones en blanco) donde no había opciones líquidas disponibles.

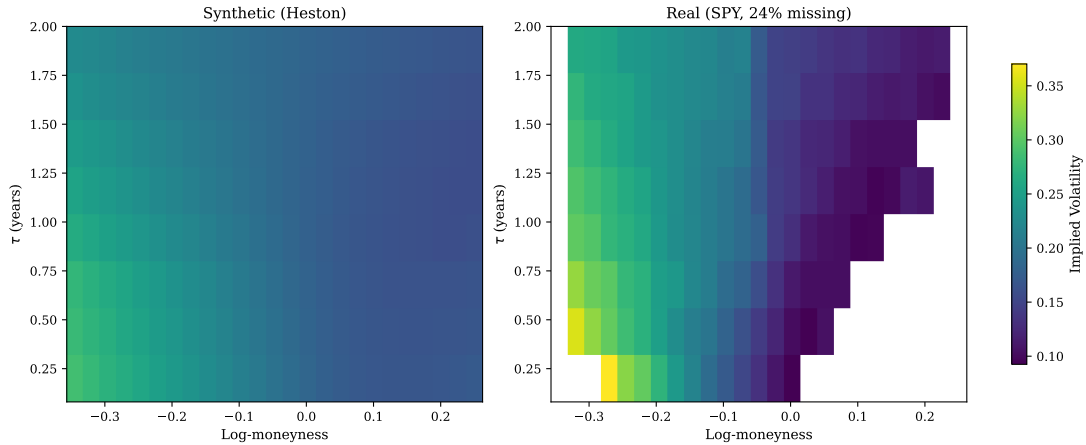


Figura 4.5: Comparación de una superficie sintética (Heston) y una real (SPY) sobre la misma grilla y escala de color. La superficie real exhibe una estructura local más rica y datos faltantes naturales.

La Figura 4.6 compara las distribuciones de volatilidad implícita de ambos conjuntos de datos. El conjunto sintético, generado con v_0 y θ muestreados hasta 0,16 ($\approx 40\%$ de

volatilidad), cubre un rango de IV más amplio que los datos reales de SPY, que se concentran entre el 10 % y el 50 % con una moda alrededor del 15–20 %. La distribución real también exhibe una cola derecha más pesada, reflejando picos de volatilidad ocasionales durante eventos de estrés de mercado (por ejemplo, la crisis financiera de 2008, el crash por COVID de 2020).

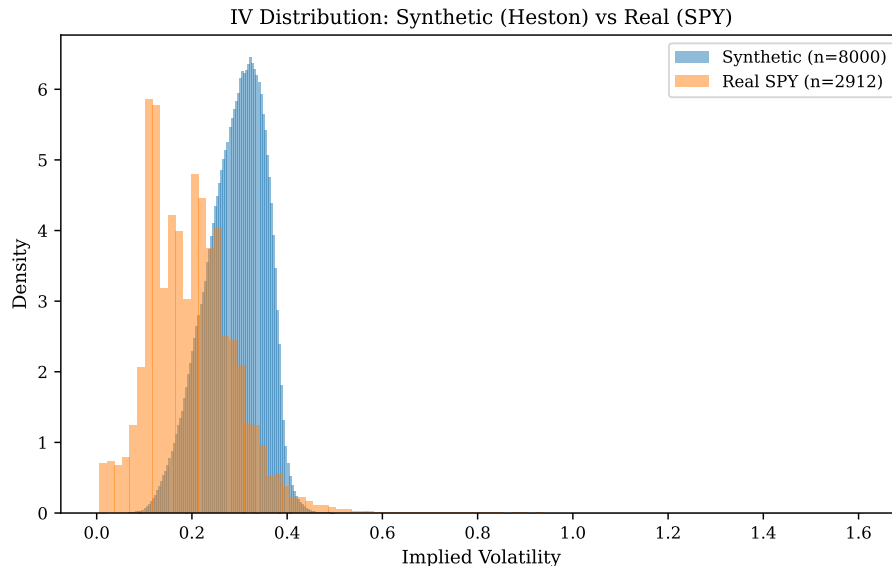


Figura 4.6: Distribuciones de volatilidad implícita para los conjuntos de datos sintético (Heston) y real (SPY). El conjunto sintético tiene una cobertura más amplia; el conjunto real tiene una moda concentrada y una cola derecha más pesada.

Estas diferencias estructurales tienen implicancias directas para el *transfer learning*, que se explora en el Capítulo 6: los modelos preentrenados con superficies de Heston deben adaptarse a las características distribucionales diferentes y a la estructura más fina de los datos de mercado reales.

4.5. Estrategia de Enmascaramiento

La tarea central de reconstrucción requiere que los modelos predigan las volatilidades implícitas faltantes a partir de observaciones parciales. Esto se simula aplicando una máscara binaria a cada superficie completa durante el entrenamiento.

4.5.1. Enmascaramiento Aleatorio

La estrategia de enmascaramiento por defecto elimina cada punto de la grilla independientemente con probabilidad p , donde $p = 0,3$ (30 % faltante) es la configuración de referencia. Formalmente, la máscara $\mathbf{M} \in \{0, 1\}^{8 \times 25}$ se genera como:

$$M_{ij} \sim \text{Bernoulli}(1 - p), \quad \forall i \in \{1, \dots, 8\}, j \in \{1, \dots, 25\} \quad (4.3)$$

donde $M_{ij} = 1$ significa que el punto es observado y $M_{ij} = 0$ que falta. Esto produce máscaras con $p \times 200 = 60$ puntos faltantes esperados por superficie, aunque el número real varía entre muestras debido a la aleatoriedad.

4.5.2. Enmascaramiento Dinámico

Una decisión de diseño clave es que las máscaras se generan *dinámicamente* durante el entrenamiento: cada vez que una superficie se extrae del conjunto de datos, se genera una nueva máscara aleatoria. Esto significa que la misma superficie subyacente se ve con diferentes patrones de observación a lo largo de las iteraciones de entrenamiento, actuando como una forma de **aumento de datos**. El modelo no puede memorizar qué puntos específicos faltan para una superficie dada; en cambio, debe aprender una estrategia de reconstrucción general que funcione para cualquier patrón de datos faltantes.

4.5.3. El Enmascaramiento como Parámetro de Evaluación

Si bien los modelos se entrenan con una fracción fija de datos faltantes ($p = 0,3$), se evalúan en un rango de niveles de escasez desde $p = 0,1$ (10 % faltante, casi completo) hasta $p = 0,9$ (90 % faltante, extremadamente disperso). Esto permite evaluar cuán gradualmente se degrada cada arquitectura a medida que las observaciones se vuelven más escasas, sin necesidad de reentrenar. Los resultados de este barrido se presentan en el Capítulo 6.

La Figura 4.7 ilustra el proceso de enmascaramiento: una superficie de Heston completa (izquierda), la máscara de observación con el 30 % de los puntos marcados como faltantes (centro), y la entrada enmascarada resultante que recibe el modelo (derecha).

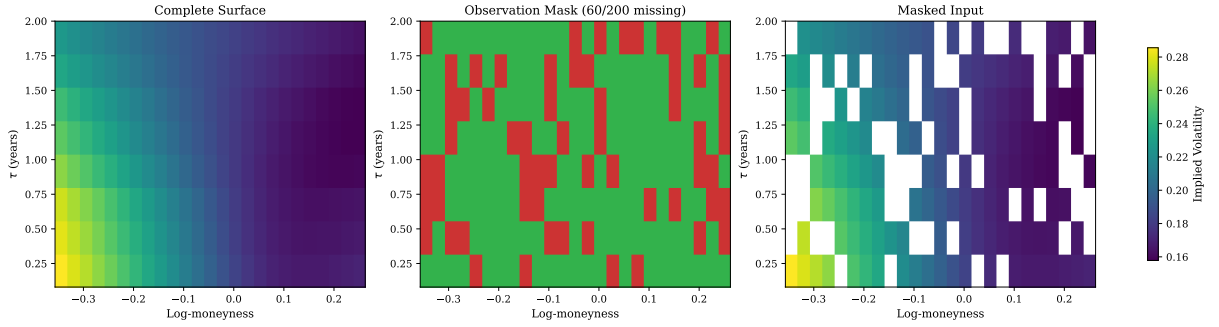


Figura 4.7: Ilustración del enmascaramiento. Izquierda: superficie de volatilidad completa. Centro: máscara de observación (verde = observado, rojo = faltante, 30 % de tasa de faltantes). Derecha: la superficie de entrada enmascarada, donde los puntos faltantes se establecen en cero.

4.5.4. Enmascaramiento en Datos Reales

Para los datos reales de SPY, las superficies ya tienen datos faltantes naturales del mercado. Durante el entrenamiento, se aplica una máscara sintética adicional sobre los huecos existentes: los puntos observados se submuestrean adicionalmente con tasa p . Esto asegura que el modelo se entrene para reconstruir a partir de aún menos observaciones de las que el mercado provee naturalmente. La función de costo de entrenamiento se calcula solo en puntos que tienen valores de referencia válidos (es decir, puntos que estaban observados en los datos de mercado originales pero fueron artificialmente enmascarados durante el entrenamiento).

4.6. Pipeline del Conjunto de Datos

El pipeline del conjunto de datos está diseñado para reproducibilidad y eficiencia, separando el paso costoso de generación de superficies del ciclo rápido de entrenamiento.

4.6.1. Generación Offline

Las superficies se pre-generan y almacenan en disco. La función `generate_and_save()` toma una semilla aleatoria, el número de superficies y la especificación de la grilla, y luego:

1. Muestra parámetros de Heston (imponiendo la condición de Feller).
2. Calcula precios de opciones mediante la función característica.
3. Recupera volatilidades implícitas mediante Newton-Raphson.
4. Guarda el array de IV resultante como un archivo NumPy comprimido (`.npz`).
5. Guarda los metadatos (parámetros, especificación de la grilla, semilla) como JSON.

El uso de semillas fijas asegura que el mismo conjunto de datos se produce en cualquier máquina, lo que permite experimentos reproducibles.

4.6.2. Carga en Tiempo de Entrenamiento

La clase `VolSurfaceDataset` implementa la interfaz `Dataset` de PyTorch. Al inicializarse, carga las superficies pre-generadas en memoria. Cada llamada a `__getitem__` retorna una tupla de cuatro elementos:

1. **Entrada** $\mathbf{X} \in \mathbb{R}^{2 \times 8 \times 25}$: la superficie enmascarada de dos canales (Sección 4.1).
2. **Objetivo** $\Sigma \in \mathbb{R}^{1 \times 8 \times 25}$: la superficie de IV completa.
3. **Máscara** $\mathbf{M} \in \{0, 1\}^{8 \times 25}$: la máscara de observación.
4. **Máscara de objetivo** $\mathbf{T} \in \{0, 1\}^{8 \times 25}$: qué puntos tienen valores de referencia válidos (todos unos para datos sintéticos; refleja los datos faltantes naturales para datos reales).

La máscara se genera nueva en cada acceso, lo que aporta el aumento de datos descrito en la Sección 4.5. Para datos reales, la máscara sintética se combina (mediante AND lógico) con la máscara de datos faltantes naturales preexistente, asegurando que nunca se le pida al modelo reconstruir puntos para los cuales no existe valor de referencia.

Capítulo 5

Metodología

El capítulo anterior describió la representación de los datos, los conjuntos de datos sintético y real, y la estrategia de enmascaramiento que enmarcan la reconstrucción de superficies de volatilidad como un problema de aprendizaje supervisado. Este capítulo formaliza la tarea de reconstrucción, presenta las siete arquitecturas de modelos evaluadas en este trabajo, y describe el protocolo de entrenamiento, las restricciones de no arbitraje y las métricas de evaluación utilizadas a lo largo del análisis experimental.

5.1. Formulación del Problema

5.1.1. Reconstrucción como Aprendizaje Supervisado

La tarea de reconstrucción de superficies de volatilidad toma como entrada una superficie parcialmente observada junto con una máscara de observación y produce una superficie completa como salida. Formalmente, el modelo f_θ define el mapeo:

$$f_\theta : \mathbf{X} \in \mathbb{R}^{2 \times 8 \times 25} \longrightarrow \hat{\Sigma} \in \mathbb{R}^{1 \times 8 \times 25} \quad (5.1)$$

donde los dos canales de entrada son la superficie de volatilidad implícita enmascarada y la máscara binaria de observación (tal como se definieron en la Sección 4.1), y la salida es la superficie completa reconstruida.

5.1.2. Objetivo de Entrenamiento

La función de costo de entrenamiento se calcula exclusivamente sobre los puntos faltantes (aquellos donde $M_{ij} = 0$) para evitar que el modelo simplemente copie los valores observados a la salida:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{|\{(i, j) : M_{ij} = 0\}|} \sum_{\substack{i, j \\ M_{ij} = 0}} \left(\hat{\Sigma}_{ij} - \Sigma_{ij} \right)^2 \quad (5.2)$$

donde $\hat{\Sigma}_{ij}$ es la predicción del modelo y Σ_{ij} es la volatilidad implícita de referencia. Para datos reales, donde algunos puntos de la grilla carecen completamente de valor de referencia, una máscara de objetivo adicional restringe el costo a puntos con valores de referencia válidos (Sección 4.6).

5.1.3. Interfaz Abstracta

Todos los modelos de redes neuronales heredan de una clase base abstracta común, `SurfaceReconstructor`, que impone una interfaz estandarizada:

- **Entrada:** un tensor por lote de forma $(B, 2, N_\tau, N_K)$, donde B es el tamaño del lote.
- **Salida:** un tensor por lote de forma $(B, 1, N_\tau, N_K)$.

Esta interfaz común asegura que todos los modelos puedan entrenarse con el mismo pipeline de datos, optimizador y código de evaluación, haciendo las comparaciones estrictamente justas.

5.1.4. Conexión con *Inpainting* de Imágenes

Como se discutió en la Sección 3.8, la tarea de reconstrucción es estructuralmente análoga al *inpainting* de imágenes y a la autocodificación enmascarada: dada una grilla 2D parcialmente observada y una máscara, el modelo debe completar las entradas faltantes. Las diferencias clave con el *inpainting* genérico son que esta tarea es completamente supervisada con superficies de referencia, y que los datos tienen estructura específica del dominio (suavidad en la dimensión de strike y restricciones de monotonicidad en la dimensión de vencimiento) que puede explotarse mediante las penalizaciones de no arbitraje descritas en la Sección 5.5.

5.2. Arquitecturas de Modelos

La contribución central de esta tesis es una arquitectura Transformer codificador-decodificador para la reconstrucción de superficies de volatilidad, que se propone como una nueva aplicación de mecanismos de atención a este problema. Para evaluar rigurosamente esta propuesta, se compara contra cinco modelos de referencia de redes neuronales y un método paramétrico tradicional, abarcando un amplio rango de sesgos arquitectónicos: desde modelos sin estructura espacial (MLP) hasta modelos con estructura local (CNN, U-Net), modelos generativos (VAE, Conv VAE), y la parametrización SVI estándar de la industria. Todos los modelos neuronales se diseñan para tener aproximadamente 288.000 parámetros entrenables, lo que permite comparaciones justas con capacidad igualada (Sección 5.3).

Esta sección presenta primero las arquitecturas de referencia (Secciones 5.2.1–5.2.3), luego describe el modelo Transformer propuesto en detalle (Sección 5.2.4), las variantes generativas VAE (Sección 5.2.5), y finalmente la referencia paramétrica SVI (Sección 5.2.6).

5.2.1. Perceptrón Multicapa (MLP)

El MLP sirve como arquitectura de referencia sin ningún sesgo espacial. Trata la superficie de volatilidad como un vector plano, descartando toda la estructura espacial 2D.

Arquitectura. El tensor de entrada $\mathbf{X} \in \mathbb{R}^{2 \times 8 \times 25}$ se aplanar a un vector de dimensión $2 \times 8 \times 25 = 400$. Este vector pasa por tres capas ocultas de 256 unidades cada una con activaciones ReLU, seguidas de una capa de salida que proyecta a 200 dimensiones (8×25), que se redimensiona de vuelta a la grilla de la superficie:

$$\text{MLP} : \mathbb{R}^{400} \xrightarrow{\text{Linear} + \text{ReLU}}_{256} \xrightarrow{\text{Linear} + \text{ReLU}}_{256} \xrightarrow{\text{Linear} + \text{ReLU}}_{256} \xrightarrow{\text{Linear}} \mathbb{R}^{200} \quad (5.3)$$

Propiedad clave. El MLP es *espacialmente agnóstico*: no tiene mecanismo para explotar la estructura de grilla 2D ni las correlaciones espaciales entre strikes y vencimientos vecinos. Cualquier patrón espacial debe aprenderse enteramente a partir de los datos, lo que lo convierte en una cota inferior importante sobre lo que los sesgos arquitectónicos espaciales aportan.

Cantidad de parámetros. Aproximadamente 286k parámetros con dimensiones ocultas (256, 256, 256).

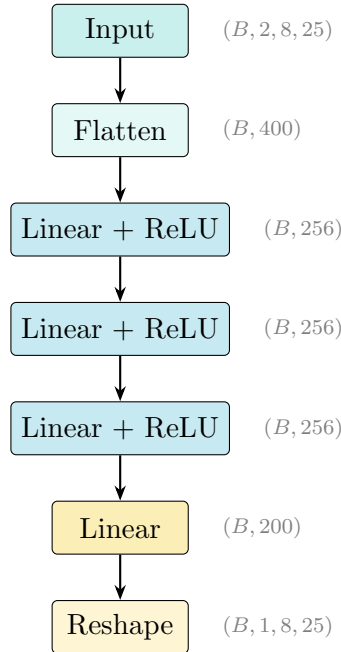


Figura 5.1: Arquitectura MLP. La grilla 2D de la superficie se aplanar a un vector, se procesa por tres capas ocultas completamente conectadas con activaciones ReLU, y se redimensiona de vuelta a la grilla de la superficie. Aproximadamente 286k parámetros.

5.2.2. Red Neuronal Convolutiva (CNN)

La CNN aplica filtros espaciales locales a la grilla 2D, explotando la continuidad del smile de volatilidad y la estructura temporal mediante pesos compartidos y campos receptivos locales.

Arquitectura. La red consiste en cinco capas convolucionales, todas utilizando kernels de 3×3 con *same-padding* (relleno = 1) para preservar las dimensiones espaciales:

$$\begin{aligned} & \text{Conv2d}(2 \rightarrow n_c, 3 \times 3, \text{pad} = 1) + \text{ReLU} \\ & \text{Conv2d}(n_c \rightarrow n_c, 3 \times 3, \text{pad} = 1) + \text{ReLU} \quad \times 3 \\ & \text{Conv2d}(n_c \rightarrow 1, 3 \times 3, \text{pad} = 1) \end{aligned} \quad (5.4)$$

donde $n_c = 104$ es el número de canales en cada capa oculta. No se utiliza *batch normalization*, ni *pooling*, ni activación en la capa final. Las dimensiones espaciales permanecen en 8×25 durante todo el paso forward.

Propiedad clave. La CNN tiene un *campo receptivo local*: con cinco capas de 3×3 , cada punto de salida agrega información de una vecindad de 11×11 (limitada por el tamaño de la grilla a los 8×25 completos a lo largo del eje de vencimiento). Esta estructura local es adecuada para la naturaleza suave y continua de las superficies de volatilidad, donde puntos cercanos en el espacio de strike y vencimiento tienen volatilidades implícitas similares.

Cantidad de parámetros. Aproximadamente 295k parámetros con $n_c = 104$.

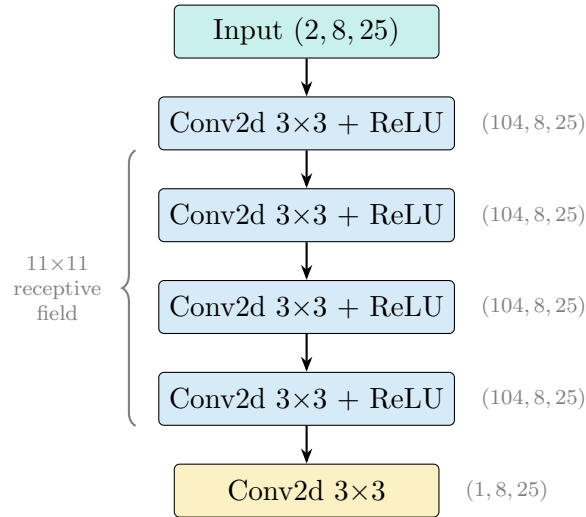


Figura 5.2: Arquitectura CNN. Cinco capas convolucionales con kernels de 3×3 y *same-padding* mantienen las dimensiones espaciales de 8×25 en todo el recorrido. No se utiliza *pooling* ni *batch normalization*. El campo receptivo crece a 11×11 a través de las capas sucesivas. Aproximadamente 295k parámetros con $n_c = 104$.

5.2.3. U-Net

La U-Net [36] combina extracción de características a múltiples escalas con conexiones de salto, permitiéndole capturar tanto el detalle local como el contexto global. Originalmente desarrollada para segmentación de imágenes biomédicas, la estructura codificador-decodificador es naturalmente adecuada para tareas de predicción densa como la reconstrucción de superficies.

Arquitectura. La red utiliza dos niveles de submuestreo, adaptados para la grilla pequeña de 8×25 (arquitecturas más profundas reducirían las dimensiones espaciales por debajo de una resolución útil). Cada nivel utiliza un *ConvBlock* consistente en dos convoluciones consecutivas de 3×3 con activaciones ReLU:

$$\text{ConvBlock}(c_{\text{in}}, c_{\text{out}}) : \text{Conv2d}(c_{\text{in}}, c_{\text{out}}) \rightarrow \text{ReLU} \rightarrow \text{Conv2d}(c_{\text{out}}, c_{\text{out}}) \rightarrow \text{ReLU} \quad (5.5)$$

La arquitectura completa con canales base $b = 24$ es:

Codificador:

$$\begin{aligned} \mathbf{e}_1 &= \text{ConvBlock}(2, b) & \text{forma: } (b, 8, 25) \\ \mathbf{p}_1 &= \text{MaxPool2d}(2)(\mathbf{e}_1) & \text{forma: } (b, 4, 12) \\ \mathbf{e}_2 &= \text{ConvBlock}(b, 2b) & \text{forma: } (2b, 4, 12) \\ \mathbf{p}_2 &= \text{MaxPool2d}(2)(\mathbf{e}_2) & \text{forma: } (2b, 2, 6) \end{aligned} \quad (5.6)$$

Cuello de botella:

$$\mathbf{z} = \text{ConvBlock}(2b, 4b) \quad \text{forma: } (4b, 2, 6) \quad (5.7)$$

Decodificador:

$$\begin{aligned} \mathbf{u}_1 &= \text{Upsample}(\mathbf{z}) \parallel \mathbf{e}_2 & \text{forma: } (4b + 2b, 4, 12) \\ \mathbf{d}_1 &= \text{ConvBlock}(6b, 2b) & \text{forma: } (2b, 4, 12) \\ \mathbf{u}_2 &= \text{Upsample}(\mathbf{d}_1) \parallel \mathbf{e}_1 & \text{forma: } (2b + b, 8, 25) \\ \mathbf{d}_2 &= \text{ConvBlock}(3b, b) & \text{forma: } (b, 8, 25) \end{aligned} \quad (5.8)$$

Salida:

$$\hat{\Sigma} = \text{Conv2d}(b, 1, 1 \times 1)(\mathbf{d}_2) \quad \text{forma: } (1, 8, 25) \quad (5.9)$$

El símbolo $\parallel$ denota concatenación a lo largo de la dimensión de canales. El submuestreo utiliza interpolación bilineal seguida de ajuste de tamaño para manejar las dimensiones espaciales que no son potencias de dos (ya que $25/2 = 12$ después del *pooling*, recuperar los 25 originales requiere interpolación al tamaño objetivo exacto). Las conexiones de salto del codificador al decodificador preservan la información espacial de alta frecuencia que de otro modo se perdería durante el submuestreo.

Cantidad de parámetros. Aproximadamente 265k parámetros con $b = 24$.

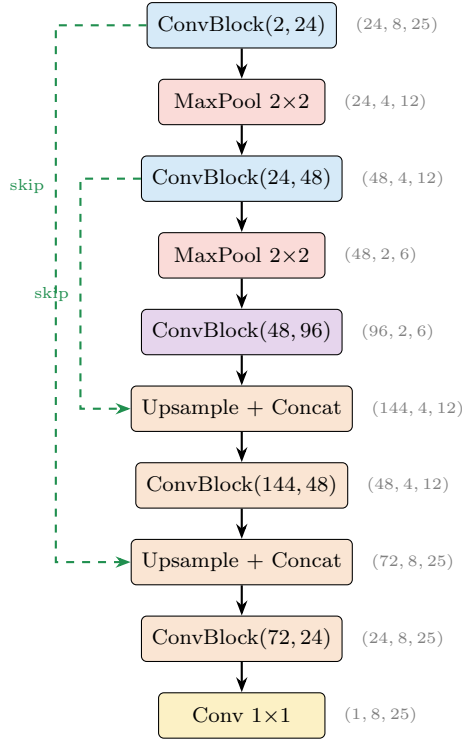


Figura 5.3: Arquitectura U-Net adaptada para la grilla de 8×25 de la superficie de volatilidad. Dos niveles de submuestreo con conexiones de salto (líneas verdes punteadas) balancean el detalle local (de las características del codificador) con el contexto global (del cuello de botella). Aproximadamente 265k parámetros con canales base $b = 24$.

5.2.4. Arquitectura Propuesta: Transformer Codificador-Decodificador

Se propone una arquitectura Transformer codificador-decodificador [39] para la reconstrucción de superficies de volatilidad. La idea central detrás de esta arquitectura es que el problema de reconstrucción tiene tres propiedades que son naturalmente abordadas por la atención:

1. **Patrones de observación variables.** El conjunto de puntos observados cambia de superficie a superficie. A diferencia de los modelos convolucionales, que procesan una grilla fija independientemente de dónde caigan las observaciones, el codificador Transformer procesa solo los tokens observados mediante auto-atención, adaptando su cómputo al patrón de observación específico.
2. **Dependencias de largo alcance.** La información en un punto de la superficie de volatilidad es informativa sobre puntos distantes: la curvatura del smile en tenores cortos restringe la forma del smile en tenores largos, y la estructura temporal ATM restringe el comportamiento en los extremos. La atención captura estas relaciones globales directamente, sin requerir que la información se propague a través de múltiples capas convolucionales.
3. **Uso explícito de coordenadas financieras.** Cada punto de la grilla tiene un significado físico: un vencimiento τ y un log-moneyness m específicos. Al codificar estas coordenadas como características de Fourier, el mecanismo de atención puede aprender patrones dependientes de la posición (por ejemplo, que los puntos de tenor corto son más informativos sobre la curvatura del smile que los de tenor largo).

Un trabajo concurrente relacionado de Zhang et al. [40] aplica un proceso neuronal basado en atención al *ajuste* de superficies de volatilidad implícita a partir de cotizaciones de mercado irregulares, utilizando superficies generadas con SABR [19] como prior estructural mediante meta-learning. Si bien la arquitectura subyacente de atención codificador-decodificador es similar en espíritu, las formulaciones del problema son distintas: VolNP ajusta a partir de un número variable de cotizaciones reales ruidosas en posiciones arbitrarias, mientras que nuestra formulación opera sobre una grilla discretizada fija con enmascaramiento aleatorio controlado, permitiendo la evaluación sistemática entre niveles de escasez. Además, se integran penalizaciones diferenciables de no arbitraje directamente en la función de costo (en lugar de verificar el cumplimiento a posteriori), y se realiza una comparación controlada contra cinco arquitecturas alternativas con cantidad de parámetros igualada para aislar la contribución específica de la atención a la calidad de reconstrucción.

Representación de Tokens

Los $8 \times 25 = 200$ puntos de la grilla se aplanan en una secuencia de 200 tokens. Cada token corresponde a un único punto (τ_i, m_j) de la superficie de volatilidad y porta dos tipos de información:

- **Valor:** la volatilidad implícita σ_{ij} en ese punto de la grilla (establecida en cero para los puntos faltantes).
- **Posición:** una codificación de coordenadas de la posición del token en el espacio (τ, m) .

Codificación Posicional Basada en Coordenadas

En lugar de utilizar codificaciones posicionales aprendidas o sinusoidales estándar indexadas por posición en la secuencia, se utilizan *características de Fourier basadas en coordenadas* inspiradas en Tancik et al. [38] que codifican directamente las coordenadas físicas (τ_i, m_j) de cada token. Mientras que la formulación original utiliza frecuencias muestreadas aleatoriamente, aquí se usan frecuencias determinísticas espaciadas logarítmicamente, que son suficientes para nuestro espacio de entrada de baja dimensionalidad (2D). Para una coordenada escalar x , la codificación con L bandas de frecuencia es:

$$\gamma(x) = [x, \sin(\pi x), \cos(\pi x), \sin(2\pi x), \cos(2\pi x), \dots, \sin(2^{L-1}\pi x), \cos(2^{L-1}\pi x)] \quad (5.10)$$

produciendo $1 + 2L$ características por coordenada. Para las coordenadas bidimensionales (τ_i, m_j) , la codificación completa concatena $\gamma(\tau_i)$ y $\gamma(m_j)$:

$$\mathbf{p}_{ij} = [\gamma(\tau_i) \parallel \gamma(m_j)] \in \mathbb{R}^{d_{\text{coord}}} \quad (5.11)$$

donde $d_{\text{coord}} = 2(1 + 2L)$. Con $L = 8$ bandas de frecuencia (frecuencias $\pi, 2\pi, \dots, 128\pi$), esto da $d_{\text{coord}} = 34$ características por token.

Esta codificación de coordenadas tiene dos ventajas sobre las codificaciones posicionales estándar. Primero, codifica directamente el *significado físico* de cada posición: el vencimiento τ y el log-moneyness m son cantidades continuas con significado financiero, no índices arbitrarios de secuencia. Segundo, las características sinusoidales multi-escala

permiten que el mecanismo de atención distinga posiciones a diferentes resoluciones, desde la distinción gruesa entre vencimientos cortos y largos hasta diferencias finas entre strikes adyacentes. La codificación de coordenadas se computa una vez a partir de la grilla fija y se almacena como un buffer no entrenable.

Para validar esta elección de diseño, se realiza un estudio de ablación (Sección 6.8) reemplazando la codificación de Fourier con embeddings posicionales aprendibles estándar de la misma dimensionalidad ($d_{\text{coord}} = 34$). La codificación de Fourier reduce el error de reconstrucción en aproximadamente un 18 %, confirmando que codificar coordenadas físicas es una contribución arquitectónica significativa y no una elección de diseño incidental.

Codificador

El codificador procesa solo los tokens *observados*. La entrada de cada token se forma concatenando su valor de IV (un escalar) con su codificación de coordenadas:

$$\mathbf{x}_{ij}^{\text{enc}} = [\sigma_{ij} \parallel \mathbf{p}_{ij}] \in \mathbb{R}^{1+d_{\text{coord}}} \quad (5.12)$$

Una proyección de entrada mapea esto a la dimensión del modelo:

$$\mathbf{h}_{ij}^{(0)} = \text{Linear}(1 + d_{\text{coord}}, d_{\text{model}})(\mathbf{x}_{ij}^{\text{enc}}) \quad (5.13)$$

Los tokens proyectados pasan por $N_{\text{enc}} = 3$ capas de codificador Transformer. Cada capa consiste en auto-atención multi-cabeza seguida de una red feed-forward posición por posición, con normalización pre-capas y conexiones residuales:

$$\mathbf{a}^{(\ell)} = \mathbf{h}^{(\ell-1)} + \text{MHA}(\text{LN}(\mathbf{h}^{(\ell-1)}), \text{LN}(\mathbf{h}^{(\ell-1)}), \text{LN}(\mathbf{h}^{(\ell-1)})) \quad (5.14)$$

$$\mathbf{h}^{(\ell)} = \mathbf{a}^{(\ell)} + \text{FFN}(\text{LN}(\mathbf{a}^{(\ell)})) \quad (5.15)$$

donde MHA denota atención multi-cabeza con $h = 4$ cabezas y FFN es una red de dos capas con dimensión oculta $d_{\text{ff}} = 256$ y activación GELU. Los tokens faltantes se excluyen del cómputo de auto-atención mediante una `src_key_padding_mask` que marca las posiciones donde $M_{ij} = 0$.

Decodificador

El decodificador toma *todos* los 200 tokens como consultas, tanto observados como faltantes, y realiza atención cruzada con la memoria de salida del codificador. Para los tokens faltantes, el valor de IV en la consulta se establece en cero (una inicialización fija, no un token de máscara aprendido). La entrada de consulta es:

$$\mathbf{x}_{ij}^{\text{dec}} = [\sigma_{ij} \cdot M_{ij} \parallel \mathbf{p}_{ij}] \in \mathbb{R}^{1+d_{\text{coord}}} \quad (5.16)$$

proyectada a través de una capa lineal separada a d_{model} dimensiones.

El decodificador consiste en $N_{\text{dec}} = 2$ capas de decodificador Transformer. Cada capa aplica auto-atención entre todos los tokens de consulta, luego atención cruzada a la memoria del codificador:

$$\mathbf{a}_{\text{self}}^{(\ell)} = \mathbf{q}^{(\ell-1)} + \text{MHA}_{\text{self}}(\text{LN}(\mathbf{q}^{(\ell-1)})) \quad (5.17)$$

$$\mathbf{a}_{\text{cross}}^{(\ell)} = \mathbf{a}_{\text{self}}^{(\ell)} + \text{MHA}_{\text{cross}}(\text{LN}(\mathbf{a}_{\text{self}}^{(\ell)}), \text{LN}(\mathbf{m}_{\text{enc}})) \quad (5.18)$$

$$\mathbf{q}^{(\ell)} = \mathbf{a}_{\text{cross}}^{(\ell)} + \text{FFN}(\text{LN}(\mathbf{a}_{\text{cross}}^{(\ell)})) \quad (5.19)$$

donde $\mathbf{m}_{\text{enc}}$ es la salida del codificador. Una `memory key padding mask` asegura que la atención cruzada solo atienda a posiciones donde existen observaciones. Este mecanismo es la clave de la capacidad de reconstrucción del Transformer: los tokens faltantes agregan información de todos los tokens observados, ponderada por pesos de atención aprendidos que pueden capturar tanto correlaciones locales (strikes cercanos) como dependencias de largo alcance (entre vencimientos).

Proyección de Salida

Una capa lineal final mapea la representación de cada token desde d_{model} a un escalar de volatilidad implícita predicha:

$$\hat{\sigma}_{ij} = \text{Linear}(d_{\text{model}}, 1)(\mathbf{q}_{ij}^{(N_{\text{dec}})}) \quad (5.20)$$

Las 200 predicciones escalares se redimensionan de vuelta a la grilla $(1, 8, 25)$ de la superficie.

Cantidad de parámetros. Aproximadamente 288k parámetros con $d_{\text{model}} = 64$, $N_{\text{enc}} = 3$, $N_{\text{dec}} = 2$, $h = 4$ y $d_{\text{ff}} = 256$.

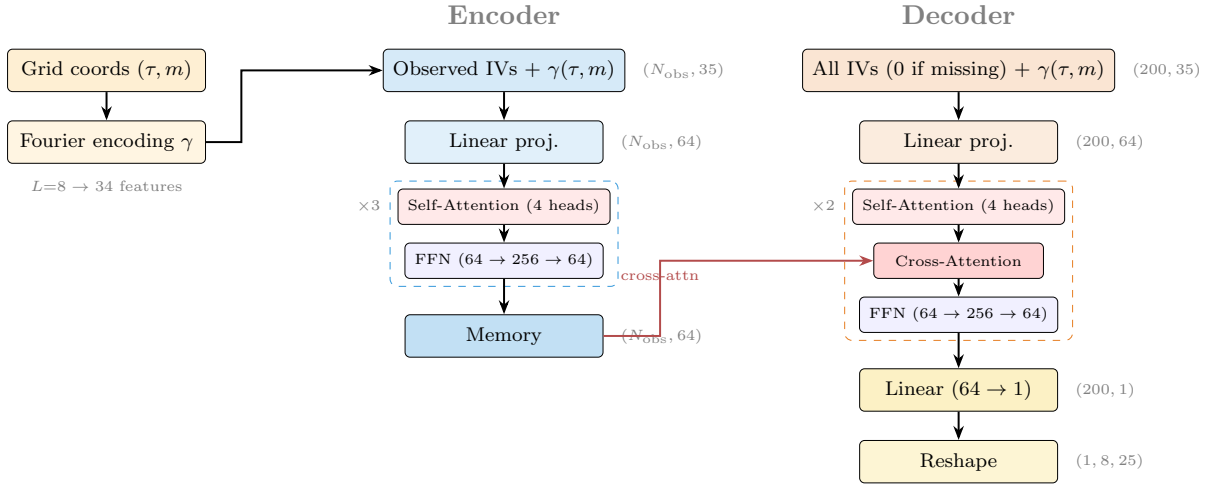


Figura 5.4: Arquitectura Transformer para reconstrucción de superficies de volatilidad. Izquierda: la codificación de Fourier basada en coordenadas mapea las posiciones de la grilla (τ, m) a vectores de características de 34 dimensiones. Centro: el codificador procesa solo los tokens observados a través de 3 capas de auto-atención. Derecha: el decodificador toma los 200 tokens como consultas y realiza atención cruzada con la memoria del codificador, reconstruyendo la superficie completa. Aproximadamente 288k parámetros.

5.2.5. Autoencoders Variacionales

Los autoencoders variacionales [28] aprenden una representación latente de baja dimensionalidad de la variedad de superficies de volatilidad. A diferencia de los modelos discriminativos anteriores, que mapean directamente entradas a salidas, los VAEs modelan la distribución de los datos y reconstruyen superficies proyectando sobre la variedad aprendida. Se evalúan dos variantes: un VAE completamente conectado (FC) y un VAE convolucional (Conv).

FC VAE

El VAE completamente conectado aplana la superficie de entrada a un vector y lo procesa a través de una arquitectura codificador-decodificador simétrica:

Codificador:

$$\begin{aligned} \mathbf{h}_1 &= \text{ELU}(\text{Linear}(400, 288)(\text{flatten}(\mathbf{X}))) \\ \mathbf{h}_2 &= \text{ELU}(\text{Linear}(288, 144)(\mathbf{h}_1)) \\ \mathbf{h}_3 &= \text{ELU}(\text{Linear}(144, 72)(\mathbf{h}_2)) \\ \boldsymbol{\mu} &= \text{Linear}(72, d_z)(\mathbf{h}_3), \quad \log \boldsymbol{\sigma}^2 = \text{Linear}(72, d_z)(\mathbf{h}_3) \end{aligned} \quad (5.21)$$

Reparametrización:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (5.22)$$

Durante la inferencia, el muestreo estocástico se reemplaza por la media determinística de la posterior $\mathbf{z} = \boldsymbol{\mu}$.

Decodificador:

$$\begin{aligned} \mathbf{g}_1 &= \text{ELU}(\text{Linear}(d_z, 72)(\mathbf{z})) \\ \mathbf{g}_2 &= \text{ELU}(\text{Linear}(72, 144)(\mathbf{g}_1)) \\ \mathbf{g}_3 &= \text{ELU}(\text{Linear}(144, 288)(\mathbf{g}_2)) \\ \hat{\boldsymbol{\Sigma}} &= \text{reshape}(\text{Linear}(288, 200)(\mathbf{g}_3)) \end{aligned} \quad (5.23)$$

El codificador comprime progresivamente la entrada de 400 dimensiones a través de capas de ancho 288, 144 y 72 hasta el espacio latente, mientras que el decodificador refleja esta estructura en reversa. Se utilizan activaciones ELU (en lugar de ReLU) porque su gradiente no nulo para entradas negativas mejora el flujo de gradientes a través de las capas estrechas del cuello de botella. La dimensión latente es $d_z = 32$, comprimiendo la superficie de 200 dimensiones a una representación de 32 dimensiones. Aproximadamente 285k parámetros.

Conv VAE

El VAE convolucional explota la estructura espacial 2D de la grilla de la superficie utilizando convoluciones con stride para el submuestreo e interpolación bilineal para el sobremuestreo:

Codificador:

$$\begin{aligned} \mathbf{e}_1 &= \text{ReLU}(\text{Conv2d}(2, b, 3 \times 3, \text{stride} = 1, \text{pad} = 1)) & (b, 8, 25) \\ \mathbf{e}_2 &= \text{ReLU}(\text{Conv2d}(b, 2b, 3 \times 3, \text{stride} = 2, \text{pad} = 1)) & (2b, 4, 13) \\ \mathbf{e}_3 &= \text{ReLU}(\text{Conv2d}(2b, 4b, 3 \times 3, \text{stride} = 2, \text{pad} = 1)) & (4b, 2, 7) \end{aligned} \quad (5.24)$$

La salida del codificador se aplana a $4b \times 2 \times 7$ dimensiones, a partir de las cuales dos capas lineales producen $\boldsymbol{\mu}$ y $\log \boldsymbol{\sigma}^2$ de la distribución latente. La reparametrización sigue la Ecuación (5.22).

Decodificador:

$$\begin{aligned}
\mathbf{g}_0 &= \text{reshape}(\text{ReLU}(\text{Linear}(d_z, 4b \times 2 \times 7))) & (4b, 2, 7) \\
\mathbf{g}_1 &= \text{ReLU}(\text{Conv2d}(4b, 2b, 3 \times 3, \text{pad} = 1)) \circ \text{Upsample}(4, 13) & (2b, 4, 13) \\
\mathbf{g}_2 &= \text{ReLU}(\text{Conv2d}(2b, b, 3 \times 3, \text{pad} = 1)) \circ \text{Upsample}(8, 25) & (b, 8, 25) \\
\hat{\Sigma} &= \text{Conv2d}(b, 1, 3 \times 3, \text{pad} = 1) & (1, 8, 25) \quad (5.25)
\end{aligned}$$

El sobremuestreo utiliza interpolación bilineal a tamaños objetivo, evitando los artefactos de tablero de ajedrez asociados con las convoluciones transpuestas. Con canales base $b = 32$ y dimensión latente $d_z = 16$, el Conv VAE tiene aproximadamente 273k parámetros.

Función de Costo de Entrenamiento: Cota Inferior de Evidencia

Ambas variantes de VAE se entrenan maximizando la cota inferior de evidencia (ELBO), que se descompone en un término de reconstrucción y un término de regularización:

$$\mathcal{L}_{\text{ELBO}} = \mathcal{L}_{\text{MSE}} + \beta \cdot D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{X}) \parallel p(\mathbf{z})) \quad (5.26)$$

donde $\mathcal{L}_{\text{MSE}}$ es la función de costo de reconstrucción estándar (Ecuación (5.2)) y la divergencia KL para el codificador gaussiano tiene la expresión cerrada:

$$D_{\text{KL}} = -\frac{1}{2} \sum_{k=1}^{d_z} (1 + \log \sigma_k^2 - \mu_k^2 - \sigma_k^2) \quad (5.27)$$

El factor de ponderación $\beta = 10^{-4}$ mantiene el término KL pequeño respecto al costo de reconstrucción, priorizando la reconstrucción exacta de la superficie sobre la regularidad del espacio latente.

Optimización del Espacio Latente en Inferencia

Un enfoque directo para la reconstrucción con un VAE sería codificar la observación parcial y decodificar el vector latente resultante. Sin embargo, codificar una superficie parcialmente observada (donde los valores faltantes se reemplazan con ceros) distorsiona la estimación de la posterior. En su lugar, se utiliza *optimización del espacio latente*: dada una observación parcial, se busca el vector latente $\mathbf{z}^*$ que minimiza el error de reconstrucción sobre los puntos observados:

$$\mathbf{z}^* = \arg \min_{\mathbf{z}} \sum_{\substack{i,j \\ M_{ij}=1}} (\text{decode}(\mathbf{z})_{ij} - \sigma_{ij}^{\text{obs}})^2 \quad (5.28)$$

Esto se resuelve mediante descenso de gradiente: $\mathbf{z}$ se inicializa desde la media de la posterior del codificador y luego se optimiza con Adam durante 200 pasos con una tasa de aprendizaje de 0,01. Los pesos del decodificador se mantienen congelados durante esta optimización. La reconstrucción final $\text{decode}(\mathbf{z}^*)$ proyecta la observación incompleta sobre la variedad aprendida de superficies completas, completando naturalmente los valores faltantes con completaciones suaves y plausibles.

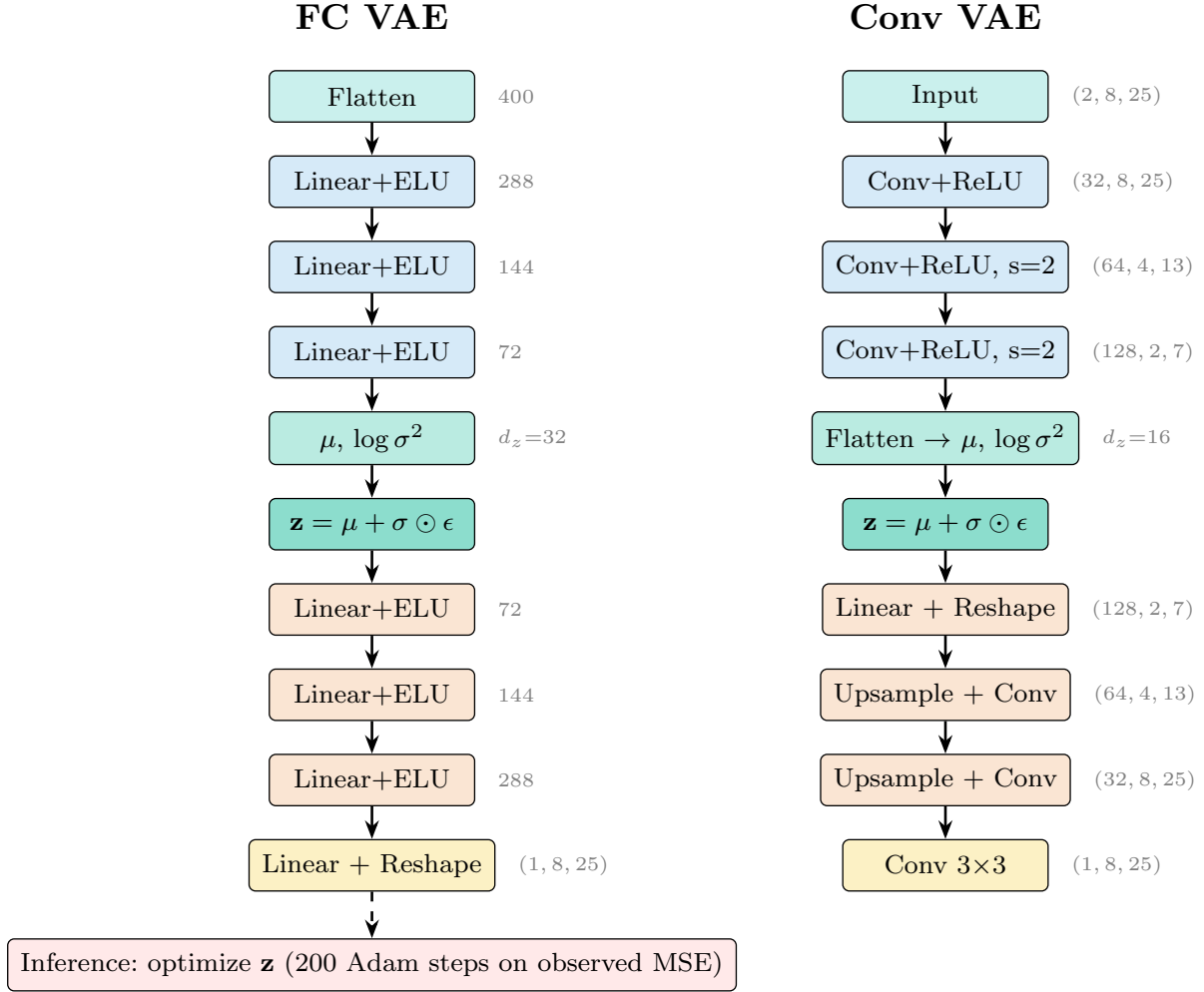


Figura 5.5: Arquitecturas VAE. Izquierda: FC VAE con codificador-decodificador completamente conectado y dimensión latente $d_z = 32$. Derecha: Conv VAE con convoluciones con stride y dimensión latente $d_z = 16$. Ambos utilizan muestreo reparametrizado durante el entrenamiento. En inferencia, el vector latente $\mathbf{z}$ se optimiza mediante 200 pasos de Adam para minimizar el error de reconstrucción sobre los puntos observados, proyectando las observaciones parciales sobre la variedad aprendida de superficies.

5.2.6. SVI como Referencia

La parametrización SVI (Sección 2.7) sirve como referencia tradicional (sin aprendizaje), utilizando la fórmula SVI cruda (Ecuación (2.19)) con cinco parámetros por corte de vencimiento.

Procedimiento de ajuste. Para cada uno de los 8 cortes de vencimiento de forma independiente, los 5 parámetros SVI se optimizan para minimizar el error cuadrático medio entre las varianzas implícitas parametrizadas y las observadas en los strikes observados, utilizando el optimizador L-BFGS-B [10] con restricciones de contorno ($b \geq 0$, $|\rho| \leq 1$, $\sigma > 0$). Esto da $5 \times 8 = 40$ parámetros por superficie. La limitación clave es que SVI ajusta cada corte de forma independiente sin compartir información entre vencimientos, lo que se vuelve cada vez más severo a medida que la escasez crece (Capítulo 6).

5.3. Normalización de la Cantidad de Parámetros

Una comparación justa entre arquitecturas requiere controlar la capacidad del modelo. Un modelo más grande podría lograr mejor rendimiento simplemente por tener más parámetros para memorizar patrones de entrenamiento, ocultando la contribución de los sesgos arquitectónicos. Por lo tanto, se normalizan todos los modelos neuronales a aproximadamente 288.000 parámetros entrenables ajustando el ancho (número de unidades ocultas o canales) de cada arquitectura.

Tabla 5.1: Cantidad de parámetros para todos los modelos, ajustados a aproximadamente 288k. El modelo de referencia SVI es fundamentalmente diferente (optimización por superficie, sin parámetros neuronales entrenables).

Modelo	Parámetro de ancho clave	Parámetros
MLP	hidden_dims = (256, 256, 256)	~286k
CNN	n_channels = 104	~295k
U-Net	base_channels = 24	~265k
Transformer	d_model = 64	~288k
FC VAE	hidden_dims = (288, 144, 72), latent = 32	~285k
Conv VAE	base_channels = 32, latent = 16	~273k
SVI	5 params $\times$ 8 cortes	40/superficie

El modelo de referencia SVI no puede compararse de forma significativa en cantidad de parámetros: tiene 40 parámetros por superficie (reoptimizados desde cero para cada superficie de test), mientras que los modelos neuronales tienen ~288k parámetros compartidos que generalizan a todas las superficies. Esto refleja una diferencia fundamental de enfoque: SVI optimiza sus parámetros desde cero para cada superficie individual, mientras que los modelos neuronales aprenden parámetros compartidos una única vez durante el entrenamiento y luego los aplican a cualquier superficie nueva sin reoptimización.

5.4. Protocolo de Entrenamiento

Todos los modelos se implementan en PyTorch [34] y se entrenan con un protocolo estandarizado para asegurar que las diferencias de rendimiento reflejen elecciones arquitectónicas en lugar de la configuración de entrenamiento.

5.4.1. Optimización

Tabla 5.2: Hiperparámetros de entrenamiento. Todos los modelos utilizan la misma configuración excepto donde se indica.

Hiperparámetro	Valor
Optimizador	Adam
Tasa de aprendizaje	10^{-3} (Transformer: 10^{-4})
<i>Weight decay</i>	0
Tamaño de lote	32
Máximo de iteraciones	500
Paciencia de parada temprana	30 iteraciones
<i>Schedule</i> de tasa de aprendizaje	Ninguno (constante)
Función de costo	MSE (VAEs: ELBO con $\beta = 10^{-4}$)

El Transformer utiliza una tasa de aprendizaje menor (10^{-4}) porque las arquitecturas basadas en atención son sensibles a actualizaciones de gradiente grandes, particularmente en las etapas tempranas del entrenamiento cuando los patrones de atención aún no están establecidos. Se evitan deliberadamente los *schedules* de tasa de aprendizaje (como *cosine annealing*) para eliminar un factor de confusión: diferentes arquitecturas pueden beneficiarse desigualmente de tasas de aprendizaje programadas, dificultando la atribución de diferencias de rendimiento a la arquitectura misma.

5.4.2. Parada Temprana

El entrenamiento termina cuando el costo de validación no ha mejorado durante 30 iteraciones consecutivas. Se retiene el checkpoint del modelo con el menor costo de validación para la evaluación. Esto previene el sobreajuste mientras permite tiempo de entrenamiento suficiente para que todas las arquitecturas converjan. En la práctica, la mayoría de los modelos convergen dentro de 100–200 iteraciones; el máximo de 500 rara vez se alcanza.

5.4.3. Protocolo de Ajuste Fino

Para los experimentos de *transfer learning* (datos sintéticos $\rightarrow$ reales), los modelos preentrenados se ajustan finamente con tasas de aprendizaje reducidas para preservar las representaciones aprendidas a partir de datos sintéticos:

Tabla 5.3: Hiperparámetros de ajuste fino para *transfer learning* de datos sintéticos a reales.

Modelo	Tasa de aprendizaje	Cambios adicionales
MLP, CNN, U-Net	10^{-5}	Ninguno
Transformer	10^{-4}	<i>Dropout</i> 0,0 $\rightarrow$ 0,05

El Transformer retiene una tasa de aprendizaje de ajuste fino más alta y agrega un *dropout* pequeño (0,05) para prevenir el sobreajuste al conjunto de datos real más pequeño

(2.912 superficies de entrenamiento vs. 8.000 sintéticas). Todos los demás parámetros de entrenamiento (tamaño de lote, paciencia, etc.) permanecen sin cambios.

5.5. Restricciones de No Arbitraje

Las superficies de volatilidad están sujetas a condiciones de no arbitraje estático que deben cumplirse para que los precios de opciones sean consistentes con una única medida de probabilidad subyacente. Las violaciones de estas condiciones crean oportunidades de ganancia sin riesgo e indican que la superficie no es económicamente significativa. Se incorporan dos restricciones clave como términos de penalización diferenciables en la función de costo de entrenamiento.

5.5.1. Restricción de Calendar Spread

La condición de *calendar spread* requiere que la varianza implícita total $w(m, \tau) = \sigma^2(m, \tau) \cdot \tau$ sea no decreciente en el vencimiento τ para cada log-moneyness fijo m :

$$w(m, \tau_1) \leq w(m, \tau_2) \quad \text{para todo } \tau_1 < \tau_2 \text{ y todo } m \quad (5.29)$$

La violación de esta condición implica varianza forward negativa, lo que permitiría una ganancia sin riesgo vendiendo una opción de menor plazo y comprando una de mayor plazo al mismo strike.

La penalización diferenciable se calcula usando diferencias forward a lo largo del eje de vencimiento:

$$\mathcal{P}_{\text{cal}} = \frac{1}{N} \sum_{i,j} [\text{ReLU}(-\Delta w_{ij})]^2, \quad \Delta w_{ij} = w(\tau_{i+1}, m_j) - w(\tau_i, m_j) \quad (5.30)$$

donde N es el número de pares de vencimientos adyacentes multiplicado por el número de strikes. El ReLU asegura que solo las violaciones (diferencias negativas) contribuyan a la penalización, y el cuadrado hace que la penalización sea suave y diferenciable en la frontera.

5.5.2. Restricción de Butterfly Spread

La condición de *butterfly spread* requiere que la varianza implícita total sea convexa en log-moneyness m para cada vencimiento fijo τ :

$$\frac{\partial^2 w}{\partial m^2}(m, \tau) \geq 0 \quad \text{para todo } m \text{ y } \tau \quad (5.31)$$

La violación de esta condición implica una densidad de probabilidad negativa en la distribución neutral al riesgo, lo que carece de sentido económico y permite arbitraje mediante butterfly spreads.

Para nuestra grilla de log-moneyness con espaciado no uniforme, la segunda derivada se aproxima mediante diferencias finitas con espaciado desigual. Sean $h_j^- = m_j - m_{j-1}$ y $h_j^+ = m_{j+1} - m_j$ los tamaños de paso izquierdo y derecho. La diferencia finita de segundo orden es:

$$\left. \frac{\partial^2 w}{\partial m^2} \right|_{m_j} \approx \frac{2}{h_j^- + h_j^+} \left(\frac{w_{j-1}}{h_j^-} - w_j \left(\frac{1}{h_j^-} + \frac{1}{h_j^+} \right) + \frac{w_{j+1}}{h_j^+} \right) \quad (5.32)$$

La penalización sigue la misma estructura que la penalización de calendar:

$$\mathcal{P}_{\text{but}} = \frac{1}{N} \sum_{i,j} \left[\text{ReLU} \left(-\frac{\partial^2 w}{\partial m^2} \Big|_{(\tau_i, m_j)} \right) \right]^2 \quad (5.33)$$

5.5.3. Función de Costo Combinada con Restricciones

La función de costo completa con penalizaciones de no arbitraje es:

$$\mathcal{L} = \mathcal{L}_{\text{MSE}} + \lambda_{\text{cal}} \cdot \mathcal{P}_{\text{cal}} + \lambda_{\text{but}} \cdot \mathcal{P}_{\text{but}} \quad (5.34)$$

En la práctica, se establece $\lambda_{\text{cal}} = 0$ porque las violaciones de calendar spread son despreciables en todos los modelos (tasas inferiores al 0,1 %, ya que el eje suave de vencimiento del modelo de Heston y la grilla relativamente gruesa de 8 tenores hacen que las violaciones de calendar sean raras). La función de costo con restricciones efectiva es por lo tanto:

$$\mathcal{L} = \mathcal{L}_{\text{MSE}} + \lambda \cdot \mathcal{P}_{\text{but}} \quad (5.35)$$

donde λ controla el compromiso entre exactitud de reconstrucción y suavidad libre de arbitraje. El impacto de variar λ se analiza en el Capítulo 7.

Ambas penalizaciones son completamente diferenciables y se calculan sobre la superficie *predicha* (no sobre los valores de referencia), aportando una señal de gradiente que orienta al modelo hacia salidas libres de arbitraje durante el entrenamiento. Es importante destacar que las penalizaciones operan sobre la salida del modelo independientemente de qué puntos fueron observados o faltantes, fomentando la coherencia global de la superficie.

5.6. Métricas de Evaluación

El rendimiento de los modelos se evalúa a lo largo de dos ejes: *exactitud de reconstrucción* (cuán bien el modelo completa los valores faltantes) y *consistencia económica* (si la superficie reconstruida respeta las condiciones de no arbitraje).

5.6.1. Exactitud de Reconstrucción

La métrica principal es la raíz del error cuadrático medio sobre los puntos faltantes:

$$\text{RMSE}_{\text{miss}} = \sqrt{\frac{1}{|\mathcal{M}|} \sum_{(i,j) \in \mathcal{M}} (\hat{\Sigma}_{ij} - \Sigma_{ij})^2} \quad (5.36)$$

donde $\mathcal{M} = \{(i, j) : M_{ij} = 0\}$ es el conjunto de puntos faltantes. Esta es la métrica más informativa porque mide directamente la capacidad del modelo para predecir valores que nunca ha visto, sin contaminación de los puntos observados trivialmente exactos.

Métricas adicionales ofrecen perspectivas complementarias:

- RMSE_{obs} : RMSE sobre puntos observados, midiendo cuán bien el modelo preserva los valores conocidos.
- MAE: Error absoluto medio, menos sensible a valores atípicos que RMSE.

- **MaxErr:** Error absoluto máximo sobre todos los puntos, capturando el comportamiento en el peor caso.

Todas las métricas se calculan por superficie y luego se promedian sobre el conjunto de test. También se reportan distribuciones de RMSE por superficie para evaluar la variabilidad: un modelo con RMSE promedio bajo pero alta varianza puede producir reconstrucciones ocasionalmente deficientes que son inaceptables en la práctica.

5.6.2. Análisis Regional

La dificultad de reconstrucción varía a lo largo de la superficie de volatilidad: las regiones muy *out of the money* tienen datos menos líquidos y gradientes más pronunciados, mientras que las regiones cercanas al ATM son más densamente observadas y suaves. Para capturar esta heterogeneidad, se particiona la grilla en regiones a lo largo de ambos ejes.

Tabla 5.4: Regiones de moneyness y tenor para el análisis regional de errores. Los límites de log-moneyness se eligen para separar puts muy OTM, la zona cercana al ATM y calls muy OTM.

Regiones de Moneyness		
Región	Rango de log-moneyness	Interpretación
Put muy OTM	$m < -0,2$	Extremo izquierdo
Put OTM	$-0,2 \leq m < -0,05$	Ala izquierda
ATM	$-0,05 \leq m \leq 0,05$	At-the-money
Call OTM	$0,05 < m \leq 0,2$	Ala derecha
Call muy OTM	$m > 0,2$	Extremo derecho
Regiones de Tenor		
Región	Rango de tenor	Puntos de la grilla
Corto	$\tau \leq 0,25$ año	$\tau \in \{0,08; 0,17; 0,25\}$
Medio	$0,25 < \tau \leq 1,0$ año	$\tau \in \{0,5; 0,75; 1,0\}$
Largo	$\tau > 1,0$ año	$\tau \in \{1,5; 2,0\}$

El RMSE se calcula por separado dentro de cada región, produciendo una matriz de errores de 5×3 que revela dónde sobresale o tiene dificultades cada modelo.

5.6.3. Métricas de Violación de Arbitraje

La consistencia económica se evalúa utilizando las mismas condiciones de calendar y butterfly descritas en la Sección 5.5, aplicadas a las superficies predichas en tiempo de evaluación. Las métricas clave son:

- **Tasa de violación:** La fracción de puntos de la grilla (o pares adyacentes) donde la condición es violada. Se reporta por separado para las restricciones de calendar y butterfly.
- **Magnitud media de violación:** El tamaño promedio de las violaciones (en unidades de varianza total), calculado solo sobre los puntos que violan la condición.

- **Severidad esperada:** El producto de la tasa de violación y la magnitud media de violación:

$$S = r_{\text{viol}} \times \bar{v} \quad (5.37)$$

Esta métrica combinada captura tanto la frecuencia como la severidad con que un modelo viola las condiciones de no arbitraje. Un modelo con alta tasa de violación pero violaciones mínimas puede ser preferible a uno con violaciones raras pero grandes; la severidad esperada unifica estas dimensiones.

Referencia de valores de referencia. Las superficies de referencia exhiben una tasa de violación de butterfly no nula de aproximadamente 8,6 %, que surge de la discretización del modelo de Heston continuo sobre una grilla finita. La aproximación por diferencias finitas de $\partial^2 w / \partial m^2$ sobre nuestra grilla de 25 puntos en log-moneyness puede registrar violaciones numéricas pequeñas incluso para el modelo de Heston, que es analíticamente libre de arbitraje. Esto establece un piso por debajo del cual las violaciones son atribuibles a la discretización de la grilla en lugar de al error del modelo, y sirve como línea de referencia en los análisis de Pareto del Capítulo 7.

Capítulo 6

Experimentos y Resultados

El capítulo anterior describió las siete arquitecturas de modelos, el protocolo de entrenamiento y las métricas de evaluación. Este capítulo presenta los resultados experimentales a lo largo de múltiples dimensiones: exactitud de reconstrucción de referencia sobre datos sintéticos, robustez ante escasez creciente de datos, patrones regionales de error a lo largo de la superficie de volatilidad, rendimiento sobre opciones reales de SPY, dinámica de *transfer learning*, interpretabilidad de la atención y eficiencia computacional. El análisis de restricciones de no arbitraje se difiere al Capítulo 7.

6.1. Resultados de Referencia sobre Datos Sintéticos

Se comienza con la comparación central: los siete modelos evaluados sobre el conjunto de test sintético de Heston (1.000 superficies) con el 30 % de los puntos de la grilla enmascarados aleatoriamente.

6.1.1. Rendimiento General

La Tabla 6.1 resume la exactitud de reconstrucción y las tasas de violación de arbitraje para todos los modelos.

Tabla 6.1: Rendimiento de referencia sobre el conjunto de test sintético de Heston (1.000 superficies, 30 % faltante). Los modelos están ordenados por RMSE sobre puntos faltantes. Todos los modelos neuronales tienen $\sim 288k$ parámetros; SVI tiene 40 parámetros por superficie. Las columnas de calendar y butterfly reportan tasas de violación.

Model	Params	RMSE _{miss}	MAE	Max Error	Calendar	Butterfly
Transformer	288k	0.0045	0.0012	0.2005	0.0 %	45.0 %
U-Net	265k	0.0048	0.0008	0.2641	0.0 %	44.9 %
CNN	295k	0.0049	0.0011	0.2464	0.0 %	45.1 %
MLP	286k	0.0057	0.0025	0.2380	0.0 %	44.3 %
SVI	40/surf	0.0065	0.0008	0.3251	0.0 %	0.0 %
FC VAE	285k	0.0070	0.0030	0.2568	0.0 %	45.0 %
Conv VAE	273k	0.0072	0.0024	0.3316	0.0 %	43.2 %

El Transformer alcanza la mejor exactitud de reconstrucción con un RMSE de 0,0045 sobre puntos faltantes, seguido de cerca por U-Net (0,0048) y CNN (0,0049). El MLP, sin

sesgo espacial, queda en 0,0057, un 27 % peor que el Transformer. El modelo de referencia SVI alcanza 0,0065, ubicándose entre el MLP y las variantes de VAE. Ambas arquitecturas de VAE quedan en último lugar entre los modelos neuronales (FC VAE: 0,0070, Conv VAE: 0,0072), sugiriendo que el cuello de botella de información impuesto por el espacio latente de baja dimensionalidad ($d_z = 32$) limita la fidelidad de la reconstrucción.

La Figura 6.1 visualiza esta comparación de RMSE.

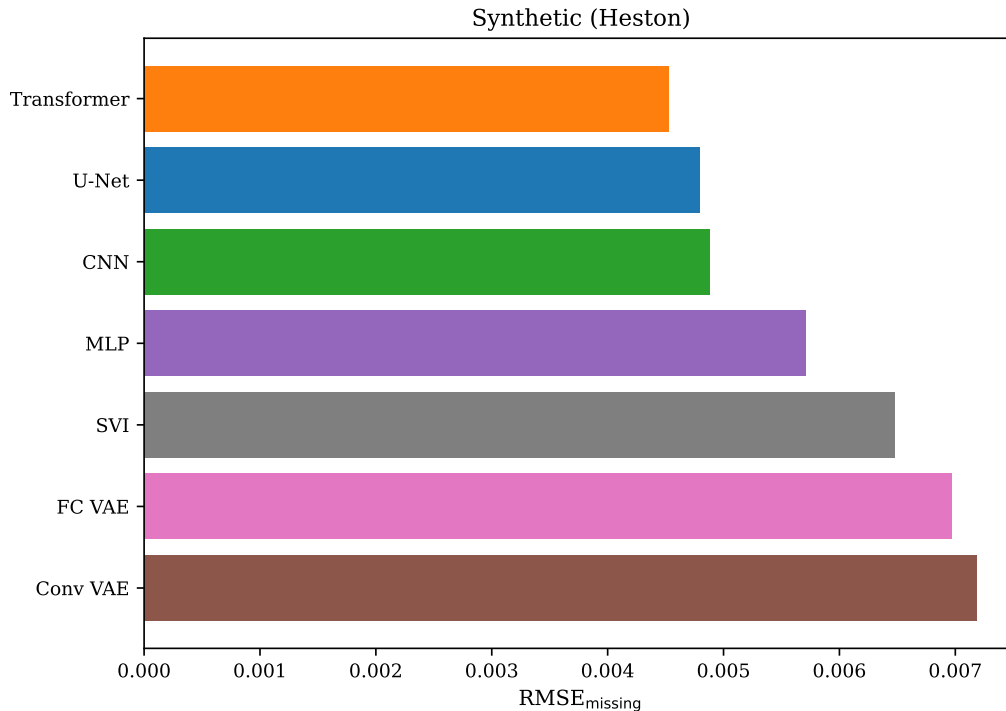


Figura 6.1: RMSE sobre puntos faltantes para todos los modelos en datos sintéticos de Heston (30 % faltante). Los modelos están ordenados por RMSE ascendente. Transformer y U-Net lideran, seguidos de cerca por CNN.

Para evaluar la estabilidad de estos rankings, se reentrenan los cuatro mejores modelos (CNN, U-Net, Transformer, MLP) con tres semillas aleatorias diferentes. La Tabla 6.2 reporta la media y desviación estándar del RMSE sobre puntos faltantes.

Tabla 6.2: Validación con múltiples semillas (3 semillas). RMSE sobre puntos faltantes reportado como media $\pm$ desviación estándar. El Transformer y la U-Net son estadísticamente indistinguibles en exactitud de reconstrucción. La U-Net muestra la menor varianza entre semillas.

Model	RMSE _{miss} (mean $\pm$ std)	Original	n
CNN	0,0049 $\pm$ 0,0002	0.0049	3
U-Net	0,0045 $\pm$ 0,0000	0.0048	3
Transformer	0,0045 $\pm$ 0,0004	0.0045	3
MLP	0,0054 $\pm$ 0,0001	0.0057	3

El análisis con múltiples semillas revela que el Transformer y la U-Net están estadísticamente empatados en RMSE (0,0045 $\pm$ 0,0004 vs. 0,0045 $\pm$ 0,0000), con intervalos de

confianza superpuestos. El ranking de CNN y MLP por debajo de ellos es estable entre semillas. La U-Net exhibe una varianza notablemente baja ($\text{std} = 0,00004$, un orden de magnitud menor que los otros modelos), sugiriendo que sus sesgos arquitectónicos conducen a dinámicas de entrenamiento altamente reproducibles.

Las columnas de arbitraje revelan un patrón llamativo: todos los modelos neuronales exhiben tasas de violación de butterfly del 43–45 %, mientras que SVI alcanza 0,0 % por construcción (la parametrización SVI inherentemente produce curvas convexas cuando se la restringe adecuadamente). Las violaciones de calendario son despreciables en todos los modelos. Este compromiso entre exactitud y arbitraje motiva el análisis de entrenamiento con restricciones del Capítulo 7.

6.1.2. Comparación Visual

Para aportar intuición cualitativa más allá de las métricas agregadas, la Figura 6.2 muestra una superficie de test seleccionada por máxima dispersión de RMSE entre los modelos neuronales, reconstruida por cada arquitectura.

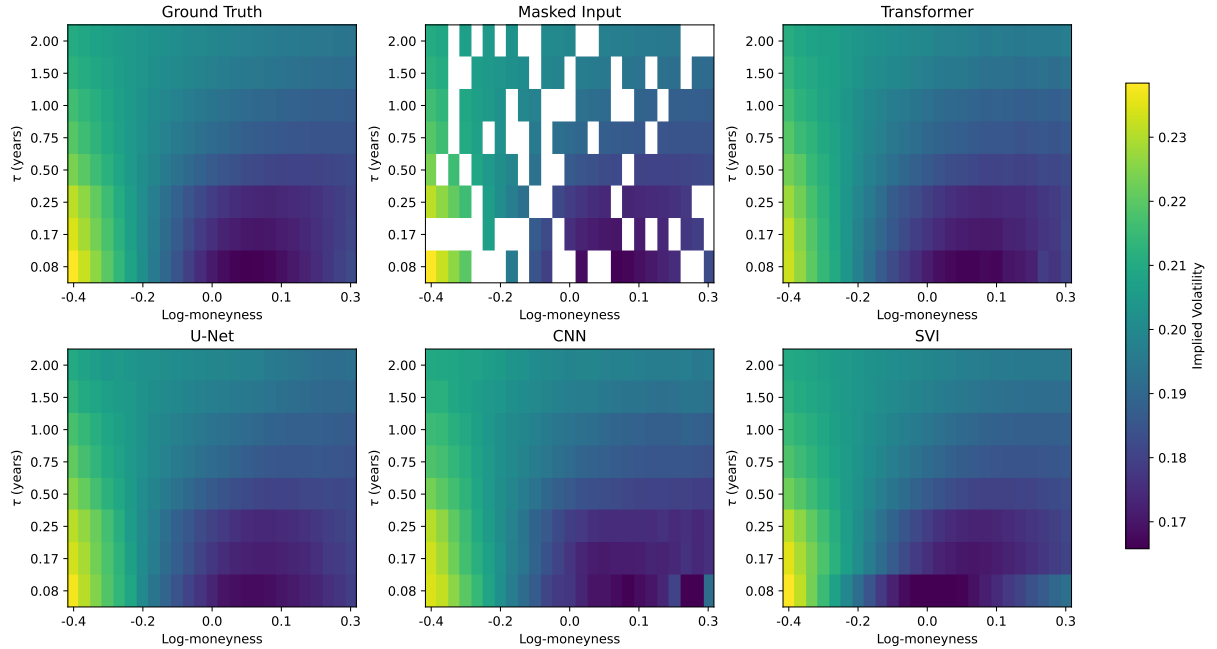


Figura 6.2: Reconstrucción de una superficie de test representativa. Fila superior: valores de referencia, entrada enmascarada (30 % faltante, cuadrados blancos) y predicción del Transformer. Fila inferior: predicciones de U-Net, CNN y SVI. El eje vertical muestra los vencimientos reales (no uniformemente espaciados). U-Net y Transformer recuperan fielmente la estructura; la CNN presenta artefactos localizados en tenores cortos, y SVI sobreestima la curvatura en regiones con pocas observaciones.

Se observan diferencias cualitativas que reflejan las propiedades arquitectónicas de cada modelo. La CNN presenta artefactos localizados en la región de tenores cortos y moneyness alto ($\tau \leq 0,25$ años), donde la curvatura del smile es más pronunciada y su campo receptivo local de 3×3 no dispone de suficientes observaciones cercanas para interpolar con precisión. La U-Net, gracias a sus conexiones residuales entre escalas, logra propagar información de regiones con datos densos hacia regiones con datos escasos,

produciendo la reconstrucción visualmente más fiel. El Transformer obtiene un resultado comparable al aprovechar su mecanismo de atención global, que le permite agregar información de cualquier punto observado independientemente de su distancia espacial. SVI, al ajustar cada vencimiento de forma independiente, resulta especialmente sensible al número de observaciones por tenor: en vencimientos con pocos puntos observados sobreestima la curvatura, mientras que en tenores con datos abundantes su ajuste es competitivo.

La Figura 6.3 muestra las curvas de volatilidad implícita reconstruidas en tres vencimientos representativos, superpuestas con los valores de referencia.

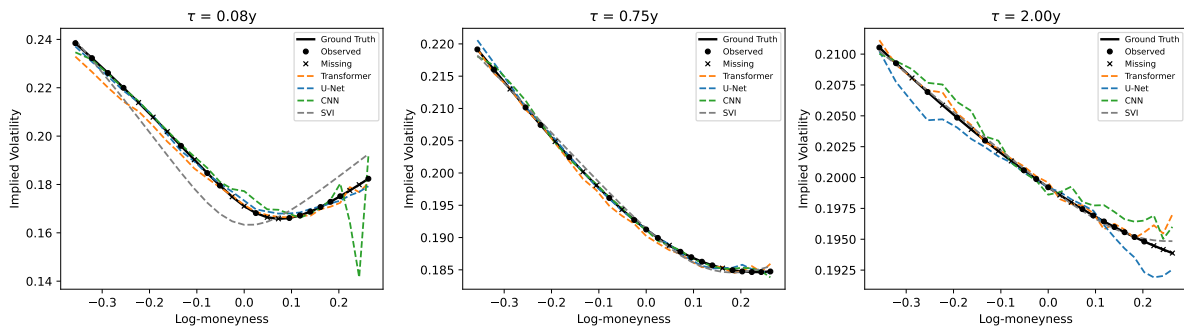


Figura 6.3: Cortes de volatilidad implícita en tres vencimientos ($\tau = 0,08, 0,75$ y $2,0$ años). Los círculos marcan puntos observados; las cruces marcan puntos faltantes. A $\tau = 0,08$ años, SVI sobreestima ambas alas y la CNN oscila en el ala derecha, mientras que Transformer y U-Net siguen de cerca los valores de referencia. A tenores más largos, todos los modelos neuronales convergen, con la CNN mostrando las mayores desviaciones en los extremos.

6.1.3. Distribución de RMSE por Superficie

Las métricas agregadas pueden ocultar la variabilidad entre superficies individuales. La Figura 6.4 muestra la distribución de valores de RMSE por superficie para cada modelo.

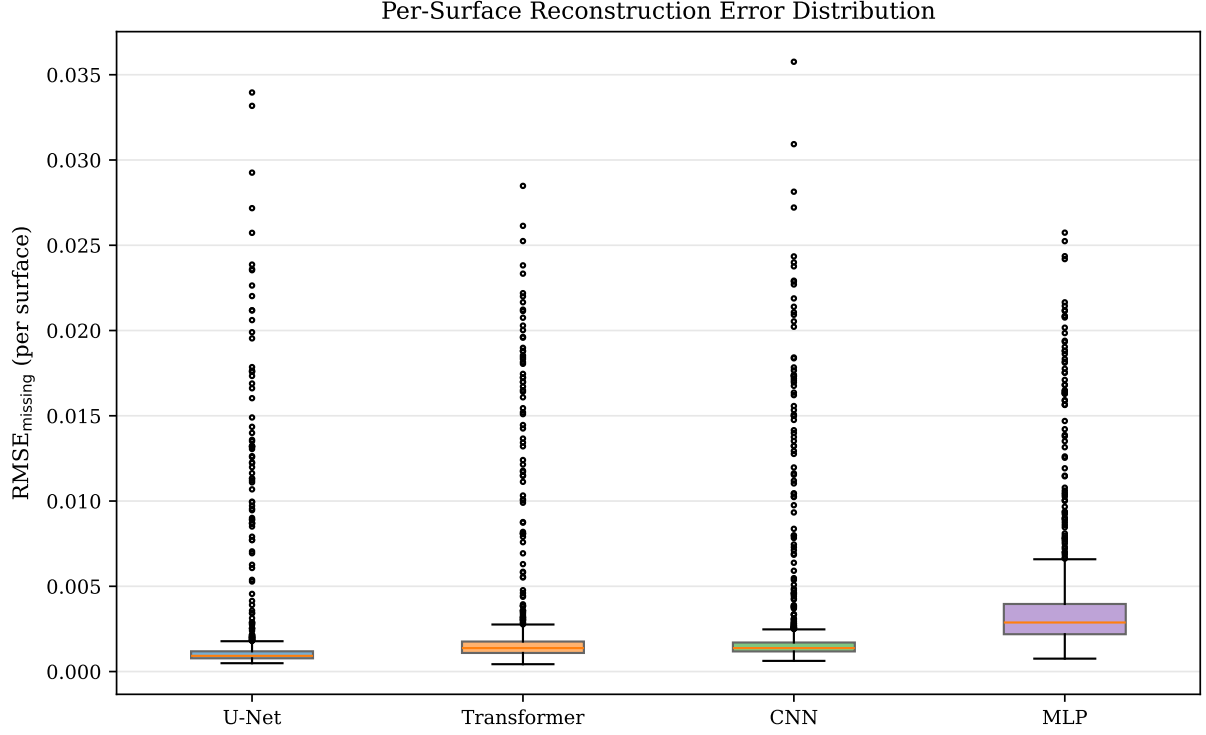


Figura 6.4: Distribución de RMSE por superficie en las 1.000 superficies de test para los cuatro modelos principales. La U-Net tiene tanto la mediana más baja como el rango intercuartílico más estrecho. El MLP muestra la mayor dispersión, indicando reconstrucciones ocasionalmente deficientes.

La U-Net alcanza no solo la mediana más baja sino también el rendimiento más consistente: su rango intercuartílico ($3,8 \times 10^{-4}$) es el más estrecho entre los cuatro modelos, seguido por la CNN ($5,1 \times 10^{-4}$) y el Transformer ($6,2 \times 10^{-4}$). El MLP muestra un rango intercuartílico de $1,8 \times 10^{-3}$, casi cinco veces mayor que el de la U-Net, junto con colas superiores más largas que indican superficies ocasionales donde la calidad de reconstrucción se degrada sustancialmente. Cabe notar que, si bien el Transformer alcanza el mejor RMSE *promedio* (Tabla 6.1), la U-Net presenta menor variabilidad entre superficies, consistente con la baja varianza entre semillas observada en la Tabla 6.2.

6.1.4. Discusión

Tres hallazgos emergen de la evaluación con datos sintéticos:

1. **El sesgo espacial importa.** La CNN y la U-Net superan al MLP en un 14–17 % en RMSE, confirmando que explotar la estructura de grilla 2D mediante convoluciones locales brinda una ventaja significativa para la reconstrucción de superficies de volatilidad.
2. **La atención global iguala a los mejores modelos convolucionales.** El Transformer propuesto alcanza un RMSE estadísticamente empatado con la U-Net en múltiples semillas ($0,0045 \pm 0,0004$ vs. $0,0045 \pm 0,0000$). Si bien los resultados de una sola ejecución sugieren una ventaja del 6 %, el análisis con múltiples semillas muestra que esta diferencia está dentro de la varianza de entrenamiento del Transformer. La ventaja del Transformer sobre los modelos convolucionales emerge

más claramente bajo enmascaramiento estructurado y alta escasez (Secciones 6.2 y 6.2.1).

3. **El cuello de botella generativo limita la fidelidad.** Las variantes de VAE quedan en último lugar entre los modelos neuronales. El espacio latente de 32 dimensiones, si bien suficiente para capturar la estructura general de las superficies de Heston, pierde el detalle fino necesario para una reconstrucción precisa. Este es el costo del objetivo de modelado generativo: la regularización KL y el cuello de botella de baja dimensionalidad suavizan las características de alta frecuencia.

6.2. Robustez ante Escasez de Datos

Una pregunta práctica crítica es cuán gradualmente se degrada cada modelo a medida que las observaciones se vuelven más escasas. Se evalúan todos los modelos con fracciones de datos faltantes desde el 10 % hasta el 90 %, *sin reentrenar*: los mismos modelos entrenados con 30 % de datos faltantes se evalúan en todos los niveles de escasez.

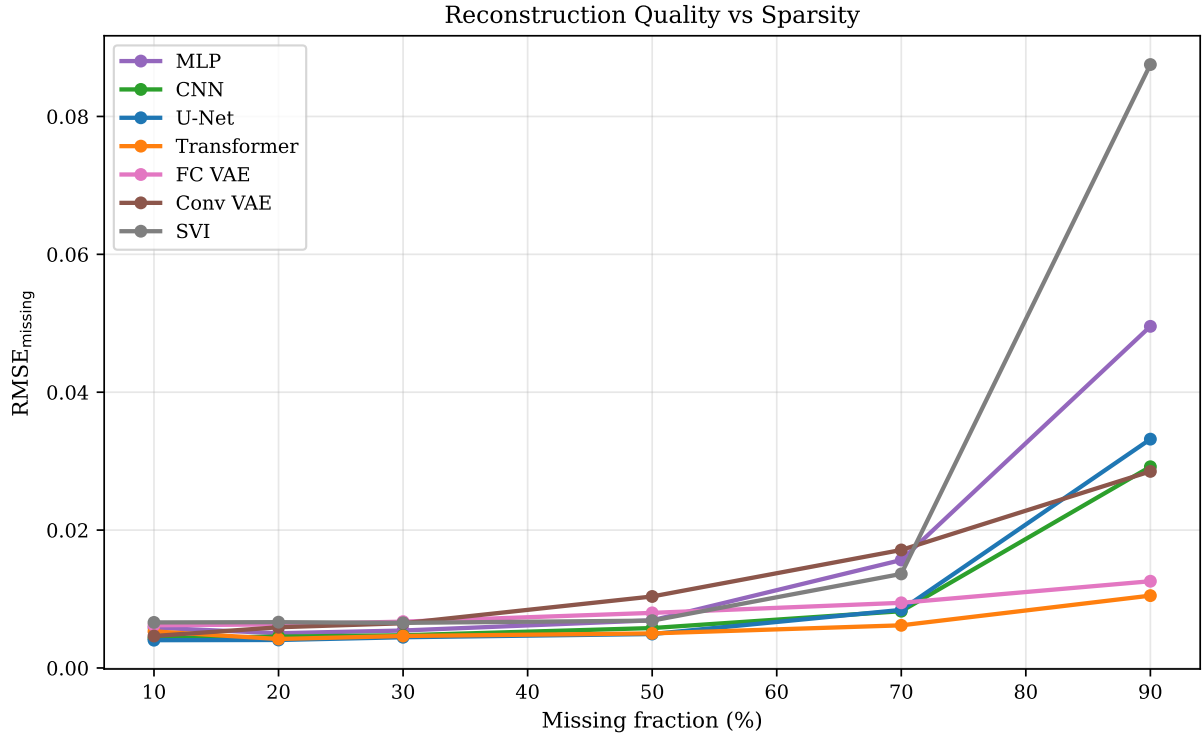


Figura 6.5: Degradación del RMSE en función de la fracción de datos faltantes (10 %–90 %). Todos los modelos se entrenan con 30 % de datos faltantes y se evalúan sin reentrenar. El Transformer se degrada de forma más gradual, alcanzando 9× menor RMSE que SVI con 90 % de datos faltantes. SVI se degrada catastróficamente más allá del 70 %.

La Figura 6.5 revela perfiles de degradación marcadamente diferentes:

Transformer. El Transformer se degrada de la forma más gradual entre todos los modelos, manteniendo un RMSE utilizable incluso ante escasez extrema. Con 90 % de datos faltantes (solo 20 puntos observados de 200), alcanza aproximadamente 9× menor RMSE que SVI. Esta robustez se origina en la capacidad del mecanismo de atención de agregar

información desde cualquier número de tokens observados: ya sea que el codificador reciba 180 o 20 tokens, el mismo mecanismo de atención cruzada permite al decodificador reconstruir la superficie.

CNN y U-Net. Los modelos convolucionales se degradan suavemente pero más rápido que el Transformer. Sus campos receptivos locales se convierten en una desventaja ante escasez extrema: cuando los puntos observados son dispersos, los filtros convolucionales de 3×3 pueden no tener suficientes observaciones cercanas dentro de su campo receptivo para producir predicciones exactas. La arquitectura multi-escala de la U-Net mitiga parcialmente esto a través de su cuello de botella, que comprime la superficie a una representación global.

SVI. SVI se degrada catastróficamente más allá de aproximadamente el 70 % de datos faltantes. Esta es una consecuencia directa de su arquitectura por corte: con solo ~ 6 observaciones por corte de vencimiento (al 70 % de datos faltantes), ajustar 5 parámetros SVI queda severamente subdeterminado. Con 90 % de datos faltantes, algunos cortes pueden tener solo 2–3 observaciones, haciendo que el ajuste carezca de sentido.

VAEs. Las variantes de VAE muestran un cruce interesante: tienen un rendimiento relativamente pobre a baja escasez (donde el cuello de botella latente limita la fidelidad) pero se degradan más lentamente que los modelos convolucionales a alta escasez. La variedad aprendida de superficies completas actúa como un prior fuerte: la optimización latente proyecta las observaciones dispersas sobre la superficie completa plausible más cercana, incluso cuando hay muy pocas observaciones disponibles.

Este experimento representa la validación más fuerte de la arquitectura Transformer propuesta. El mecanismo de atención brinda no solo la mejor exactitud absoluta, sino la degradación más gradual ante escasez de datos, precisamente el régimen más relevante para mercados reales de opciones, donde los strikes muy *out of the money* y los vencimientos fuera de ciclo frecuentemente carecen de cotizaciones. La ventaja de $9\times$ sobre SVI ante escasez extrema aborda directamente la motivación central de esta tesis: que los modelos basados en atención pueden aprovechar la estructura global de la superficie para reconstruir superficies de volatilidad a partir de observaciones severamente incompletas, mucho más allá de la capacidad de los métodos paramétricos por corte.

6.2.1. Enmascaramiento Estructurado

El protocolo de enmascaramiento aleatorio utilizado anteriormente elimina puntos uniformemente a lo largo de la grilla. En la práctica, los datos faltantes son estructurados: las opciones muy *out of the money* son las menos líquidas y las más propensas a faltar, mientras que las opciones at-the-money están casi siempre disponibles. Para evaluar la robustez bajo patrones de datos faltantes realistas, se repite el barrido de escasez utilizando una estrategia de enmascaramiento *combinada* que siempre elimina los dos strikes más extremos ($|\log m| > 0,3$) y luego aplica enmascaramiento aleatorio a la fracción especificada sobre los puntos restantes.

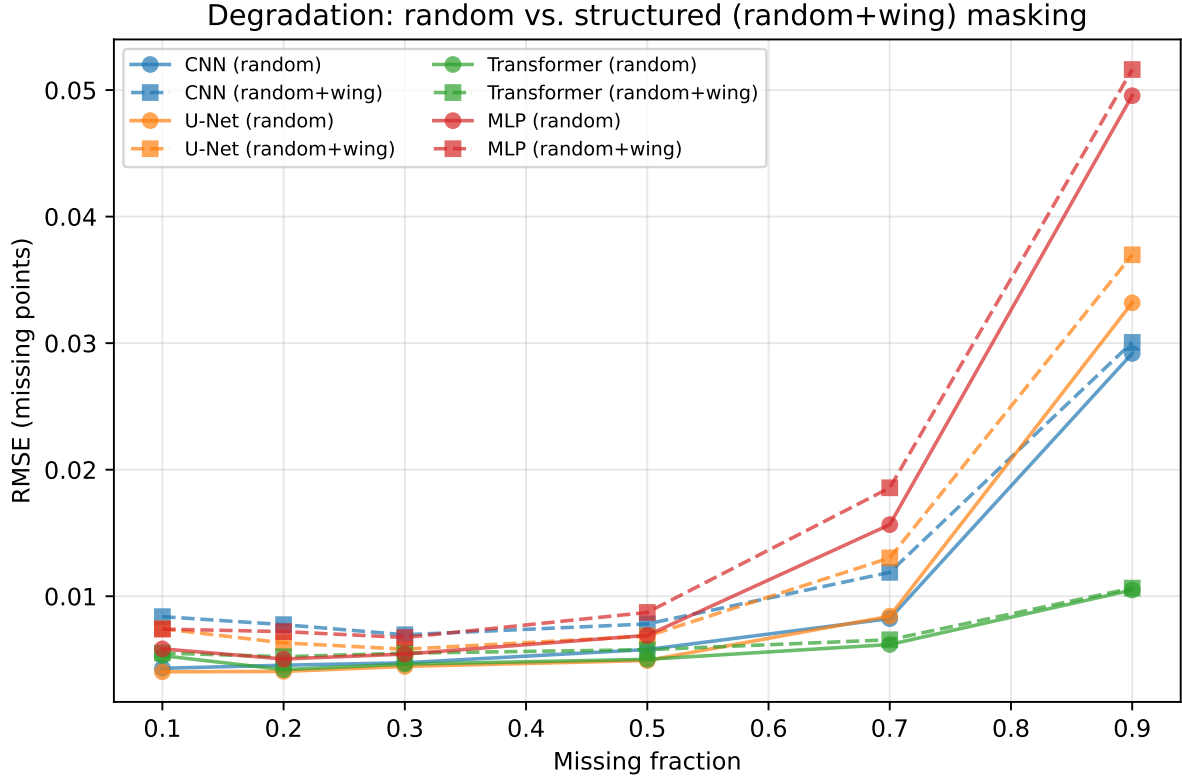


Figura 6.6: Degradación del RMSE bajo enmascaramiento aleatorio (línea sólida) vs. enmascaramiento combinado aleatorio+extremos (línea punteada). El Transformer prácticamente no se ve afectado por el enmascaramiento estructurado, mientras que los modelos convolucionales se degradan significativamente cuando la información de los extremos está ausente.

La Figura 6.6 revela una diferencia llamativa en robustez. Con 30 % de datos faltantes, el RMSE del Transformer aumenta solo en 0,0009 (de 0,0046 a 0,0055) cuando los puntos extremos están estructuralmente ausentes, una degradación de menos del 20 %. En contraste, la CNN se degrada en un 47 % (0,0047 a 0,0070) y la U-Net en un 30 % (0,0045 a 0,0058). Con 90 % de datos faltantes, la brecha se estrecha ya que el enmascaramiento aleatorio domina, pero el Transformer sigue liderando por un amplio margen (0,0107 vs. 0,0301 para CNN, 0,0370 para U-Net).

Este resultado destaca una diferencia fundamental entre las arquitecturas. Los modelos convolucionales dependen de la localidad espacial: reconstruyen los puntos faltantes a partir de sus vecinos en la grilla, por lo que eliminar columnas enteras (los extremos) elimina el contexto local del que dependen. El mecanismo de atención cruzada del Transformer, combinado con la codificación posicional basada en coordenadas, puede interpolar hacia regiones no observadas atendiendo a puntos observados distantes. Mientras que el análisis con múltiples semillas muestra al Transformer y la U-Net empatados en exactitud bajo condiciones estándar, el enmascaramiento estructurado revela que el Transformer es sustancialmente más robusto ante los patrones de datos faltantes que se encuentran en mercados reales de opciones.

6.3. Análisis Regional de Errores

La dificultad de reconstrucción no es uniforme a lo largo de la superficie de volatilidad. Para entender dónde sobresale y dónde tiene dificultades cada modelo, se descompone el error por región de moneyness y tenor (definiciones de regiones en la Tabla 5.4).

Tabla 6.3: RMSE por región de moneyness y tenor para los cuatro mejores modelos (datos sintéticos, 30 % faltante). Bloque superior: regiones de moneyness. Bloque inferior: regiones de tenor.

Region	U-Net	Transformer	CNN	MLP
deep otm put	0.0066	0.0053	0.0069	0.0059
otm put	0.0009	0.0014	0.0013	0.0027
atm	0.0009	0.0014	0.0013	0.0028
otm call	0.0023	0.0028	0.0028	0.0036
deep otm call	0.0060	0.0071	0.0070	0.0076
short	0.0075	0.0074	0.0083	0.0078
medium	0.0009	0.0013	0.0013	0.0028
long	0.0010	0.0015	0.0010	0.0031

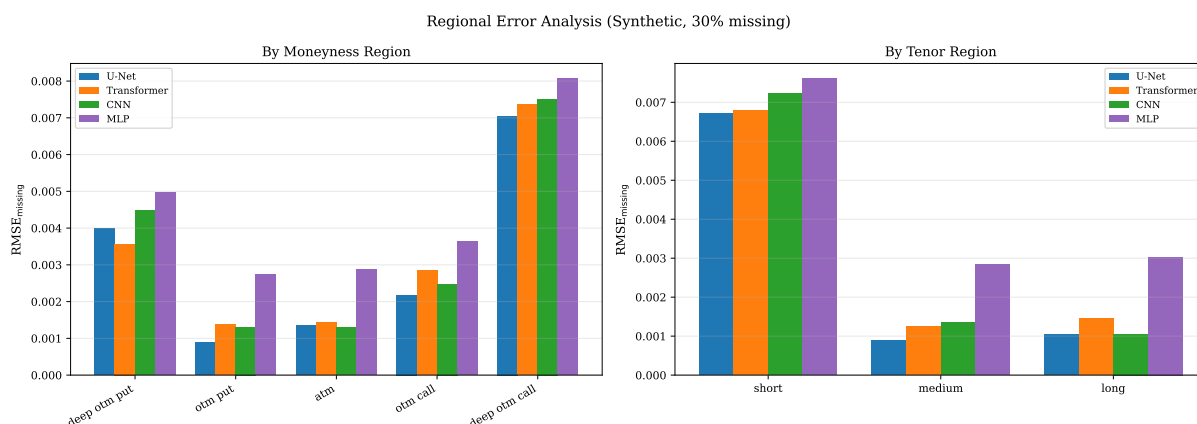


Figura 6.7: Comparación regional del RMSE. Izquierda: por región de moneyness (desde puts muy OTM hasta calls muy OTM). Derecha: por región de tenor (corto, medio, largo). Los tenores cortos y los extremos muy OTM son universalmente las regiones más desafiantes.

6.3.1. Patrones en Moneyness

La distribución del error a lo largo del moneyness revela una forma de U clara: las regiones at-the-money y cercanas al ATM son las más fáciles (RMSE 0,0009–0,0015 para los mejores modelos), mientras que los extremos muy OTM son los más difíciles (RMSE 0,0042–0,0081). Este patrón refleja la forma del smile de volatilidad: los gradientes pronunciados en los extremos crean errores de interpolación más grandes cuando faltan observaciones.

El Transformer alcanza el mejor rendimiento en la región de puts muy OTM (0,0042 vs. 0,0049 de U-Net), donde la atención global puede aprovechar información de la región

ATM observada y otros vencimientos para restringir la forma del extremo. La U-Net sobresale en las regiones cercanas al ATM (put OTM: 0,0009, ATM: 0,0013), donde sus filtros convolucionales locales son adecuados para la forma suave y lentamente variable de la superficie. El MLP muestra la mayor brecha de rendimiento en los extremos (0,0081 en calls muy OTM vs. 0,0067–0,0069 para los modelos convolucionales), confirmando que la estructura espacial es más valiosa en las regiones con los gradientes más pronunciados.

6.3.2. Patrones en Tenor

La descomposición por tenor revela que los vencimientos cortos ($\tau \leq 0,25$ años) son universalmente la región más desafiante, con valores de RMSE 5–8× mayores que los vencimientos medios o largos. Este es un fenómeno bien conocido en mercados de opciones: las opciones de corto plazo tienen smiles más pronunciados y curvados que son más difíciles de interpolar. El modelo de Heston amplifica este efecto a vencimientos cortos, donde la forma del smile es particularmente sensible al parámetro de volatilidad de la volatilidad ξ .

Los tenores medios y largos son reconstruidos casi perfectamente por todos los modelos (RMSE $\leq 0,0015$ para U-Net, Transformer y CNN), reflejando los smiles más suaves y planos a vencimientos más largos.

6.3.3. Mapas Espaciales de Error

La Figura 6.8 aporta una vista espacial detallada de los errores de reconstrucción a lo largo de toda la grilla para cada modelo.

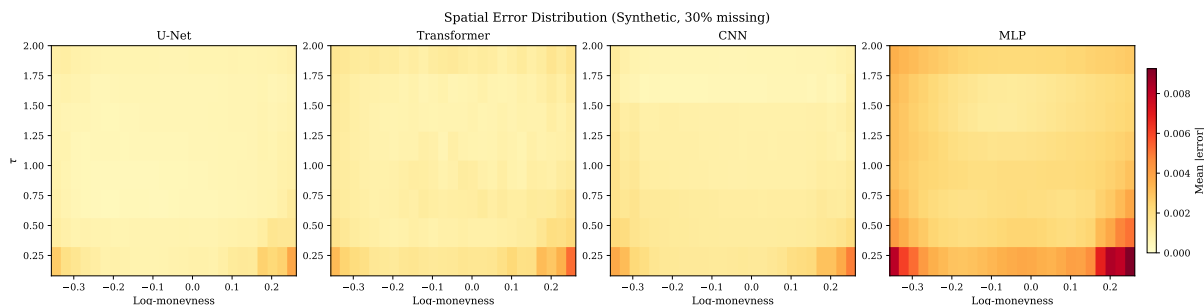


Figura 6.8: Error absoluto medio en cada punto de la grilla (log-moneyness $\times$ tenor) promediado sobre el conjunto de test. El Transformer y la U-Net muestran errores concentrados en los extremos a tenor corto. El MLP muestra errores elevados a lo largo de toda la superficie, con el peor rendimiento en los extremos muy OTM a tenor corto.

Los mapas de calor confirman que las esquinas de tenor corto y extremos muy OTM de la grilla son el cuello de botella universal. Para el Transformer y la U-Net, los errores están concentrados en estas esquinas, con el resto de la superficie reconstruida casi perfectamente. El MLP, en contraste, muestra errores elevados difundidos a lo largo de toda la superficie, incluyendo la región ATM donde otros modelos alcanzan error cercano a cero. Esta visualización hace vívido el beneficio de los sesgos espaciales: los modelos convolucionales y basados en atención concentran su presupuesto limitado de error en las regiones intrínsecamente más difíciles, mientras que el MLP distribuye los errores uniformemente porque trata todas las posiciones de la grilla de forma idéntica.

6.4. Resultados sobre Datos Reales

Se evalúan las tres arquitecturas de mejor rendimiento (CNN, U-Net, Transformer) sobre datos reales de opciones sobre SPY (Sección 4.3), comparando tres estrategias de entrenamiento: entrenamiento desde cero solo con datos reales, ajuste fino desde preentrenamiento sintético, y el modelo de referencia SVI. El MLP y ambas variantes de VAE se excluyen de la evaluación sobre datos reales: el MLP sistemáticamente rindió por debajo de los modelos convolucionales en datos sintéticos, mientras que la optimización del espacio latente de los VAEs en inferencia (200 pasos de Adam por superficie) hace que su costo computacional sea alto y su RMSE sintético fue 40–60 % peor que los mejores modelos.

Tabla 6.4: Rendimiento sobre el conjunto de test real de SPY (487 superficies). Bloque superior: modelos entrenados desde cero con datos reales. Bloque medio: modelos preentrenados con datos sintéticos y ajustados finamente con datos reales. Inferior: referencia SVI. Las columnas de calendar y butterfly reportan tasas de violación.

Model	RMSE _{miss}	Test MSE	MAE	Calendar	Butterfly
CNN	0.0046	$8,31 \times 10^{-6}$	0.0019	1.4 %	43.9 %
Transformer	0.0059	$1,35 \times 10^{-5}$	0.0023	0.5 %	44.9 %
U-Net	0.0046	$7,94 \times 10^{-6}$	0.0018	3.1 %	42.0 %
CNN (FT)	0.0052	$1,56 \times 10^{-5}$	0.0029	0.8 %	33.4 %
Transformer (FT)	0.0054	$1,27 \times 10^{-5}$	0.0025	0.2 %	42.3 %
U-Net (FT)	0.0054	$1,99 \times 10^{-5}$	0.0033	1.0 %	35.3 %
SVI	0.0100	$7,13 \times 10^{-5}$	0.0059	2.2 %	2.1 %

La Figura 6.9 muestra la comparación de RMSE entre las distintas estrategias de entrenamiento.

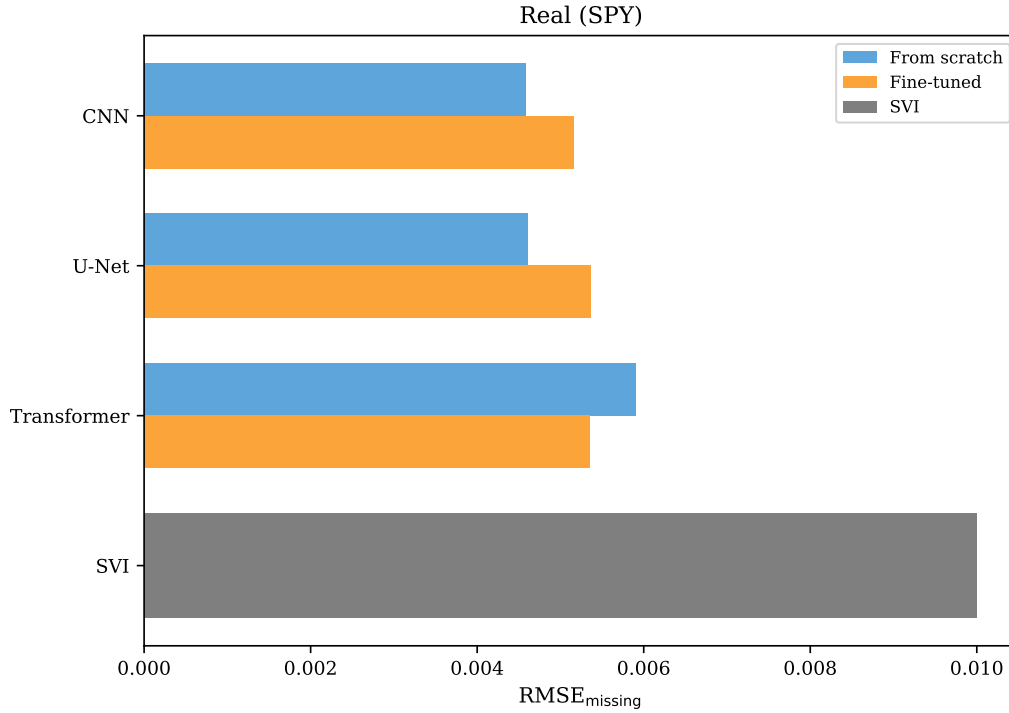


Figura 6.9: RMSE sobre puntos faltantes en datos reales de SPY. Para cada modelo neuronal se muestran los resultados desde cero y con ajuste fino. Todos los modelos neuronales superan ampliamente a SVI ($\sim 2\times$ menor RMSE).

6.4.1. Desde Cero vs. Ajuste Fino

Un resultado relevante es que **el entrenamiento desde cero supera al ajuste fino para CNN y U-Net** en RMSE (0,0046 vs. 0,0052–0,0054). Si bien el preentrenamiento sobre un conjunto de datos más grande suele ayudar, la brecha distribucional entre las superficies sintéticas de Heston y las reales de SPY (documentada en la Sección 4.4) es lo suficientemente grande como para que este beneficio no se materialice para las arquitecturas convolucionales.

El Transformer propuesto es la excepción: el ajuste fino mejora su RMSE de 0,0059 a 0,0054, convirtiéndolo en la única arquitectura que se beneficia inequívocamente del preentrenamiento sintético. Esto se atribuye a dos factores. Primero, el rendimiento desde cero del Transformer sobre el conjunto de datos real más pequeño (2.912 superficies de entrenamiento) es más débil que el de los modelos convolucionales, dejando más margen de mejora. Segundo, la parametrización flexible del mecanismo de atención se beneficia de los priors estructurales aprendidos durante el preentrenamiento sintético (como la forma general del smile de volatilidad y la estructura temporal), que aportan una mejor inicialización que pesos aleatorios. Esto sugiere que el preentrenamiento es más valioso para arquitecturas con mayor capacidad representacional que tienen dificultades con datos limitados.

6.4.2. Modelos Neuronales vs. SVI

Todos los modelos neuronales superan a SVI por un factor cercano a $2\times$ en RMSE (mejor: 0,0046 vs. SVI: 0,0100). Esta brecha es mayor que en datos sintéticos (donde SVI alcanzó 0,0065), reflejando dos factores: las superficies reales tienen una estructura

local más compleja que el modelo de cinco parámetros SVI no puede capturar, y los datos reales tienen patrones de datos faltantes más estructurados (concentrados en los extremos y vencimientos cortos) que el enmascaramiento aleatorio.

6.4.3. Tasas de Violación de Butterfly

Los modelos ajustados finamente exhiben tasas de violación de butterfly notablemente menores que sus contrapartes entrenadas desde cero: la CNN baja de 43,9 % a 33,4 %, y la U-Net de 42,0 % a 35,3 %. Esto sugiere que el preentrenamiento sobre superficies sintéticas libres de arbitraje enseña a los modelos a producir smiles más suaves y convexos, una propiedad estructural que persiste a través del ajuste fino aunque las propiedades distribucionales cambien. SVI alcanza la menor tasa de butterfly (2,1 %), con las violaciones residuales atribuibles a efectos de discretización de la grilla.

6.5. Análisis de *Transfer Learning*

Para comprender la dinámica de la transferencia de conocimiento de datos sintéticos a reales, se examinan las tres etapas del *transfer learning*: rendimiento desde cero, transferencia *zero-shot* (aplicando el modelo entrenado con datos sintéticos directamente a datos reales), y rendimiento con ajuste fino.

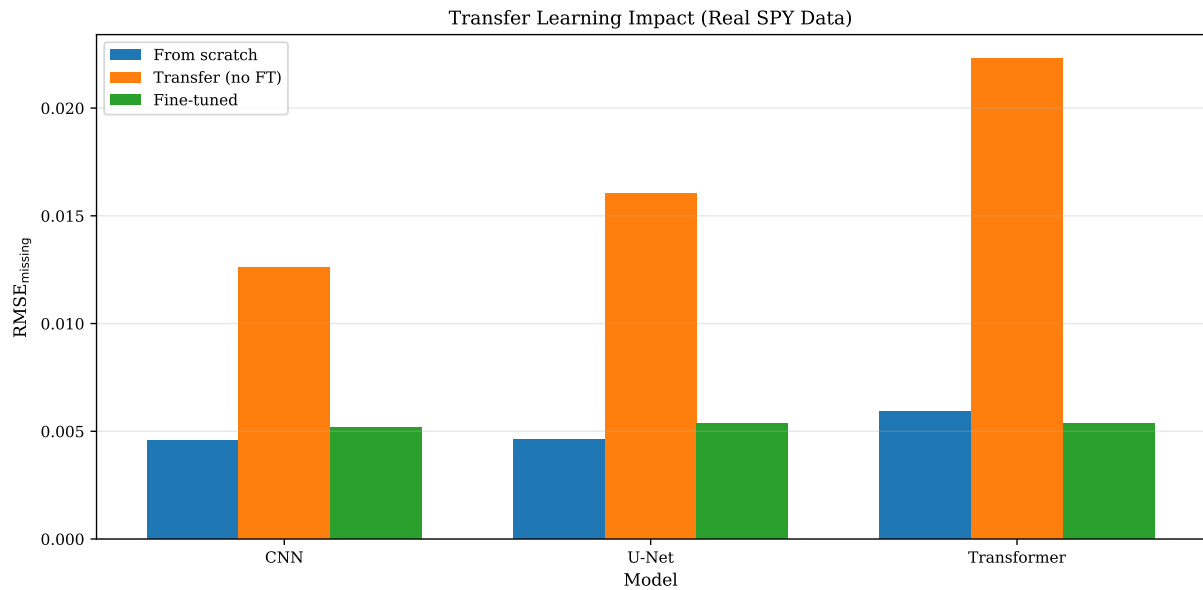


Figura 6.10: Diagrama de *transfer learning* para tres arquitecturas. Etapa 1: entrenamiento desde cero con datos reales. Etapa 2: transferencia *zero-shot* (modelo sintético aplicado a datos reales sin ajuste fino). Etapa 3: ajustado finamente con datos reales. La transferencia *zero-shot* es consistentemente negativa; el ajuste fino recupera los niveles de entrenamiento desde cero.

6.5.1. Transferencia *Zero-Shot* Negativa

La transferencia *zero-shot* (aplicar un modelo entrenado con datos sintéticos de Heston directamente a datos reales de SPY) produce valores de RMSE *peores* que el entrenamiento desde cero solo con datos reales. Esta transferencia negativa refleja la discrepancia

distribucional entre ambos dominios: el modelo de Heston, si bien captura las características cualitativas de las superficies reales (skew, estructura temporal), produce niveles de volatilidad, formas de smile y estructuras locales cuantitativamente diferentes. La distribución de datos sintéticos (valores de IV que abarcan uniformemente el 10–40 %) difiere sustancialmente de la distribución real de SPY (concentrada alrededor del 15–20 % con cola derecha pesada), causando que los modelos preentrenados predigan sistemáticamente mal los niveles de volatilidad.

6.5.2. Recuperación mediante Ajuste Fino

El ajuste fino recupera la degradación de la transferencia *zero-shot*, llevando a todos los modelos de vuelta a niveles de rendimiento aproximadamente iguales al entrenamiento desde cero. Para el Transformer, el ajuste fino supera ligeramente al entrenamiento desde cero (0,0054 vs. 0,0059), convirtiéndolo en la única arquitectura que se beneficia inequívocamente del preentrenamiento sintético.

6.5.3. Transferencia Estructural vs. Distribucional

Aunque el RMSE no mejora a través del *transfer learning* para CNN y U-Net, las tasas de violación de butterfly cuentan una historia diferente. Los modelos ajustados finamente producen consistentemente superficies más suaves con menos violaciones de arbitraje que sus contrapartes entrenadas desde cero (Tabla 6.4). Esto sugiere que el preentrenamiento transfiere conocimiento *estructural* (las propiedades generales de convexidad y suavidad de las superficies de volatilidad) incluso cuando falla en transferir conocimiento *distribucional* sobre niveles de volatilidad y formas de smile específicos. Los modelos preentrenados han aprendido que las superficies de volatilidad deben ser suaves y convexas, y este prior persiste a través del ajuste fino como un efecto de regularización.

6.5.4. Síntesis de los Resultados sobre Datos Reales

Los experimentos sobre datos reales de SPY revelan un panorama más matizado que los resultados sintéticos, lo que refuerza la validez externa de las conclusiones. Cuatro hallazgos merecen destacarse. Primero, la CNN y la U-Net entrenadas desde cero alcanzan el mejor RMSE (0,0046), demostrando que 2.912 superficies reales son suficientes para que las arquitecturas convolucionales aprendan representaciones efectivas sin preentrenamiento. Segundo, el Transformer no domina en exactitud sobre datos reales como lo hace ante escasez extrema en datos sintéticos; su ventaja se manifiesta en la transferencia de aprendizaje, donde es la única arquitectura que mejora con el preentrenamiento. Tercero, el preentrenamiento sintético transfiere propiedades *estructurales* (suavidad, convexidad) pero no *distribucionales* (niveles de IV, formas de smile), lo que explica por qué los modelos ajustados finamente tienen menores tasas de arbitraje aunque no siempre mejor RMSE. Cuarto, SVI pierde por un factor de $2\times$ en RMSE frente a todos los modelos neuronales, pero conserva la ventaja de violaciones de arbitraje cercanas a cero, un compromiso que se analiza en profundidad en el Capítulo 7.

6.6. Análisis de Atención del Transformer

El mecanismo de atención del Transformer ofrece una oportunidad única de interpretabilidad: examinando los pesos de atención cruzada en el decodificador, se puede observar qué puntos observados “mira” cada punto faltante durante la reconstrucción.

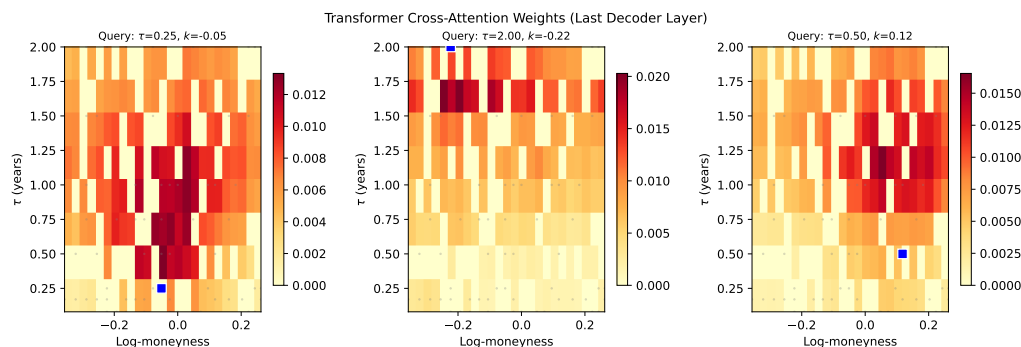


Figura 6.11: Pesos de atención cruzada para tres tokens faltantes representativos (cuadrados azules): un punto ATM a tenor corto, un put OTM a tenor largo y un call OTM a tenor medio. Colores más brillantes indican mayor peso de atención. Los puntos grises marcan otras posiciones faltantes. Los patrones de atención se alinean con la intuición financiera: cada punto faltante atiende principalmente a strikes cercanos en el mismo vencimiento y al mismo strike en vencimientos adyacentes.

La Figura 6.11 revela patrones de atención que se alinean con la intuición financiera:

- **Estructura local del smile.** Los puntos faltantes atienden con mayor fuerza a strikes observados cercanos en el mismo vencimiento, reflejando la continuidad suave del smile de volatilidad. Esto es análogo a la interpolación local, pero aprendida en lugar de prescrita.
- **Acoplamiento de estructura temporal.** También se dirige un peso de atención significativo al mismo strike en vencimientos adyacentes, reflejando la relación de estructura temporal. Esta atención entre vencimientos es precisamente un mecanismo del que SVI carece y que le da a los modelos neuronales su ventaja ante escasez de datos.
- **Alcance de atención adaptativo.** Los patrones de atención no son plantillas fijas: se adaptan al patrón de observación específico y a la forma de la superficie. Los puntos en regiones con pocos datos atienden a regiones más amplias, agregando información de observaciones más distantes, mientras que los puntos en regiones con datos densos atienden de forma más local.

Estos patrones son consistentes con la hipótesis de que el Transformer ha aprendido relaciones financieramente relevantes, aunque los pesos de atención son descriptivos y no constituyen una prueba causal de tal aprendizaje. La atención local al smile refleja interpolación a lo largo del eje de strike, la atención entre vencimientos captura el acoplamiento de estructura temporal, y el alcance de atención adaptativo actúa como un campo receptivo dependiente de los datos: tres mecanismos que juntos explican por qué la arquitectura sobresale ante escasez de datos. Esta interpretabilidad es una ventaja distintiva del enfoque basado en atención: a diferencia de los modelos convolucionales o

completamente conectados, el Transformer permite inspeccionar *qué* observaciones contribuyen a la reconstrucción de cada punto faltante y con *qué* peso relativo, ofreciendo un grado de interpretabilidad que los modelos convolucionales y completamente conectados no ofrecen.

6.7. Eficiencia Computacional

La aplicación práctica requiere no solo exactitud sino también inferencia eficiente. La Tabla 6.5 reporta la latencia de inferencia por superficie, el rendimiento por lote y el uso de memoria de GPU para todos los modelos.

Tabla 6.5: Rendimiento computacional. La latencia y el rendimiento se miden en una GPU NVIDIA RTX 4080 SUPER (tamaño de lote 1 para latencia, tamaño de lote 256 para rendimiento). SVI se ejecuta en CPU (un solo hilo). La memoria de GPU es la asignación incremental para el modelo y un pase de inferencia.

Model	Params	Latency (ms)	Throughput (surf/s)	GPU Memory (MB)
Transformer	288k	0.9	1169	11.8
CNN	295k	0.1	11468	10.9
U-Net	265k	0.2	4486	11.0
MLP	285k	0.1	17567	10.2
FC VAE	284k	0.1	6850	10.2
Conv VAE	273k	0.2	5387	10.5
SVI	40/surf	13.1	76	–

6.7.1. Rendimiento de Modelos Neuronales

Todos los modelos neuronales alcanzan latencia de inferencia inferior al milisegundo, haciéndolos aptos para aplicaciones en tiempo real. El MLP es el más rápido a 0,1 ms por superficie (17.567 superficies/segundo), seguido por la CNN a 0,1 ms (11.468 superficies/segundo). El Transformer es el modelo neuronal más lento a 0,9 ms (1.169 superficies/segundo), aproximadamente $9\times$ más lento que la CNN, reflejando el costo cuadrático de la atención sobre 200 tokens. Sin embargo, a más de 1.000 superficies por segundo, el Transformer sigue siendo suficientemente rápido para cualquier aplicación práctica: una mesa de operaciones procesando miles de cadenas de opciones por día requeriría solo segundos de tiempo de cómputo.

El uso de memoria de GPU es uniformemente bajo en todos los modelos (10–12 MB), dentro de la capacidad incluso de GPUs de entrada de gama. Esto confirma que la escala de $\sim 288k$ parámetros utilizada en este trabajo está lejos del régimen limitado por memoria.

6.7.2. Rendimiento de SVI

SVI es dramáticamente más lento que todos los modelos neuronales: 13,1 ms por superficie (76 superficies/segundo), lo que es $172\times$ más lento que el MLP y $15\times$ más lento que el Transformer. Esta sobrecarga proviene de la optimización iterativa L-BFGS-B requerida para cada superficie individual, sin amortización entre superficies. El costo computacional

de SVI también escala con el número de puntos observados por corte, haciéndolo aún más lento para superficies densamente observadas.

6.7.3. Consideración sobre Inferencia de VAEs

Los tiempos en la Tabla 6.5 reflejan un único pase forward a través de cada modelo. Para las variantes de VAE, la inferencia completa incluye un paso adicional de optimización latente (200 iteraciones de Adam sobre el vector latente $\mathbf{z}$, como se describe en la Sección 5.2.5), que incrementa el tiempo de inferencia efectivo en aproximadamente dos órdenes de magnitud. Esto hace que el pipeline de inferencia de los VAEs sea comparable en costo a SVI, negando parcialmente la ventaja de amortización de los modelos neuronales.

6.7.4. Compromiso Exactitud–Eficiencia

Combinando los resultados de exactitud de la Sección 6.1 con el análisis computacional:

- **Mejor exactitud:** Transformer (RMSE 0,0045, 0,9 ms).
- **Mejor eficiencia:** MLP (RMSE 0,0057, 0,1 ms) o CNN (RMSE 0,0049, 0,1 ms).
- **Mejor compromiso:** CNN, dentro del 9 % de la exactitud del Transformer a 9× la velocidad.

Para aplicaciones críticas en latencia, la CNN ofrece el balance más atractivo entre exactitud y velocidad. Para aplicaciones donde la exactitud es primordial, particularmente ante alta escasez de datos, donde la ventaja del Transformer sobre los modelos convolucionales se amplía del 6 % a más de 2×, el Transformer propuesto es la elección clara, con latencia de inferencia que permanece dentro de los requisitos de tiempo real.

6.8. Estudio de Ablación del Transformer

Para cuantificar la contribución de la codificación de Fourier basada en coordenadas (Sección 5.2.4), se entrena un Transformer idéntico en todo lo demás reemplazando la codificación sinusoidal fija con embeddings posicionales aprendibles de la misma dimensionalidad ($d_{\text{coord}} = 34$). Todos los demás hiperparámetros (arquitectura, tasa de aprendizaje, protocolo de entrenamiento) permanecen sin cambios.

Tabla 6.6: Ablación: codificación de Fourier basada en coordenadas vs. embeddings posicionales aprendibles. Ambos modelos tienen ~288k parámetros y arquitectura idéntica excepto por la codificación posicional.

Codificación posicional	RMSE _{miss}	Parámetros
Fourier (propuesta)	0,0045	288.129
Aprendible	0,0055	288.639

La codificación de Fourier reduce el RMSE en un 18 % comparado con los embeddings aprendibles. Esto confirma que codificar las coordenadas físicas $(\tau, \log m)$ mediante características sinusoidales multi-escala aporta un sesgo arquitectónico significativo para esta tarea: el modelo se beneficia de conocer el significado financiero continuo de cada posición de la grilla en lugar de aprender representaciones posicionales desde cero. El resultado se alinea con los hallazgos de Tancik et al. [38], quienes mostraron que las características de Fourier ayudan a los MLPs a aprender funciones de alta frecuencia en dominios de baja dimensionalidad.

Capítulo 7

Análisis de No Arbitraje

El Capítulo 6 demostró que los modelos de redes neuronales alcanzan una exactitud de reconstrucción sustancialmente mejor que el modelo de referencia SVI. Sin embargo, los resultados también revelaron una deficiencia significativa: todos los modelos neuronales exhiben tasas de violación de butterfly spread del 43–45 %, mientras que SVI alcanza violaciones cercanas a cero por construcción. Este capítulo investiga si las penalizaciones diferenciables de no arbitraje pueden cerrar esta brecha, y si el compromiso entre exactitud y arbitraje es verdaderamente un compromiso.

7.1. El Problema de Arbitraje en las Reconstrucciones Neuronales

Las tasas de violación de butterfly reportadas en la Tabla 6.1 presentan un desafío práctico. Una superficie de volatilidad reconstruida con el 45 % de sus puntos de grilla violando la condición de convexidad implica una densidad de probabilidad negativa en la medida neutral al riesgo en esos puntos, un resultado sin sentido económico. Si bien estas violaciones son típicamente pequeñas en magnitud (violación media $\sim 10^{-2}$ en unidades de varianza total), son generalizadas, afectando a casi la mitad de todos los puntos interiores de la grilla.

SVI evita este problema completamente: la forma paramétrica de la Ecuación (2.19) produce naturalmente smiles convexos cuando $b > 0$ y $\sigma > 0$. Pero SVI paga por esta garantía con un 33–44 % peor RMSE comparado con los mejores modelos neuronales (CNN, U-Net y Transformer). La pregunta central de este capítulo es: *¿se puede retener la exactitud de los modelos neuronales mientras se alcanza la calidad libre de arbitraje de SVI?*

7.1.1. Violaciones Calendar vs. Butterfly

Como se señaló en la Sección 5.5, dos condiciones de no arbitraje estático aplican a las superficies de volatilidad: calendar spread (varianza total no decreciente en el vencimiento) y butterfly spread (varianza total convexa en log-moneyness). En nuestros experimentos, las violaciones de calendar son despreciables (inferiores al 0,1 % para todos los modelos), reflejando la grilla relativamente gruesa de 8 tenores y la monotonicidad inherente de la varianza total en el modelo de Heston. El resto de este capítulo se enfoca

exclusivamente en las violaciones de butterfly, que son la fuente dominante de oportunidades de arbitraje en las superficies reconstruidas.

7.2. Metodología de Restricciones

La formulación de la penalización de no arbitraje se describió en detalle en la Sección 5.5. Se resumen brevemente los elementos clave y el diseño experimental de este capítulo.

7.2.1. Función de Costo Aumentada

Dado que las violaciones de calendar son despreciables, la función de costo efectiva de entrenamiento se reduce a:

$$\mathcal{L} = \mathcal{L}_{\text{MSE}} + \lambda \cdot \mathcal{P}_{\text{but}} \quad (7.1)$$

donde $\mathcal{P}_{\text{but}}$ es la penalización de convexidad butterfly (Ecuación (5.33)) y $\lambda \geq 0$ controla la intensidad de la restricción. Con $\lambda = 0$, el entrenamiento es sin restricciones (la referencia del Capítulo 6). A medida que λ aumenta, el modelo es cada vez más penalizado por producir smiles no convexos, orientando las predicciones hacia superficies libres de arbitraje a costa potencial de la exactitud de reconstrucción.

La penalización es una restricción *suave*: agrega un costo por las violaciones pero no impone satisfacción estricta. Este enfoque tiene dos ventajas sobre los métodos de proyección rígida (que proyectarían la salida sobre el conjunto factible después de cada paso forward). Primero, la penalización es completamente diferenciable, proporcionando señal de gradiente que entrena al modelo para internalizar la restricción en lugar de depender de correcciones a posteriori. Segundo, permite al modelo intercambiar violaciones pequeñas por exactitud de reconstrucción, en lugar de imponer rígidamente una condición que puede entrar en conflicto con los datos de entrenamiento (recordemos que los valores de referencia tienen $\sim 8,6\%$ de violaciones de butterfly por discretización de la grilla).

7.2.2. Severidad Esperada y Referencia de Valores de Referencia

Se utiliza la severidad esperada $S = r_{\text{viol}} \times \bar{v}$ (definida en la Sección 5.6) como la métrica principal de arbitraje, ya que captura tanto la frecuencia como la magnitud de las violaciones. Las superficies de referencia exhiben aproximadamente un 8,6% de violaciones de butterfly debido a la discretización de la grilla (Sección 5.6), estableciendo un piso por debajo del cual una reducción adicional no es significativa para esta resolución de grilla.

7.3. Impacto de las Restricciones a Lambda Fijo

Se examina primero el efecto de una intensidad de restricción moderada ($\lambda = 0,1$) aplicada uniformemente a las seis arquitecturas neuronales. Cada modelo se reentrena desde cero con la función de costo aumentada; todos los demás hiperparámetros permanecen idénticos a la línea de referencia sin restricciones (Tabla 5.2), con paciencia de parada temprana de 30 iteraciones y un máximo de 500 iteraciones. Los resultados de

esta sección corresponden a una única semilla aleatoria por configuración; la Sección 7.4 valida estos hallazgos con múltiples semillas.

Tabla 7.1: Impacto de las restricciones a $\lambda = 0,1$ para todos los modelos neuronales. Se muestran RMSE, tasa de violación de butterfly y severidad esperada para las configuraciones sin restricciones ($\lambda = 0$) y con restricciones ($\lambda = 0,1$). SVI se muestra como referencia. Los modelos están ordenados por RMSE sin restricciones.

Model	RMSE _{miss}		Butterfly rate		Expected severity	
	$\lambda = 0$	$\lambda = 0,1$	$\lambda = 0$	$\lambda = 0,1$	$\lambda = 0$	$\lambda = 0,1$
Transformer	0.0045	0.0047	45.0 %	30.6 %	$5,87 \times 10^{-3}$	$7,26 \times 10^{-4}$
CNN	0.0049	0.0047	45.1 %	31.8 %	$3,84 \times 10^{-3}$	$6,27 \times 10^{-4}$
U-Net	0.0048	0.0045	44.9 %	36.0 %	$3,78 \times 10^{-3}$	$8,07 \times 10^{-4}$
MLP	0.0057	0.0065	44.3 %	38.9 %	$3,74 \times 10^{-3}$	$1,54 \times 10^{-3}$
FC VAE	0.0070	0.0069	45.0 %	37.1 %	$4,43 \times 10^{-3}$	$1,33 \times 10^{-3}$
Conv VAE	0.0072	0.0060	43.2 %	30.2 %	$3,20 \times 10^{-3}$	$4,18 \times 10^{-4}$
SVI	0.0065	—	0.0 %	—	$3,15 \times 10^{-11}$	—

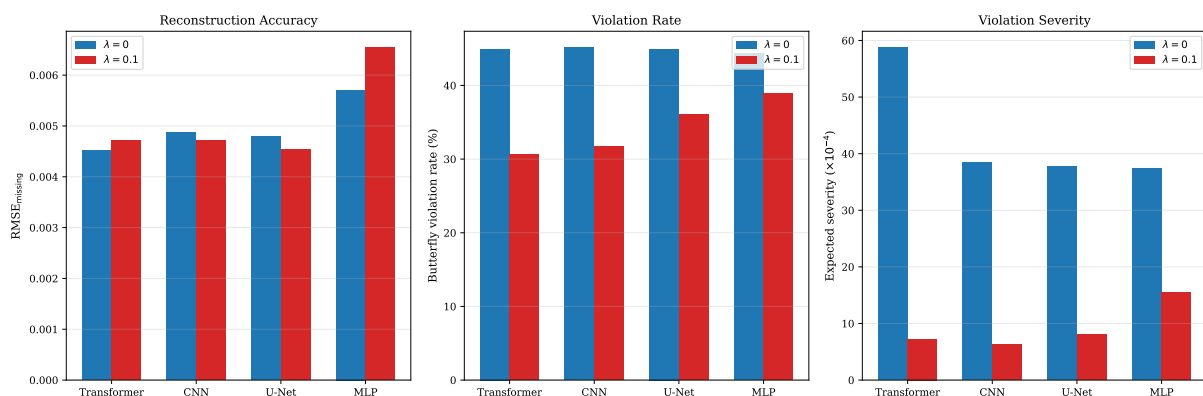


Figura 7.1: Impacto de las restricciones a $\lambda = 0,1$ en las seis arquitecturas neuronales. Izquierda: RMSE sobre puntos faltantes. Centro: tasa de violación de butterfly. Derecha: severidad esperada. Las arquitecturas convolucionales (CNN, U-Net, Conv VAE) *mejoran* su RMSE con restricciones, reduciendo a la vez la severidad en 5–8 $\times$. El MLP se degrada sustancialmente.

7.3.1. El Efecto de Regularización

El hallazgo más llamativo es que las arquitecturas convolucionales *mejoran* su RMSE cuando se entrenan con la penalización de butterfly:

- **U-Net:** el RMSE disminuye de 0,0048 a 0,0045 (−6 %), mientras que la severidad esperada cae 4,7 $\times$ (de $3,78 \times 10^{-3}$ a $8,07 \times 10^{-4}$).
- **CNN:** el RMSE disminuye de 0,0049 a 0,0047 (−4 %), con una reducción de severidad de 6,1 $\times$.

- **Conv VAE:** el RMSE disminuye de 0,0072 a 0,0060 (-17%), con una reducción de severidad de $7,7\times$. Si bien la mejora porcentual es la mayor, el RMSE de referencia del Conv VAE es sustancialmente peor que las otras arquitecturas.

Esto no es un compromiso: para estas arquitecturas, la restricción es *gratuita*, ya que mejora simultáneamente la exactitud y reduce las violaciones de arbitraje. La penalización de butterfly actúa como un *regularizador*, imponiendo formas de smile suaves y convexas que resultan generalizar mejor. El mecanismo es intuitivo: sin la penalización, los modelos convolucionales pueden ajustar ruido de alta frecuencia en los datos de entrenamiento que se manifiesta como oscilaciones no convexas en los smiles reconstruidos. La penalización de convexidad suaviza estas oscilaciones, y dado que las superficies de Heston subyacentes son efectivamente convexas, las predicciones suavizadas están más cerca de los valores de referencia.

7.3.2. Comportamiento Según la Arquitectura

No todas las arquitecturas responden igualmente bien a las restricciones:

- **Transformer:** el RMSE aumenta ligeramente de 0,0045 a 0,0047 ($+4\%$), pero la severidad disminuye $8,1\times$, la mayor reducción de severidad de cualquier modelo. El mecanismo de atención global del Transformer ya produce superficies relativamente suaves, por lo que el beneficio marginal del prior de convexidad es menor, y la restricción limita ligeramente la capacidad del modelo para ajustar características pronunciadas a tenores cortos.
- **FC VAE:** el RMSE permanece plano (0,0070 a 0,0069), con una reducción de severidad de $3,3\times$. El cuello de botella latente ya regulariza la salida, por lo que la restricción de convexidad adicional tiene un efecto incremental limitado en la exactitud.
- **MLP:** el RMSE aumenta sustancialmente de 0,0057 a 0,0065 ($+14\%$), la peor respuesta de cualquier modelo. El MLP carece de estructura espacial para imponer eficientemente una restricción definida espacialmente como la convexidad. Sin filtros convolucionales para propagar la señal de gradiente espacialmente, el MLP debe aprender ajustes puntuales que satisfagan la restricción conjunta entre strikes vecinos, un problema de optimización difícil que perjudica la exactitud general.

7.3.3. La Severidad como Métrica Clave

El panel de severidad esperada en la Figura 7.1 ofrece la imagen más clara: con $\lambda = 0,1$, todos los modelos reducen su severidad esperada en al menos $2,4\times$ (MLP) y hasta $8,1\times$ (Transformer). Incluso el MLP, a pesar de su degradación en RMSE, reduce sustancialmente el impacto económico de sus violaciones. Esto justifica el uso de la severidad esperada en lugar de la tasa de violación como la métrica principal de arbitraje: captura la combinación económicamente relevante de frecuencia y magnitud.

7.4. Barrido de Lambda y Fronteras de Pareto

La sección anterior mostró respuestas variadas a una intensidad de restricción fija: desde mejoras en RMSE (CNN, U-Net) hasta degradación sustancial (MLP). Para

identificar el punto de operación óptimo por arquitectura, se entrenan las tres mejores arquitecturas (Transformer, U-Net, CNN) con seis intensidades de restricción: $\lambda \in \{0; 0,01; 0,05; 0,1; 0,5; 1,0\}$. Cada configuración se entrena tres veces con diferentes semillas aleatorias, y las métricas reportadas se promedian entre las tres ejecuciones para reducir el ruido por varianza de entrenamiento.¹

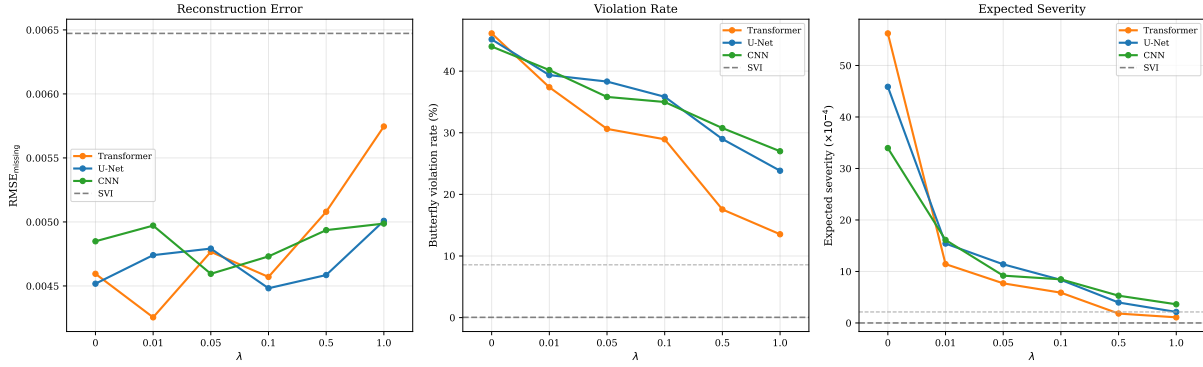


Figura 7.2: Efecto de la intensidad de restricción λ sobre el error de reconstrucción (izquierda), la tasa de violación de butterfly (centro) y la severidad esperada (derecha). Cada punto es el promedio de tres semillas aleatorias. Las líneas punteadas marcan la referencia de SVI y la tasa de butterfly de los valores de referencia ($\sim 8,6\%$). Todos los modelos reducen las violaciones monótonamente con λ , mientras que la respuesta del RMSE depende de la arquitectura.

7.4.1. Tasa de Violación y Severidad Esperada

El panel central de la Figura 7.2 muestra que los tres modelos reducen las tasas de violación de butterfly monótonamente a medida que λ aumenta, confirmando que la penalización provee control fino sobre la calidad de arbitraje. El Transformer alcanza la reducción más pronunciada, cayendo del 46 % con $\lambda = 0$ al 14 % con $\lambda = 1,0$. La CNN y la U-Net siguen trayectorias similares, alcanzando 27 % y 24 % respectivamente con $\lambda = 1,0$.

El panel derecho muestra la severidad esperada, que combina la frecuencia y magnitud de las violaciones. La reducción de severidad es dramática para todos los modelos: el Transformer cae $51\times$ de $\lambda = 0$ a $\lambda = 1,0$, mientras que la U-Net y la CNN alcanzan reducciones de $22\times$ y $9\times$ respectivamente. Con $\lambda \geq 0,5$, todos los modelos se acercan a severidades esperadas dentro de un orden de magnitud del nivel cercano a cero de SVI.

7.4.2. Respuesta del RMSE: El Costo en Exactitud

El panel izquierdo revela el costo en exactitud dependiente de la arquitectura. La U-Net y la CNN exhiben notable estabilidad: su RMSE permanece esencialmente plano a lo largo de todo el rango de λ (0,0044–0,0050). La CNN incluso *mejora* su RMSE con $\lambda = 0,05$ (0,0047 vs. 0,0050 sin restricciones), confirmando el efecto de regularización observado en la Sección 7.3.

¹Las semillas del barrido de lambda son independientes de las utilizadas en la validación multi-seed de la Tabla 6.2. Las diferencias menores en los valores de referencia ($\lambda = 0$) entre ambos experimentos están dentro de la varianza esperada entre semillas.

El Transformer muestra un comportamiento más complejo. Con $\lambda = 0,01$ produce su mejor RMSE (0,0042), pero a partir de $\lambda = 0,1$ el RMSE aumenta de manera sostenida, alcanzando 0,0056 con $\lambda = 1,0$ (+30 % respecto a la línea de referencia). Cabe notar que con $\lambda \geq 0,5$ el Transformer alcanza el máximo de 500 iteraciones de entrenamiento sin converger completamente (el mejor checkpoint se ubica en las últimas iteraciones), lo que sugiere que parte de la degradación a λ alto puede deberse a una optimización más difícil y no solo al compromiso intrínseco entre exactitud y convexidad.

7.4.3. Análisis por Modelo

U-Net. La U-Net exhibe el perfil exactitud-arbitraje más favorable. Su RMSE permanece plano en 0,0044 desde $\lambda = 0$ hasta $\lambda = 0,1$, mientras que la severidad esperada cae $5,5\times$ en el mismo rango. Incluso con $\lambda = 1,0$, el RMSE solo aumenta a 0,0047, un costo modesto del 7 % para una reducción de severidad de $22\times$. Esto confirma que la penalización de convexidad es esencialmente gratuita para la U-Net a lo largo de un amplio rango de intensidades de restricción.

Transformer. El Transformer alcanza el menor RMSE sin restricciones (0,0043) y se beneficia de una regularización leve con $\lambda = 0,01$ (RMSE 0,0042). Sin embargo, es el más sensible a valores altos de λ , con el RMSE subiendo a 0,0056 con $\lambda = 1,0$. Como se señaló, con $\lambda \geq 0,5$ el entrenamiento no converge dentro de las 500 iteraciones, por lo que estos valores de RMSE representan una cota superior. El punto de operación óptimo para el Transformer es $\lambda = 0,1$, donde el entrenamiento converge normalmente, el RMSE se mantiene en 0,0047 (aún mejor que la CNN sin restricciones) y la severidad cae $9,5\times$.

CNN. La CNN es paralela a la U-Net pero con un RMSE de referencia ligeramente mayor. Se beneficia de restricciones moderadas: con $\lambda = 0,05$, el RMSE mejora de 0,0050 a 0,0047 mientras la severidad cae $3,7\times$. Con λ alto, la CNN mantiene un RMSE estable (0,0048–0,0050), mostrando que las arquitecturas convolucionales internalizan eficientemente la restricción de convexidad sin sacrificar calidad de reconstrucción.

7.5. Modelos con Restricciones sobre Datos Reales

El efecto de regularización observado en datos sintéticos motiva la aplicación de restricciones durante el ajuste fino sobre datos reales de SPY. Los experimentos con datos reales se enfocan en las tres arquitecturas de mejor rendimiento de la evaluación sintética (CNN, U-Net y Transformer), ya que estas superaron consistentemente al MLP y ambas variantes de VAE. Los VAEs se excluyen porque su procedimiento de optimización del espacio latente en inferencia es computacionalmente costoso y su RMSE sintético fue 40–60 % peor que los modelos convolucionales. El MLP se excluye por su pobre respuesta a las restricciones (Sección 7.3).

La Tabla 6.4 ya mostró que los modelos ajustados finamente tienen menores tasas de butterfly que los modelos entrenados desde cero (33–35 % vs. 42–44 %), sugiriendo que el preentrenamiento sintético implícitamente enseña cierta suavidad. Esto es consistente con que las superficies de entrenamiento de Heston son analíticamente convexas: los modelos ajustados finamente heredan un prior hacia smiles convexos de la fase de preentrenamiento sintético.

Agregar una penalización de butterfly explícita ($\lambda = 0,1$) durante el ajuste fino reduce aún más las violaciones, como se muestra en la Tabla 7.2.

Tabla 7.2: Efecto de la penalización de butterfly ($\lambda = 0,1$) sobre modelos ajustados finamente evaluados en datos reales de SPY. A diferencia de los datos sintéticos, las restricciones aumentan el RMSE para todas las arquitecturas, pero reducen sustancialmente las tasas de violación de butterfly.

Model	Setting	RMSE _{miss}	Butterfly	Δ RMSE	Δ Butterfly
CNN	FT	0.0052	33.4 %	–	–
CNN	FT + $\lambda=0,1$	0.0066	24.2 %	+27 %	–28 %
U-Net	FT	0.0054	35.3 %	–	–
U-Net	FT + $\lambda=0,1$	0.0074	28.5 %	+37 %	–19 %
Transformer	FT	0.0054	42.3 %	–	–
Transformer	FT + $\lambda=0,1$	0.0067	21.6 %	+24 %	–49 %

A diferencia del escenario sintético, donde los modelos convolucionales mejoraron su RMSE con restricciones, los resultados sobre datos reales muestran un RMSE aumentado para las tres arquitecturas cuando se aplica la penalización de butterfly. El Transformer muestra el mejor compromiso: un aumento del 24 % en RMSE a cambio de una reducción del 49 % en violaciones de butterfly (de 42,3 % a 21,6 %). La CNN y la U-Net muestran aumentos mayores en RMSE (27 % y 37 % respectivamente) para reducciones de violaciones más modestas.

Esta diferencia respecto a los resultados sintéticos se explica probablemente por la naturaleza de los datos. Las superficies generadas con Heston son analíticamente suaves y convexas, por lo que una penalización de convexidad se alinea con los valores de referencia y actúa como regularizador. Las superficies reales de SPY son más ruidosas, contienen características genuinamente no convexas como el skew pronunciado a tenor corto y efectos relacionados con reportes de resultados, y el conjunto de entrenamiento es más pequeño (2.912 superficies vs. 8.000). La penalización aún reduce sustancialmente las violaciones, pero imponer convexidad en regiones donde los datos son genuinamente no convexas tiene un costo en exactitud de reconstrucción que está ausente en el escenario sintético.

7.6. Discusión

7.6.1. Restricciones como Regularización Efectiva

El hallazgo central de este capítulo es que, para las arquitecturas convolucionales, *las restricciones de no arbitraje no implican necesariamente una disyuntiva*. El barrido de lambda confirma que la CNN y la U-Net mantienen un RMSE estable desde $\lambda = 0$ hasta $\lambda = 1,0$ mientras alcanzan reducciones de severidad de 9–22×. Esto cambia la recomendación práctica de “¿cuánta exactitud se está dispuesto a sacrificar por superficies libres de arbitraje?” a “utilizar siempre una penalización de restricción para modelos convolucionales, ya que es estrictamente beneficiosa”.

El Transformer exhibe un compromiso leve con λ alto, pero con $\lambda = 0,1$ su RMSE de 0,0047 se mantiene mejor que la CNN sin restricciones, mientras la severidad cae 9,5×.

7.6.2. Conexión con *Physics-Informed Neural Networks*

El efecto de regularización observado conecta con la literatura sobre *physics-informed neural networks* (PINNs) [35], donde las leyes físicas incorporadas como penalizaciones suaves frecuentemente mejoran la generalización. La condición de convexidad butterfly es análoga: una restricción de “física financiera” que restringe el espacio de hipótesis a superficies económicamente significativas, y que mejora la generalización cuando el proceso generador de datos respeta la condición.

7.6.3. Recomendaciones Prácticas

Basándose en los resultados de este capítulo:

1. **Utilizar siempre restricciones para CNN y U-Net.** La penalización de convexidad con $\lambda = 0,05$ a $\lambda = 0,1$ es gratuita: mantiene el RMSE mientras reduce la severidad esperada en $4-6\times$.
2. **Utilizar restricciones moderadas para el Transformer.** Una penalización de $\lambda = 0,1$ provee el mejor compromiso exactitud-severidad, reduciendo la severidad en $9,5\times$ manteniendo el RMSE en $0,0047$, aún mejor que la CNN sin restricciones.
3. **Monitorear la severidad esperada, no solo la tasa de violación.** La tasa de violación sola es engañosa: un modelo con 30% de violaciones de magnitud 10^{-4} es mucho menos problemático que uno con 5% de violaciones de magnitud 10^{-2} . La severidad esperada captura esta distinción.

En conjunto, los modelos neuronales con restricciones, particularmente la U-Net con $\lambda = 0,05-0,1$ y el Transformer con $\lambda = 0,1$, combinan la exactitud de las redes neuronales con severidad esperada reducida en uno a dos órdenes de magnitud hacia el nivel de SVI, resolviendo la tensión aparente entre exactitud y consistencia económica que motivó este capítulo.

Capítulo 8

Conclusión

Este capítulo resume las contribuciones y los hallazgos principales de esta tesis, analiza sus limitaciones y propone direcciones para trabajo futuro.

8.1. Resumen de Contribuciones

Esta tesis abordó el problema de reconstruir superficies completas de volatilidad implícita a partir de observaciones incompletas de mercado, formulado como un problema de reconstrucción de señales bidimensionales. A continuación se resumen las contribuciones principales.

1. Se propuso un Transformer codificador-decodificador con codificación posicional de Fourier basada en coordenadas para la reconstrucción de superficies de volatilidad. El codificador procesa únicamente los puntos observados de la grilla, mientras que el decodificador reconstruye todas las posiciones mediante atención cruzada hacia las observaciones codificadas. Sobre datos sintéticos, esta arquitectura alcanzó un error de reconstrucción estadísticamente empatado con el mejor modelo convolucional (U-Net) a lo largo de múltiples semillas, y se degradó de la forma más gradual ante escasez creciente de datos, logrando un error $9\times$ menor que SVI con 90 % de datos faltantes. Un estudio de ablación confirmó que la codificación posicional de Fourier aporta una mejora del 18 % en RMSE respecto de embeddings aprendibles.
2. Se compararon seis arquitecturas de *deep learning* (MLP, CNN, U-Net, Transformer, FC-VAE, Conv-VAE) y el modelo de referencia paramétrico SVI, todas igualadas en aproximadamente 288.000 parámetros. Los experimentos con múltiples semillas validaron la estabilidad del ranking: el Transformer y la U-Net están estadísticamente empatados en el primer lugar (RMSE $0,0045 \pm 0,0004$ y $0,0045 \pm 0,0000$), con la CNN y el MLP por detrás. Esta configuración controlada aisló el efecto del sesgo inductivo arquitectónico: la estructura espacial (CNN, U-Net) provee mejoras del 14–17 % sobre el MLP, que carece de sesgo espacial, mientras que el cuello de botella generativo de los VAEs limita su fidelidad de reconstrucción.
3. Se formularon las condiciones de calendar spread y convexidad butterfly como términos de penalización diferenciables, integrados en la función de costo de entrenamiento. Un barrido sistemático sobre intensidades de penalización reveló que, para las arquitecturas convolucionales, estas restricciones actúan como regularizadores: la CNN mejoró su RMSE en un 6 % con $\lambda = 0,05$, mientras que la U-Net mantuvo su

exactitud prácticamente sin cambios. Ambas arquitecturas redujeron simultáneamente la severidad esperada de arbitraje en $4\text{--}6\times$. El Transformer intercambió un incremento del 9 % en RMSE por una reducción de $9,5\times$ en severidad con $\lambda = 0,1$. Estos resultados demuestran que exactitud y cumplimiento de no arbitraje no son necesariamente opuestos.

4. Todos los modelos se entrenaron con una tasa fija de 30 % de datos faltantes y se evaluaron en el rango del 10 % al 90 % sin reentrenamiento. Además del enmascaramiento aleatorio, se evaluaron los modelos bajo ausencia estructurada (combinación de enmascaramiento aleatorio y de extremos, simulando la ausencia sistemática de cotizaciones muy *out of the money*). El Transformer resultó prácticamente no afectado por el enmascaramiento estructurado (menos del 20 % de degradación), mientras que los modelos convolucionales sufrieron incrementos del 30–47 % en RMSE cuando la información de los extremos estaba ausente. Esta ventaja en robustez, más que la exactitud bruta, constituye el diferenciador más fuerte del Transformer.
5. Se validaron todos los modelos sobre 3.900 superficies reales de opciones sobre SPY abarcando el período 2008 a 2025. Todos los modelos neuronales alcanzaron un error aproximadamente $2\times$ menor que SVI. Un análisis de *transfer learning* reveló que el entrenamiento desde cero sobre datos reales superó al ajuste fino desde preentrenamiento sintético para CNN y U-Net, mientras que solo el Transformer se benefició del preentrenamiento en exactitud de reconstrucción. Sin embargo, los modelos ajustados finamente exhibieron consistentemente menores tasas de violación de butterfly, lo que indica que el preentrenamiento sintético transmite priors estructurales útiles (suavidad, convexidad) incluso cuando no mejora la exactitud.

8.2. Hallazgos Principales y Discusión

Más allá de las contribuciones individuales, de este trabajo se desprenden varios hallazgos transversales.

El Transformer iguala a los mejores modelos convolucionales en exactitud y sobresale en robustez. Los experimentos con múltiples semillas muestran que el Transformer y la U-Net están estadísticamente empatados en RMSE de reconstrucción bajo condiciones estándar ($0,0045 \pm 0,0004$ vs. $0,0045 \pm 0,0000$). La ventaja del Transformer emerge bajo condiciones desafiantes: se degrada de la forma más gradual ante escasez creciente (error $9\times$ menor que SVI con 90 % de datos faltantes), y prácticamente no se ve afectado por el enmascaramiento estructurado de extremos (menos del 20 % de degradación vs. 30–47 % para los modelos convolucionales). Además, es la única arquitectura que se beneficia de la transferencia sintético-real en exactitud de reconstrucción. Estas propiedades hacen del Transformer la arquitectura preferida cuando la robustez ante ausencia de datos variable y estructurada es importante.

La codificación posicional de Fourier es una contribución arquitectónica significativa. El estudio de ablación que reemplazó las características sinusoidales de coordenadas por embeddings posicionales aprendibles incrementó el RMSE en un 18 %. La codificación sinusoidal multi-escala de las coordenadas físicas ($\tau, \log m$) ofrece un sesgo inductivo que ayuda al mecanismo de atención a distinguir posiciones a distintas resoluciones, confirmando el valor de incorporar estructura específica del dominio en la arquitectura.

Degradación gradual ante escasez de datos. Una fortaleza distintiva del Transformer es su comportamiento a medida que aumenta la fracción de datos faltantes. Si bien todos los modelos se entrenaron con una tasa fija de 30 % de datos faltantes, el mecanismo de atención del Transformer se adapta naturalmente a cualquier cantidad de tokens observados, manteniendo el menor error a lo largo de todo el rango del 10 % al 90 %. Con 90 % de datos faltantes, alcanza un error $9\times$ menor que SVI, que queda subdeterminado cuando los cortes individuales de vencimiento tienen muy pocas observaciones. Esta degradación gradual es posiblemente el hallazgo más relevante desde el punto de vista práctico, ya que las superficies reales frecuentemente exhiben altos niveles de ausencia de datos.

Exactitud versus arbitraje no es un compromiso binario. La expectativa convencional es que imponer restricciones sobre la salida de un modelo necesariamente implica un costo en exactitud de reconstrucción. Sobre datos sintéticos, los resultados muestran que esto depende de la arquitectura: los modelos convolucionales (CNN, U-Net) mantienen o mejoran su exactitud cuando se los entrena con penalizaciones suaves de no arbitraje, ya que la restricción de convexidad se alinea con los valores de referencia analíticamente suaves de Heston y actúa como regularizador. Sobre datos reales, la misma intensidad de restricción reduce las tasas de violación sustancialmente pero incrementa el RMSE para todas las arquitecturas. Esta diferencia refleja la naturaleza de los datos: las superficies reales de SPY contienen características genuinamente no convexas (skew pronunciado a tenor corto, efectos de reportes de ganancias) que entran en conflicto con la penalización de convexidad. El efecto de regularización es por lo tanto dependiente de los datos, funcionando mejor cuando los valores de referencia respetan la restricción impuesta.

El sesgo inductivo es el diferenciador clave. El factor más importante en el rendimiento de los modelos no es la cantidad de parámetros sino el tipo de supuesto estructural que cada arquitectura realiza. Los modelos sin sesgo espacial (MLP) rinden peor y responden mal a las restricciones espaciales. Los modelos con sesgo espacial local (CNN, U-Net) aportan resultados sólidos y se benefician más de la regularización por restricciones. La atención global (Transformer) sobresale ante escasez de datos y ausencia estructurada al capturar dependencias entre vencimientos. Los modelos generativos (VAEs) producen superficies suaves pero sacrifican fidelidad de detalle fino debido a su cuello de botella de información.

La elección de arquitectura depende del caso de uso. Ningún modelo domina en todos los criterios. Todos los modelos neuronales alcanzan inferencia en menos de un milisegundo sobre GPU, pero con diferencias significativas: para aplicaciones críticas en latencia, la CNN ofrece tiempos de inferencia de 0,1 ms con exactitud competitiva, mientras que el Transformer ofrece la mejor exactitud a 0,9 ms por superficie. Para el mejor balance entre exactitud y cumplimiento de no arbitraje, la U-Net con una penalización suave ($\lambda = 0,05-0,1$) ofrece un punto de operación atractivo. Todos los modelos neuronales son al menos $15\times$ más rápidos que SVI, que requiere optimización iterativa por superficie.

El *transfer learning* presenta matices. La transferencia sintético-real no ayuda uniformemente. El modelo de Heston genera superficies con características distribucionales diferentes a las superficies reales de SPY, y la transferencia *zero-shot* (aplicar un modelo entrenado con datos sintéticos directamente a datos reales) degrada el rendimiento para todas las arquitecturas. El ajuste fino recupera y a veces supera el rendimiento del entrenamiento desde cero, pero solo para el Transformer. Para los modelos convo-

lucionales, el conjunto de datos real es suficiente por sí solo, y el sesgo inductivo ya brinda los priors estructurales que el preentrenamiento aportaría en su lugar. El beneficio consistente del preentrenamiento son menores tasas de arbitraje, sugiriendo que las propiedades estructurales (suavidad, convexidad) se transfieren entre dominios incluso cuando las propiedades distribucionales no lo hacen.

Conexión con temas más amplios. Este trabajo se sitúa en la intersección de varias áreas de investigación activas: aprendizaje con restricciones de dominio (como en las redes neuronales informadas por la física), predicción enmascarada (como en los *masked autoencoders*), y transferencia simulación-realidad (como en robótica). El hallazgo de que las restricciones de dominio pueden actuar como regularizadores en lugar de compromisos puede ser relevante más allá de las finanzas, para cualquier tarea de reconstrucción donde la salida deba satisfacer propiedades estructurales conocidas.

8.2.1. Recomendaciones Prácticas

A partir del conjunto completo de experimentos, se ofrecen las siguientes recomendaciones concretas para profesionales:

1. **Elección por defecto: U-Net con penalización butterfly suave** ($\lambda = 0,05-0,1$). Esta combinación alcanza exactitud de reconstrucción de primer nivel mientras reduce sustancialmente las violaciones de arbitraje. La restricción actúa como regularizador gratuito para arquitecturas convolucionales sobre datos subyacentes suaves.
2. **Alta escasez o ausencia estructurada: Transformer.** Cuando porciones significativas de la superficie están ausentes, particularmente en los extremos como es común en mercados reales, el mecanismo de atención global del Transformer ofrece robustez sustancialmente mayor que las alternativas convolucionales (menos del 20 % de degradación bajo enmascaramiento de extremos vs. 30–47 % para CNN/U-Net).
3. **Aplicaciones críticas en latencia: CNN.** A 0,1 ms por superficie ($9\times$ más rápida que el Transformer) y con RMSE dentro del 9 % del mejor modelo, la CNN ofrece el compromiso velocidad-exactitud más atractivo.
4. **Siempre preferir modelos neuronales sobre SVI para reconstrucción.** Todos los modelos neuronales superan a SVI por al menos $2\times$ en RMSE, con la brecha ampliándose dramáticamente ante escasez. SVI sigue siendo valioso solo cuando se requieren garantías estrictas de ausencia de arbitraje.

8.3. Limitaciones

Se deben reconocer varias limitaciones de este trabajo.

Un único activo subyacente. Todos los experimentos con datos reales utilizan opciones sobre SPY (S&P 500 ETF), que exhiben un skew característico determinado por la dinámica de índices accionarios. No resulta claro si los hallazgos se generalizan a otras clases de activos con características de superficie diferentes, como opciones de tipo de cambio (que tienden a exhibir smiles simétricos), opciones sobre commodities (que

reflejan dinámicas de backwardation y contango), u opciones sobre acciones individuales (sujetas a eventos discretos como reportes de ganancias).

Resolución de grilla fija. La grilla de 8×25 utilizada a lo largo de este trabajo es relativamente gruesa. Grillas de mayor resolución aproximarían mejor superficies continuas pero también cambiarían los compromisos computacionales: el costo cuadrático de atención del Transformer crecería más rápido que el costo lineal de la CNN, potencialmente alterando el ranking de rendimiento. Adicionalmente, la tasa de violación de butterfly de aproximadamente 8,6 % observada en los valores de referencia sintéticos es un artefacto directo de las aproximaciones por diferencias finitas sobre esta grilla gruesa.

Sin modelado temporal. Cada superficie de volatilidad se trata como una muestra independiente. En la práctica, las superficies evolucionan suavemente a lo largo del tiempo, y la superficie de hoy es altamente informativa sobre la de mañana. Explotar esta estructura temporal podría mejorar tanto la exactitud como la suavidad.

Validación limitada con múltiples semillas. Se validó la estabilidad del ranking con tres semillas aleatorias para los cuatro mejores modelos. Si bien los resultados son consistentes (el Transformer y la U-Net están estadísticamente empatados, con CNN y MLP claramente por detrás), un mayor número de semillas proveería intervalos de confianza más estrechos y permitiría tests formales de significancia estadística.

Enmascaramiento de entrenamiento limitado. Todos los modelos se entrenan exclusivamente con enmascaramiento aleatorio uniforme al 30 %. Si bien se evalúan bajo patrones tanto aleatorios como estructurados (aleatorio + extremos), el entrenamiento no incluye enmascaramiento estructurado ni múltiples tasas simultáneamente. Entrenar con estrategias de enmascaramiento mixto podría mejorar la robustez y alterar los rankings relativos.

Solo restricciones blandas. Se exploran únicamente restricciones basadas en penalizaciones blandas. Enfoques alternativos como restricciones garantizadas arquitectónicamente (por ejemplo, redes neuronales input-convex para butterfly, capas monótonas para calendar) no se evaluaron y podrían ofrecer garantías estrictas sin compromisos en exactitud.

Ajuste limitado de hiperparámetros. Para asegurar una comparación justa, se mantuvieron deliberadamente fijos los hiperparámetros de entrenamiento entre arquitecturas (con la excepción de la tasa de aprendizaje para el Transformer). Un ajuste individual por arquitectura podría reducir las brechas de rendimiento entre modelos.

Generador de datos sintéticos. Todos los experimentos sintéticos utilizan el modelo de Heston, que produce superficies suaves y paramétricamente determinadas. Un generador diferente, como modelos de volatilidad rough, volatilidad local estocástica o remuestreo empírico, podría producir superficies con características distintas (rasgos más abruptos, regiones no convexas) que favorezcan diferentes arquitecturas. De forma análoga, combinar un generador más realista con estrategias de enmascaramiento que imiten la liquidez del mercado durante el entrenamiento podría reducir conjuntamente la brecha entre el rendimiento sintético y real.

8.4. Trabajo Futuro

De este trabajo surgen varias direcciones para investigación futura.

Modelos temporales. Incorporar secuencias de superficies mediante arquitecturas recurrentes o Transformers temporales podría explotar la fuerte autocorrelación de las

superficies de volatilidad a lo largo del tiempo. La superficie de ayer podría servir como un prior informativo para la reconstrucción de hoy, mejorando potencialmente tanto la exactitud como la suavidad.

Grillas de mayor resolución e irregulares. Evaluar sobre grillas más finas (por ejemplo, 20×50) permitiría verificar la escalabilidad y podría modificar el compromiso CNN-Transformer. Extender el enfoque a grillas irregulares, donde las observaciones no están alineadas a una grilla fija, reflejaría mejor los datos reales de mercado y podría favorecer aún más la capacidad del Transformer de procesar conjuntos no estructurados de tokens.

Restricciones estrictas. Garantizar arquitectónicamente las propiedades de no arbitraje, por ejemplo mediante redes neuronales input-convex para convexidad butterfly o capas de salida monótonas para condiciones de calendar spread, aportaría garantías estrictas que las penalizaciones blandas no pueden ofrecer. Comparar estos enfoques con el método basado en penalizaciones estudiado aquí es un paso natural a seguir.

Patrones de ausencia realistas. Entrenar con enmascaramiento estructurado que imite los patrones reales de liquidez del mercado (más datos faltantes en los extremos y tenores cortos, menos cerca de at-the-money) podría reducir la brecha entre el rendimiento sobre datos sintéticos y reales.

Generalización a múltiples activos. Evaluar sobre opciones de tipo de cambio, commodities y acciones individuales permitiría verificar si las conclusiones arquitectónicas derivadas de opciones sobre índices accionarios se mantienen en clases de activos con características de superficie fundamentalmente diferentes.

Métodos de ensamble e híbridos. Combinar las fortalezas de distintas arquitecturas, por ejemplo utilizando una CNN para una estimación inicial rápida y un Transformer para refinamiento en regiones con datos escasos, podría ofrecer mejores compromisos exactitud-latencia que cualquier modelo individual.

Mejoras generativas. Reemplazar el decodificador del VAE con modelos generativos más expresivos como modelos de difusión podría abordar el cuello de botella de información que limita la fidelidad de reconstrucción de los VAEs, manteniendo los beneficios de la optimización en el espacio latente durante la inferencia.

Despliegue en producción. Empaquetar los modelos de mejor rendimiento como servicios en tiempo real para mesas de operaciones y compararlos con herramientas comerciales de ajuste de superficies de volatilidad brindaría una validación práctica del enfoque más allá de las métricas académicas.

En síntesis, esta tesis demuestra que la reconstrucción de superficies de volatilidad a partir de observaciones incompletas es un problema donde la elección de la arquitectura y la incorporación de conocimiento del dominio importan más que la escala del modelo. El mecanismo de atención no domina universalmente, pero ofrece la mayor robustez ante las condiciones que caracterizan a los mercados reales: datos escasos, ausencia estructurada y distribuciones que cambian con el tiempo. Las restricciones de no arbitraje, lejos de ser un costo inevitable, pueden actuar como regularizadores que mejoran simultáneamente la exactitud y la consistencia económica. Estos hallazgos sugieren que el paradigma de combinar arquitecturas flexibles con restricciones específicas del dominio puede ser útil más allá del dominio financiero, en cualquier problema de reconstrucción donde la salida deba respetar propiedades estructurales conocidas.

Bibliografía

- [1] Damien Ackerer, Natasa Tagasovska, and Thibault Vatter. Deep smoothing of the implied volatility surface. *arXiv preprint arXiv:2004.11015*, 2020.
- [2] Hansjörg Albrecher, Philipp Mayer, Wim Schoutens, and Jurgen Tistaert. The little Heston trap. *Wilmott Magazine*, pages 83–92, 2007.
- [3] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [4] Christian Bayer and Benjamin Stemper. Deep calibration of rough stochastic volatility models. *Quantitative Finance*, 19(1):71–86, 2019.
- [5] Maxime Bergeron, Nicholas Fung, John Hull, Zissis Poulos, and Andreas Veneris. Variational autoencoders: A hands-off approach to volatility. *The Journal of Financial Data Science*, 4(2):125–138, 2022.
- [6] Marcelo Bertalmío, Guillermo Sapiro, Vicent Caselles, and Coloma Ballester. Image inpainting. In *ACM SIGGRAPH*, pages 417–424, 2000.
- [7] Fischer Black. The pricing of commodity contracts. *Journal of Financial Economics*, 3(1-2):167–179, 1976.
- [8] Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.
- [9] Douglas T. Breeden and Robert H. Litzenberger. Prices of state-contingent claims implicit in option prices. *Journal of Business*, 51(4):621–651, 1978.
- [10] Richard H. Byrd, Peihuang Lu, Jorge Nocedal, and Ciyu Zhu. A limited memory algorithm for bound constrained optimization. *SIAM Journal on Scientific Computing*, 16(5):1190–1208, 1995.
- [11] Andrew A. Christie. The stochastic behavior of common stock variances: Value, leverage and interest rate effects. *Journal of Financial Economics*, 10(4):407–432, 1982.
- [12] Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. Fast and accurate deep network learning by exponential linear units (ELUs). In *International Conference on Learning Representations (ICLR)*, 2016.
- [13] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pages 4171–4186, 2019.

- [14] Tianyu Du, Luca Zhang, Aaron Harlap, and Mert R. Sabuncu. ReMasker: Imputing tabular data with masked autoencoding. In *International Conference on Learning Representations (ICLR)*, 2024.
- [15] Philipp Dubach. Historic options dataset: Spy, iwm, and qqq options 2008-2025, 2025.
- [16] Jim Gatheral. *The Volatility Surface*. Wiley, 2011.
- [17] Jim Gatheral and Antoine Jacquier. Arbitrage-free SVI volatility surfaces. *Quantitative Finance*, 14(1):59–71, 2014.
- [18] J. Gil-Pelaez. Note on the inversion theorem. *Biometrika*, 38(3-4):481–482, 1951.
- [19] Patrick S. Hagan, Deep Kumar, Andrew S. Lesniewski, and Diana E. Woodward. Managing smile risk. *Wilmott Magazine*, pages 84–108, 2002.
- [20] J. Michael Harrison and David M. Kreps. Martingales and arbitrage in multiperiod securities markets. *Journal of Economic Theory*, 20(3):381–408, 1979.
- [21] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16000–16009, 2022.
- [22] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.
- [23] Steven L. Heston. A closed-form solution for options with stochastic volatility with applications to bond and currency options. *The Review of Financial Studies*, 6(2):327–343, 1993.
- [24] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.
- [25] Blanka Horvath, Aitor Muguruza, and Mehdi Tomas. Deep learning volatility: A deep neural network perspective on pricing and calibration in (rough) volatility models. *Quantitative Finance*, 21(1):11–27, 2021.
- [26] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456, 2015.
- [27] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2015. Published at ICLR 2015.
- [28] Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. *arXiv preprint arXiv:1312.6114*, 2014.
- [29] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

- [30] Robert C. Merton. Theory of rational option pricing. *The Bell Journal of Economics and Management Science*, 4(1):141–183, 1973.
- [31] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pages 807–814, 2010.
- [32] Brian X. Ning, Sebastian Jaimungal, Xiaorong Zhang, and Maxime Bergeron. Arbitrage-free implied volatility surface generation with variational autoencoders. *SIAM Journal on Financial Mathematics*, 14(4):1004–1027, 2023.
- [33] Bienvenue Feugang Nteumagné, Hermann Azemtsa Donfack, and Celestin Wafo Soh. Variational autoencoders for completing the volatility surfaces. *Journal of Risk and Financial Management*, 18(5):239, 2025.
- [34] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.
- [35] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- [36] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241. Springer, 2015.
- [37] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.
- [38] Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 7537–7547, 2020.
- [39] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [40] Qiming Zhang, Yun Wang, and Zhuoran Ye. Meta-learning neural process for implied volatility surfaces with SABR-induced priors. *arXiv preprint arXiv:2509.11928*, 2025.